

第1章: 開発環境のセットアップ

この章では、BON-LOG（盆栽愛好家のためのSNS）を開発するために必要なツールをすべてインストールし、実際にプロジェクトが動く状態までセットアップします。

プログラミングの学習において、最初のハードルは「環境構築」です。ここでつまづいてしまう方が非常に多いので、この章ではできる限り丁寧に、一つ一つの手順を解説していきます。焦らず、順番に進めていきましょう。

1.0 実習手順の進め方と手順マップ

手順に沿って進めると、**どの手順で何をインストールし、何を確認すればよいか** が分かります。形式の説明は [チュートリアルの進め方](#) を参照してください。

手順	主な対象	完了時に確認すること
ターミナル基本	—	コマンドが実行できる
Node.js インストール	—	<code>node -v</code> / <code>npm -v</code> が表示される
VS Code・拡張機能	—	エディタでコードが編集できる
Git・GitHub	—	<code>git --version</code> が通り、リポジトリが使える
Docker Desktop	—	<code>docker --version</code> が通り、コンテナが起動する
Next.js プロジェクト作成	プロジェクトフォルダ	<code>npm run dev</code> でブラウザに表示される
PostgreSQL・Prisma	<code>prisma/schema.prisma</code> , <code>.env</code>	<code>npx prisma generate</code> / <code>db push</code> が成功する

各手順で **やること・このあと変わること・確認方法** を確認しながら進めてください。

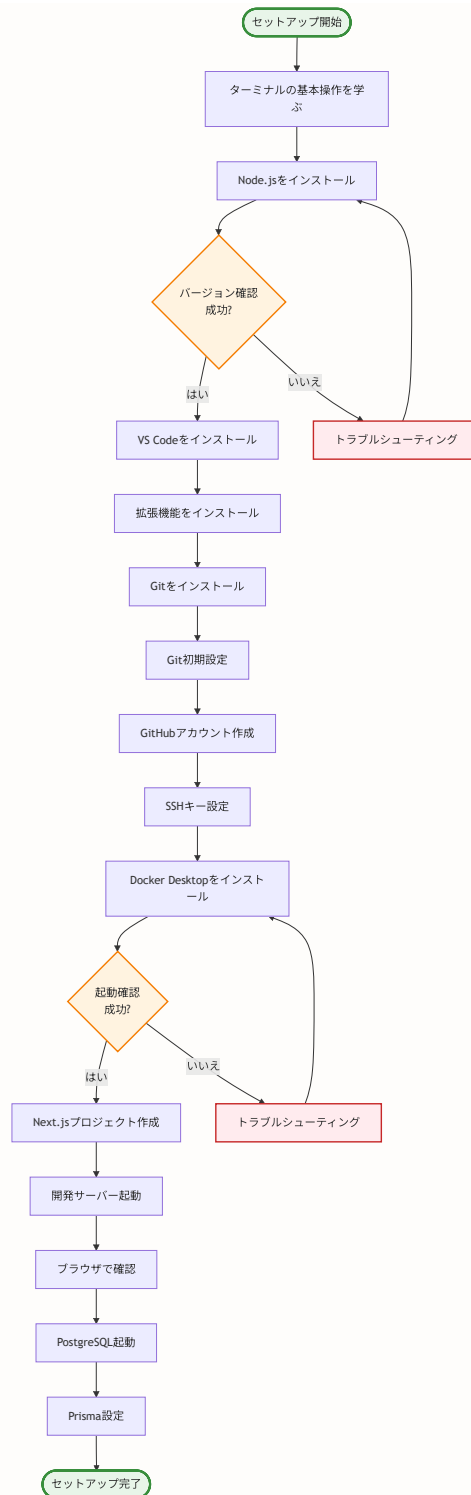
この章のゴール

- 開発に必要な6つのツールをすべてインストールする
- ターミナル（コマンドプロンプト）の基本操作ができるようになる
- Next.jsプロジェクトを作成し、ブラウザで表示できる状態にする
- PostgreSQLデータベースをDockerで起動できる

この章の所要時間目安: 60～90分（ネットワーク速度により前後します）

セットアップワークフロー全体像

この章で行う作業の流れを図で表すと以下ようになります。順番に進めていきましょう。



この章で出てくる専門用語（五十音順）

初めて目にする用語が多いかもしれませんが、心配しないでください。各用語は本文中で詳しく解説します。ここでは「こんな言葉が出てくるんだな」とざっと目を通すだけで十分です。

用語	読み方	ひとこと言うと
CLI	シーエルアイ	文字を入力してコンピュータを操作する方法
Docker	ドッカー	アプリを隔離された環境（コンテナ）で動かすツール
ESLint	イーエスリント	JavaScriptやTypeScriptのコード品質をチェックするツール
Git	ギット	コードの変更履歴を管理するツール
GitHub	ギットハブ	Gitで管理したコードをクラウドに保存・共有するサービス
GUI	ジーユーアイ	マウスやタッチでコンピュータを操作する方法
IntelliSense	インテリセンス	コード入力中に候補を提案してくれる機能
JavaScript	ジャバスクリプト	Webサイトに動きをつけるプログラミング言語
LTS	エルティーエス	Long Term Supportの略。長期間サポートされる安定版
Next.js	ネクストジェイエス	Reactベースのフルスタック型Webアプリケーションフレームワーク
Node.js	ノードジェイエス	JavaScriptをサーバーサイドで実行できるようにするランタイム環境
npm	エヌピーエム	Node Package Manager。Node.jsのパッケージ管理ツール
npx	エヌピーエックス	npmのパッケージを一時的にダウンロードして実行するツール
PATH	パス	コマンドの実行ファイルを探す場所のリスト（環境変数の一つ）
PostgreSQL	ポストグレスキューエル	高機能なオープンソースのリレーショナルデータベース
Prettier	プリティア	コードを自動的にきれいに整形するツール
Prisma	プリズマ	TypeScript対応のデータベースORM（操作を簡単にするツール）
SSH	エスエスエイチ	暗号化された安全な通信方式
Tailwind CSS	テイルウィンドシーエスエス	ユーティリティクラスでスタイルを記述するCSSフレームワーク

TypeScript	タイプスクリプト	JavaScriptに型の概念を加えた言語
VS Code	バイエスコード	Microsoft製の無料コードエディタ
WSL2	ダブルユーエスエルツー	Windows上でLinuxを動かすための仕組み
YAML	ヤムル	設定ファイルによく使われるデータ記述形式
コンテナ	コンテナ	アプリとその実行環境をまとめてパッケージ化したもの
フレームワーク	フレームワーク	アプリ開発の土台となるソフトウェアの枠組み
ランタイム	ランタイム	プログラムを実行するための環境
リポジトリ	リポジトリ	ソースコードとその変更履歴の保管場所

1.1 必要なツール一覧

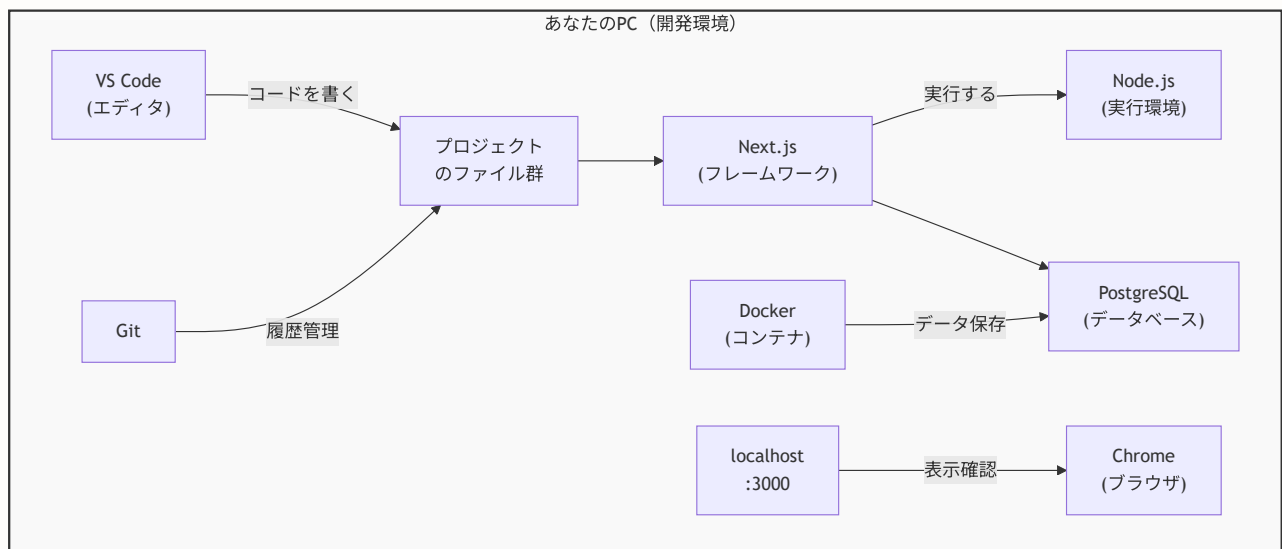
このセクションで学ぶこと

- BON-LOGの開発に必要なツールの全体像
- 各ツールがどんな役割を果たすのか

BON-LOGを作るためには、以下の6つのツールが必要です。それぞれのツールがどんな役割を果たすのか、まず全体像を掴みましょう。

ツール	用途	バージョン	なぜ必要か
Node.js（ノードジェイエス）	JavaScript（ジャバascript）：Webで最も使われるプログラミング言語）の実行環境	v20以上	Next.js（ネクストジェイエス：BON-LOGのフレームワーク）を動かすため
npm（エヌピーエム：Node Package Manager）	パッケージ管理ツール。ライブラリのインストールやプロジェクト管理に使う	Node.jsに同梱	他の人が作ったライブラリ（パッケージ）をインストールするため
VS Code（ブイエスコード：Visual Studio Code）	コードエディタ（プログラムを書くための専用テキストエディタ）	最新版	コードを効率よく書くため
Git（ギット）	バージョン管理システム（コードの変更履歴を記録するツール）	最新版	コードの変更履歴を記録し、チーム開発を可能にするため
Docker Desktop（ドッカーデスクトップ）	コンテナ実行環境（アプリを隔離された環境で動かすツール）	最新版	PostgreSQL（ポストグレスキューエル：データベース）をローカルで動かすため
Chrome（クローム）	Webブラウザ	最新版	開発者ツール（DevTools：ブラウザに内蔵されたデバッグ用の機能群）でデバッグするため

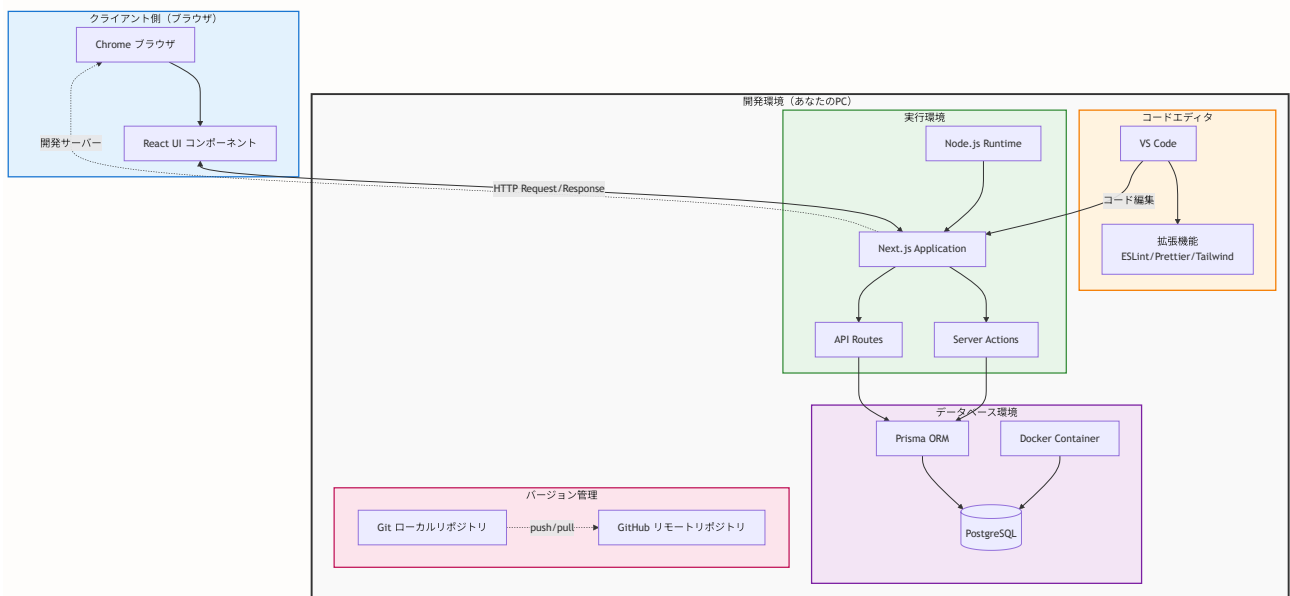
これらのツールの関係を図で表すと以下のようになります。



ここがポイント！ 全部を一度に理解する必要はありません。まずはインストールして「動く状態」にすることが大切です。各ツールの詳しい使い方は、開発を進めながら自然と身についていきます。

開発環境のアーキテクチャ

実際の開発では、これらのツールがどのように連携するのかを理解することが重要です。以下の図は、BON-LOGの開発環境全体のアーキテクチャを示しています。



理解度チェック

- ☐ 上の表の6つのツールをすべて言えますか？
- ☐ Node.jsが何のために必要か、一言で説明できますか？
- ☐ なぜDockerが必要なのか、理由を説明できますか？

技術選定の背景 -- なぜこれらのツールを選んだのか？

プログラミングの世界には、似た機能を持つツールがたくさんあります。ここでは「なぜBON-LOGではこれらを選んだのか？」「他にどんな選択肢があるのか？」を解説します。技術選定の理由を理解すると、将来自分でプロジェクトを始めるときにも役立ちます。

注意: 以下の比較は2025年時点の情報です。技術の世界は変化が速いため、最新の情報は公式サイトやコミュニティを確認してください。

Node.js vs 他のJavaScriptランタイム

JavaScriptをサーバーサイドで実行できるランタイム（実行環境）は、Node.js以外にもあります。

ランタイム	特徴	メリット	デメリット
Node.js	2009年登場。最も歴史が長い	パッケージ数が圧倒的（200万以上）、情報が豊富、企業での採用実績多数	起動がやや遅い、セキュリティはデフォルトで緩め
Deno （ディーノ）	2020年登場。Node.jsの作者が開発	TypeScriptをそのまま実行可能、セキュリティがデフォルトで厳格	npm互換が完全ではない、情報がまだ少ない
Bun （バン）	2022年登場。速度を重視	起動速度・実行速度がNode.jsの数倍速い、npm互換性が高い	比較的新しくエコシステムが発展途上、一部のnpmパッケージで互換性の問題あり

BON-LOGでNode.jsを選んだ理由:

- エコシステムの成熟度:** npm（パッケージ管理ツール）には200万以上のパッケージがあり、必要なライブラリがほぼ確実に見つかる
- 情報量の多さ:** 問題が発生した際にGoogle検索やStack Overflowで解決策が見つかりやすい
- Next.jsとの親和性:** Next.jsはNode.jsを前提に設計されており、最も安定して動作する
- 学習リソースの豊富さ:** 日本語の書籍、記事、動画教材が最も多い

将来的な展望: BunやDenoも急速に成長しており、将来的にはNode.jsの代わりに使われる場面が増えるかもしれません。ただし、Node.jsで学んだ知識はDeno/Bunでも活かせるため、まずはNode.jsで基礎を学ぶのがおすすめです。

パッケージマネージャーの選択 -- npm vs yarn vs pnpm

Node.jsのパッケージマネージャー（ライブラリを管理するツール）にも複数の選択肢があります。

ツール	特徴	メリット	デメリット
npm	Node.jsに同梱。標準のパッケージマネージャー	インストール不要、最も広く使われている、公式ドキュメントが豊富	yarn/pnpmよりやや遅い場合がある
yarn (ヤーン)	Meta (旧Facebook) が開発	npm より高速、Plug'n'Play (PnP) モードで高速化	追加インストールが必要、PnP モードで互換性の問題があることがある
pnpm (ピーエヌピーエム)	ディスク効率を重視した設計	ディスク容量を大幅に節約、インストールが高速	追加インストールが必要、一部のツールとの互換性に注意

BON-LOGでnpmを選んだ理由:

1. **追加インストール不要:** Node.jsをインストールすればそのまま使える
2. **初心者に分かりやすい:** チュートリアルや書籍のほとんどがnpmを前提に書かれている
3. **互換性の安定:** npmはNode.jsの公式ツールであり、互換性の問題が最も少ない
4. **十分な性能:** 近年のnpmはパフォーマンスが大幅に改善されており、個人プロジェクトには十分

補足: プロジェクトの規模が大きくなったり、モノレポ（1つのリポジトリで複数のプロジェクトを管理する構成）を採用する場合は、pnpmの採用を検討する価値があります。

エディタの選択 -- VS Code vs 他のエディタ

コードエディタ（IDE: 統合開発環境とも呼ばれる）も多くの選択肢があります。

エディタ	特徴	メリット	デメリット
VS Code	Microsoft製。無料	無料、軽量、拡張機能が豊富、TypeScriptとの統合が優秀	プラグインを入れすぎると重くなることもある
WebStorm（ウェブストーム）	JetBrains製。有料	高機能、リファクタリングが強力、設定不要で多くの機能が使える	有料（年額約15,000円）、メモリ使用量が多い
Vim/Neovim（ビム/ネオビム）	ターミナル上で動作するエディタ	非常に軽量、高速、SSH経由でサーバー上でも使える	学習コストが非常に高い（独特な操作方法）
Cursor（カーソル）	AI機能を統合したエディタ	AIによるコード生成が強力、VS Code互換	一部有料、AIへの依存度が高い

BON-LOGでVS Codeを選んだ理由:

1. **無料:** 費用をかけずに始められる
2. **拡張機能の充実:** ESLint、Prettier、Tailwind CSSなどの拡張機能が非常に充実している
3. **TypeScript統合:** VS CodeはTypeScriptと同じMicrosoftが開発しており、TypeScriptのサポートが最も手厚い
4. **コミュニティの大きさ:** 世界で最も使われているエディタであり、困ったときの情報が見つけやすい
5. **ターミナル内蔵:** エディタ内でターミナルを開けるため、画面を切り替える手間が減る

補足: 慣れてきたらCursorを試してみるのもおすすめです。VS Codeの拡張機能がそのまま使え、AIがコードの提案をしてくれるため、開発効率が上がります。

データベースの選択 -- PostgreSQL vs 他のデータベース

データベース（DB: データを保存・管理するソフトウェア）にも多くの選択肢があります。

データベース	種類	メリット	デメリット
PostgreSQL	リレーショナルDB (テーブル形式)	高機能、JSON対応、全文 検索、Supabase互換	セットアップが少し複雑
MySQL (マイエスキューエル)	リレーショナルDB	最も広く使われている、情 報が豊富	PostgreSQLより機能が少な い、JSON対応が弱い
SQLite (エスキュー ライト)	ファイルベースDB	インストール不要、ファイ ル1つで完結	同時アクセスに弱い、大規模デ ータに不向き
MongoDB (モンゴデ ィービー)	ドキュメントDB (JSON形式)	スキーマが柔軟、スケール しやすい	データの整合性を保つのが難し い、複雑なクエリが苦手

BON-LOGでPostgreSQLを選んだ理由:

- Supabaseとの互換性:** 本番環境で使うSupabase（スーパーベース：PostgreSQLベースのBaaS）との完全な互換性がある
- JSON対応:** SNSのメタデータ（投稿の設定や通知の詳細など）をJSONBカラムで柔軟に保存できる
- 全文検索:** pg_trgm拡張を使って日本語の投稿を検索できる（追加のサービスなしで対応可能）
- Prismaとの相性:** PrismaはPostgreSQLの機能を最も広くサポートしている
- 高い信頼性:** 30年以上の開発歴を持ち、大規模なWebサービスでの利用実績が豊富

コンテナの選択 -- Docker vs 他の方法

データベースなどの開発ツールをローカルPCにセットアップする方法には、いくつかの選択肢があります。

方法	特徴	メリット	デメリット
Docker	コンテナで隔離して実行	環境が汚れない、チーム で同じ環境を再現、簡単 に削除・作り直し可能	Docker Desktop自体がメモ リを消費（1～2GB）
ローカルイン ストール	PCに直接インストール	追加ツール不要、メモリ 消費が少ない	アンインストールが面倒、 バージョン管理が手動、環 境差異が生じやすい
Vagrant (ベ イグラント)	仮想マシン (VM) で実行	完全に隔離された環境	Dockerより重い（数GB）、 起動が遅い
クラウドDB	Supabase/PlanetScale等のクラ ウドサービスを直接使用	インストール不要、常に 最新	ネットワーク接続必須、レ イテンシー（遅延）があ る、無料枠に制限あり

BON-LOGでDockerを選んだ理由:

1. **環境を汚さない**: PostgreSQLが不要になったらコンテナを削除するだけ。PCに残骸が残らない
2. **再現性**: `docker-compose.yml` ファイル1つで、誰でも同じデータベース環境を構築できる
3. **バージョン指定が簡単**: `postgres:16-alpine` と書くだけで、特定バージョンのPostgreSQLが起動する
4. **本番環境との近さ**: Dockerで構築した環境は、クラウドの本番環境に近い動作をする
5. **学習価値**: Dockerはモダンな開発で広く使われているため、習得しておくに役立つ

1.2 ターミナル（コマンドプロンプト）の基礎

このセクションで学ぶこと

- ターミナルとは何か、なぜ使うのか
- ターミナルの開き方
- 最低限必要な基本コマンド
- 「パス」の概念

プログラミングでは、マウスでクリックする操作（GUI: Graphical User Interface、グラフィカルユーザーインターフェース）ではなく、**文字を入力してコンピュータに指示を出す**操作（CLI: Command Line Interface、コマンドラインインターフェース）を多用します。この文字入力の画面のことを「ターミナル」または「コマンドプロンプト」「コマンドライン」「シェル (Shell)」と呼びます。

用語の違い（参考）

- **ターミナル**: 文字入力を行う画面・ウィンドウそのもの
- **シェル**: ターミナル上で動作する、コマンドを解釈するプログラム（bash、zsh、PowerShell等）
- **コマンドプロンプト**: Windows特有のシェルの名前。「ターミナル」とほぼ同じ意味で使われることも多い
- **コンソール**: ターミナルと同義で使われることが多い（文脈による）

厳密には違いがありますが、初心者の段階ではすべて「文字入力でPCを操作する画面」と理解しておけば十分です。

コラム: なぜターミナルを使うのか? 「マウスで操作した方が簡単じゃない?」と思うかもしれませんが。しかしターミナルには以下のメリットがあります。

- **正確に再現できる:** コマンドをコピペすれば、誰でも同じ操作ができる
- **自動化しやすい:** コマンドを組み合わせ、複雑な作業を一瞬で実行できる
- **プログラマーの共通言語:** 世界中の開発者がターミナルで作業している
- **遠隔操作が可能:** サーバーの操作はターミナル経由で行うことが多い

ターミナルの開き方

Windowsの場合

方法1: スタートメニューから

1. キーボードの **Windows** キーを押す（画面左下のWindowsマークのキー）
2. 「PowerShell」または「ターミナル」と入力
3. 「Windows PowerShell」または「Windows Terminal」をクリック

方法2: VS Codeから（推奨）

1. VS Codeを開く
2. 画面上部のメニューから「ターミナル」→「新しいターミナル」をクリック
3. または、キーボードで **Ctrl + `**（Ctrlキーを押しながらバッククォートキー）を押す

注意! Windowsには「コマンドプロンプト (cmd)」と「PowerShell」の2種類があります。このチュートリアルでは**PowerShell**または**Git Bash**を使います。もしコマンドプロンプト（画面の上部に **cmd** と表示される黒い画面）が開いてしまったら、閉じてPowerShellを開き直してください。

PowerShell・Command Prompt・Git Bashの違い

- **PowerShell:** Windows標準のシェル。コマンドが独自形式（**ls** の代わりに **Get-ChildItem** 等）
- **Command Prompt (cmd):** 古いWindows標準。機能が限られる
- **Git Bash:** Git for Windowsに付属。Linux/Macと同じコマンドが使える（**推奨**）

本チュートリアルでは**Git Bash**を前提とします。

Macの場合

方法1: Spotlightから

1. `Cmd + Space` を押してSpotlightを開く
2. 「ターミナル」と入力
3. 「ターミナル.app」をクリック

方法2: VS Codeから（推奨）

1. VS Codeを開く
2. 画面上部のメニューから「ターミナル」→「新しいターミナル」をクリック
3. または `Cmd + `` を押す

ターミナルの画面の見方

ターミナルを開くと、以下のような表示が出ます。

```
# Windowsの場合
PS C:\Users\あなたの名前>

# Macの場合
あなたの名前@MacBook ~ %
```

この `C:\Users\あなたの名前` や `~` の部分は、「今自分がいる場所（ディレクトリ）」を表しています。 `>` や `%` や `$` はプロンプト（入力待ちの記号）と呼ばれ、ここにコマンドを入力します。

注意！ このチュートリアルのコマンド例で先頭に `$` や `>` がついている場合、それはプロンプトの記号なので入力しないでください。例えば `$ node --version` と書いてあったら、入力するのは `node --version` の部分だけです。

「パス」とは何か

コンピュータ上のファイルやフォルダの「住所」のことを**パス**と呼びます。

```
# Windowsのパスの例
C:\Users\yuya\Desktop\Bonsai\bonsai-sns-project

# Macのパスの例
/Users/yuya/Desktop/Bonsai/bonsai-sns-project
```

パスには2種類あります。

絶対パス（フルパス）：ルート（一番上）からの完全な住所

```
C:\Users\yuya\Desktop\Bonsai\bonsai-sns-project\app\page.tsx
```

相対パス：今いる場所からの相対的な住所

```
# 今 bonsai-sns-project にいるとき
./app/page.tsx      # 同じフォルダ内の app フォルダの中の page.tsx
../                 # 一つ上のフォルダ（Bonsai フォルダ）
```

パスのイメージ図（Windowsの場合）：

```
C:\ (ルート = 一番上)
├─ Users\
│   └─ yuya\
│       └─ Desktop\
│           └─ Bonsai\
│               └─ bonsai-sns-project\ ← ここがプロジェクトのルート
│                   ├── app\
│                   │   ├── page.tsx
│                   │   └─ layout.tsx
│                   ├── components\
│                   ├── lib\
│                   ├── package.json
│                   └─ .env.local
```

ここがポイント！ ターミナルでは常に「今どのフォルダにいるか」を意識することが大切です。正しい場所にいないと、コマンドが「ファイルが見つかりません」とエラーになります。

カレントディレクトリ（現在地）とは？ ターミナルは常に「今いるフォルダ」を基準に動作します。ファイルエクスプローラーでフォルダを開いているのと同じ感覚です。 `pwd`（Print Working Directory）コマンドで「今どこにいるか」を確認できます。

基本コマンド一覧

以下のコマンドは今後の開発で頻繁に使います。すべて覚える必要はありませんが、「こういうコマンドがある」ということだけ知っておいてください。

コマンド	意味	使用例	説明
<code>cd</code>	ディレクトリ移動	<code>cd Desktop</code>	Desktopフォルダに移動
<code>cd ..</code>	一つ上に移動	<code>cd ..</code>	親フォルダに移動
<code>ls</code> (Mac) / <code>dir</code> (Windows)	ファイル一覧表示	<code>ls</code>	今いるフォルダの中身を表示
<code>pwd</code> (Mac) / <code>cd</code> (Windows)	現在地を表示	<code>pwd</code>	今いるフォルダのパスを表示
<code>mkdir</code>	フォルダを作成	<code>mkdir myapp</code>	myappフォルダを作成
<code>clear</code> (Mac) / <code>cls</code> (Windows)	画面をクリア	<code>clear</code>	ターミナルの表示を消す
<code>Ctrl + C</code>	実行中のプログラムを停止	—	開発サーバーを止める時に使用

実際に試してみましょう。各コマンドの実行結果例も併せて記載します。

```
# 今いる場所を確認 (Macの場合)
pwd
# 実行結果の例:
# /Users/yuya

# 今いる場所を確認 (Windowsの場合)
cd
# 実行結果の例:
# C:\Users\yuya
```

```
# デスクトップに移動
cd Desktop

# 移動後に今いる場所を確認してみましょう
# Macの場合:
pwd
# 実行結果の例:
# /Users/yuya/Desktop

# Windowsの場合:
cd
# 実行結果の例:
# C:\Users\yuya\Desktop
```

```
# 今いるフォルダの中身を表示 (Mac / Git Bash)
ls
# 実行結果の例:
# Bonsai/    Documents/  Downloads/  Pictures/

# 今いるフォルダの中身を表示 (Windows PowerShell)
dir
# 実行結果の例:
#      Directory: C:\Users\yuya\Desktop
#
# Mode                LastWriteTime         Length Name
# ----                -
# d-----           2025/01/15   10:30             Bonsai
# d-----           2025/01/10   09:00          Documents
```

```
# 一つ上のフォルダに戻る
cd ..
# (一つ上のフォルダに移動する。この場合、Desktop から Users/yuya に戻る)
```

初心者向けポイント: コマンドの実行の仕方

1. コマンドを入力する（コピー&ペーストでもOK）
2. Enterキーを押して実行する
3. 結果が表示される
4. 次のプロンプト（`$` や `>` や `%`）が表示されたら、次のコマンドを入力できる状態

何も表示されずにプロンプトだけが表示された場合は、コマンドが正常に完了したことを意味します。
（`cd` コマンドなど、結果を表示しないコマンドもあります）

よくあるトラブル: 「cd: no such file or directory」というエラーが出る 「指定したフォルダが見つからない」というエラーです。

- スペルミスがないか確認してください
- `ls`（Mac）または `dir`（Windows）で、移動先のフォルダが存在するか確認してください
- フォルダ名にスペースが含まれる場合は、引用符で囲んでください: `cd "My Documents"`

理解度チェック

- ☐ ターミナルを開くことができますか？
- ☐ `cd` コマンドで別のフォルダに移動できますか？
- ☐ 「絶対パス」と「相対パス」の違いを説明できますか？
- ☐ 現在いるフォルダの中身を一覧表示できますか？

1.3 Node.jsのインストール

このセクションで学ぶこと

- Node.jsとは何か
- Node.jsのインストール手順
- npmとは何か
- バージョンの確認方法

Node.jsとは？

Node.js（ノード・ジェイエス）は、**JavaScriptをブラウザの外で実行するための環境**です。

もともとJavaScriptはWebブラウザの中だけで動く言語でした。しかしNode.jsが登場したことで、サーバーサイド（バックエンド）の処理もJavaScriptで書けるようになりました。

JavaScriptの動作環境:

ブラウザ（従来のJS）	Node.js（サーバー側）
ボタンクリック	サーバーの処理
画面の描画	ファイルの操作
アニメーション	データベース接続
フォーム操作	APIの提供

BON-LOGでは、Next.js（Webアプリケーションのフレームワーク）がNode.js上で動作します。つまり、Node.jsは「Next.jsの土台」のようなものです。

コラム: LTS版とCurrent版 Node.jsのダウンロードページには2つのバージョンがあります。

- **LTS（Long Term Support）**：安定版。長期間サポートされる。**こちらを選びましょう。**
- **Current**: 最新機能が入っているが、安定性が保証されない。

Windowsでのインストール手順

1. ブラウザで <https://nodejs.org/> にアクセスします
2. 画面の中央に2つの緑色のボタンが表示されます。**左側のLTS版（推奨版）**をクリックしてダウンロードします
 - ボタンには「XX.XX.X LTS（推奨）」のように表示されています
 - 数字の部分は時期によって異なりますが、20以上であればOKです
3. ダウンロードされた **.msi** ファイル（例: **node-v20.11.0-x64.msi**）をダブルクリックして開きます
4. インストーラが起動します。以下の手順で進めてください。
 - 「Welcome to the Node.js Setup Wizard」画面 → 「Next」をクリック

- 「End-User License Agreement」画面 → チェックボックスにチェック → 「Next」をクリック
- 「Destination Folder」画面 → **そのまま変更せず** → 「Next」をクリック
- 「Custom Setup」画面 → **そのまま変更せず** → 「Next」をクリック
- 「Tools for Native Modules」画面 → **チェックボックスはそのまま（チェックしない）** → 「Next」をクリック
- 「Ready to install」画面 → 「Install」をクリック
- ユーザーアカウント制御（UAC）のダイアログが出たら「はい」をクリック
- インストールが完了したら「Finish」をクリック

5. **ターミナルを新しく開き直してください**（インストール前に開いていたターミナルでは認識されないことがあります）

注意！ インストール先のフォルダ（Destination Folder）は変更しないでください。デフォルトのまま（`C:\Program Files\nodejs\`）にしておくことを強く推奨します。変更すると、環境変数の設定が必要になり、トラブルの原因になります。

Macでのインストール手順

方法A: 公式サイトからインストール（簡単）

1. ブラウザで <https://nodejs.org/> にアクセス
2. LTS版のボタンをクリックしてダウンロード
3. ダウンロードされた `.pkg` ファイルをダブルクリック
4. インストーラの指示に従って進める（すべてデフォルトでOK）

方法B: Homebrewでインストール（推奨）

Homebrew（ホームブルー）はMac用のパッケージ管理ツールです。開発者向けソフトウェアのインストール・管理が簡単になります。

```
# まず Homebrew がインストールされているか確認
brew --version

# 「command not found」と表示された場合、Homebrewをインストール
# 以下のコマンドを1行すべてコピーして貼り付けてください
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# インストール完了後、ターミナルの指示に従ってPATHを設定
# （画面に表示されるコマンドをコピー&ペーストして実行してください）

# Node.js v20をインストール
brew install node@20

# Node.jsをリンク（使えるようにする）
brew link node@20
```

インストールの確認

ターミナルを**新しく開き直して**から、以下のコマンドを実行します。（新しいターミナルで実行する理由：インストール時にPATH環境変数が更新されるため、古いターミナルでは変更が反映されていないことがあります）

```
# Node.jsのバージョンを確認
node --version

# 実行結果の例（成功時）：
# v20.11.0
#
# バージョン番号は異なってもOKですが、v20以上であることを確認してください。
# v18やv16と表示された場合は、古いバージョンです。

# npmのバージョンを確認
npm --version

# 実行結果の例（成功時）：
# 10.2.4
#
# こちらもバージョン番号は異なってもOKです。
```

```
# うまくいかない場合のチェック手順:

# 1. 「'node' は、内部コマンドまたは外部コマンド ... として認識されていません」と表示される
#   → ターミナルを閉じて、新しく開き直してから再実行してみてください

# 2. それでもダメな場合、Node.jsが正しい場所にインストールされているか確認
# Windowsの場合:
where node
# 実行結果の例: C:\Program Files\nodejs\node.exe

# Macの場合:
which node
# 実行結果の例: /usr/local/bin/node
```

npmとは？ npm (Node Package Manager: ノードパッケージマネージャー) は、Node.jsと一緒にインストールされるツールです。世界中の開発者が作ったライブラリ (パッケージ) を、コマンド1つでインストールできます。

「パッケージ」とは、誰かが作って公開してくれたプログラムの部品のことです。レゴブロックのように、必要な部品を組み合わせることでアプリケーションを作ります。

例えば BON-LOG では、以下のようなパッケージを使います:

- **next** - Webアプリケーションフレームワーク (アプリ開発の土台)
- **react** (リアクト) - UIライブラリ (画面表示を効率よく作るための部品集)
- **prisma** (プリズマ) - データベースORM (Object-Relational Mapping: データベース操作をTypeScriptのコードで行えるようにするツール)
- **tailwindcss** (テイルウィンドシーエスエス) - CSSフレームワーク (デザインを効率よく実装するためのツール)

これらをすべて自分で作る必要はなく、**npm install** コマンドで簡単に導入できます。

```
# パッケージをインストールする基本的なコマンドの例
npm install パッケージ名

# 例: react をインストールする場合
npm install react
# → node_modules フォルダに react がダウンロードされる
# → package.json にも自動的に記録される
```

「環境変数」と「PATH」について

Node.jsをインストールすると、ターミナルで `node` と入力するだけでNode.jsを実行できるようになります。これは、インストーラがコンピュータの「環境変数」の中の「PATH」という設定に、Node.jsの実行ファイルの場所を自動的に登録してくれるからです。

環境変数 PATH のイメージ:

「nodeコマンドを実行してください」

↓

コンピュータ: PATHに登録された場所を順番に探す

↓

C:\Program Files\nodejs\ にnode.exeを発見!

↓

Node.jsが起動する

コラム: 環境変数とは? 環境変数は、コンピュータ全体で共有される「設定値」のことです。「このプログラムはどこにある?」「デフォルトのエディタは何?」など、システム全体に影響する情報が保存されています。

PATHは最もよく使われる環境変数で、「コマンドの実行ファイルを探す場所のリスト」です。通常はインストーラが自動設定してくれるので、手動で操作する必要はありません。

よくあるトラブル

Q: `node --version` で「command not found」「認識されていません」と表示される

A: 以下を順番に試してください。

1. ターミナルを一度閉じて、新しく開き直す
2. PCを再起動する
3. (Windowsの場合) Node.jsをもう一度インストールし直す。その際「Add to PATH」にチェックが入っていることを確認する
4. (Macの場合) ターミナルで `which node` を実行して、Node.jsの場所を確認する

Q: バージョンが20未満と表示される

A: 古いバージョンのNode.jsがインストールされています。公式サイトから最新のLTS版をダウンロードして、上書きインストールしてください。

Q: 「npm WARN」という黄色い警告が表示される

A: 「WARN」は警告であってエラーではありません。多くの場合は無視して問題ありません。赤い「ERR!」が表示された場合のみ、対処が必要です。

理解度チェック

- ☐ Node.jsが何のためのツールか説明できますか？
- ☐ `node --version` でバージョンを確認できましたか？
- ☐ npmが何のためのツールか説明できますか？
- ☐ LTS版を選ぶ理由を説明できますか？

1.4 VS Code（エディタ）のインストール

このセクションで学ぶこと

- VS Codeとは何か、なぜ使うのか
- インストール手順
- 必須の拡張機能とその役割
- 基本的な使い方と設定

VS Codeとは？

VS Code（Visual Studio Code）は、Microsoft が開発した**無料のコードエディタ**です。世界で最も多くの開発者に使われているエディタで、以下の特徴があります。

- **無料**で使える
- **拡張機能**で機能を追加できる（プログラミング言語ごとの支援、デバッグ、Git連携など）
- **ターミナル内蔵**で、エディタ内からコマンドを実行できる
- **IntelliSense**（コード補完）が優秀で、入力候補を提案してくれる

コラム: メモ帳との違い Windowsの「メモ帳」やMacの「テキストエディット」でもコードは書けますが、VS Codeには以下のような開発を助ける機能があります。

- コードに色がつく（シンタックスハイライト）ので読みやすい
- 入力中にコードの候補を提案してくれる（IntelliSense）
- エラーを赤い波線で教えてくれる
- ファイルの検索・置換が強力
- Gitと連携して変更履歴を管理できる

ダウンロードとインストール

Windowsの場合

1. ブラウザで <https://code.visualstudio.com/> にアクセス
2. 「Download for Windows」と書かれた青いボタンをクリック
3. ダウンロードされた `.exe` ファイルをダブルクリック
4. インストーラが起動します。以下の手順で進めてください。
 - 「使用許諾契約書」→「同意する」を選択→「次へ」
 - 「インストール先」→ **そのまま変更せず** →「次へ」
 - 「スタートメニュー」→ **そのまま** →「次へ」
 - 「追加タスク」→ 以下の2つにチェックを入れることを **強く推奨**
 - 「エクスプローラーのファイルコンテキストメニューに "Codeで開く" を追加」
 - 「エクスプローラーのディレクトリコンテキストメニューに "Codeで開く" を追加」
 - 「PATHへの追加」（デフォルトでチェック済み。外さないこと）
 - 「インストール」→ クリックして待つ
 - 完了したら「完了」をクリック

Macの場合

1. ブラウザで <https://code.visualstudio.com/> にアクセス
2. 「Download for Mac」をクリック
3. ダウンロードされた `.zip` ファイルを展開（ダブルクリック）
4. 展開された `Visual Studio Code.app` を「アプリケーション」フォルダにドラッグ
5. アプリケーションフォルダから VS Code を起動

VS Codeの初期設定

VS Codeを起動したら、まず日本語化しましょう。

1. 左サイドバーの一番下にある**四角が4つのアイコン**（拡張機能）をクリック
 - または **Ctrl + Shift + X**（Mac: **Cmd + Shift + X**）を押す
2. 検索欄に「Japanese」と入力
3. 「Japanese Language Pack for Visual Studio Code」が表示されるので「Install」をクリック
4. 右下に「Restart」のポップアップが表示されたらクリックして再起動

推奨拡張機能のインストール

以下の拡張機能をインストールしてください。拡張機能は、VS Codeに機能を追加するプラグインのようなものです。

拡張機能	検索キーワード	用途	なぜ必要か
ESLint	ESLint	コードの品質チェック	文法ミスや悪い書き方を自動で検出
Prettier	Prettier	コードの自動整形	インデントや改行を自動で統一
Tailwind CSS IntelliSense	Tailwind CSS	Tailwindのクラス名補完	CSSクラス名を入力途中で候補表示
Prisma	Prisma	DBスキーマのハイライト	データベース定義ファイルを色分け表示
GitLens	GitLens	Gitの履歴を視覚化	誰がいつコードを変更したか表示
ES7+ React/Redux/React-Native	ES7 React	Reactのスニペット	よく使うコードのテンプレートを入力補助

インストール手順（各拡張機能について繰り返す）：

1. 左サイドバーの拡張機能アイコンをクリック
2. 検索欄に拡張機能名を入力
3. 表示された結果から正しい拡張機能を見つける（ダウンロード数が多いものを選ぶ）
4. 「Install」ボタンをクリック

VS Codeの設定

コードの自動整形やLintの自動修正を有効にするため、設定ファイルを編集します。

1. `Ctrl + Shift + P` (Mac: `Cmd + Shift + P`) を押す
 - 画面上部にコマンドパレット (検索欄) が表示されます
2. 「settings.json」と入力
3. 「Preferences: Open User Settings (JSON)」をクリック
4. 以下の内容を貼り付けて保存 (`Ctrl + S` / `Cmd + S`)

```
{
  // ファイル保存時にPrettierで自動整形する
  "editor.defaultFormatter": "esbenp.prettier-vscode",

  // ファイル保存時に自動整形を実行する
  "editor.formatOnSave": true,

  // ファイル保存時にESLintの自動修正を実行する
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },

  // TypeScriptのインポートパスを「@/」形式にする
  "typescript.preferences.importModuleSpecifier": "non-relative",

  // ファイルの最終行に空行を自動追加 (多くのプロジェクトで推奨)
  "files.insertFinalNewline": true,

  // 末尾の空白を自動削除
  "files.trimTrailingWhitespace": true
}
```

ここがポイント! `editor.formatOnSave` を `true` にすることで、ファイルを保存するたびにコードが自動的にきれいに整形されます。インデントがずれたり、セミコロンが抜けたりしても、保存するだけで修正されるので非常に便利です。

VS Codeの基本的なキーボードショートカット

VS Codeを効率よく使うために、以下のショートカットを覚えておきましょう。

操作	Windows	Mac	説明
ファイルを保存	Ctrl + S	Cmd + S	編集中のファイルを保存
ファイルを開く	Ctrl + P	Cmd + P	ファイル名で検索して開く
コマンドパレット	Ctrl + Shift + P	Cmd + Shift + P	VS Codeの全機能を検索
ターミナル表示	Ctrl + `	Cmd + `	内蔵ターミナルの表示/非表示
行のコピー	Ctrl + C （選択なし）	Cmd + C （選択なし）	カーソルのある行をコピー
行の削除	Ctrl + Shift + K	Cmd + Shift + K	カーソルのある行を削除
複数カーソル	Alt + クリック	Option + クリック	複数箇所を同時編集
全体検索	Ctrl + Shift + F	Cmd + Shift + F	プロジェクト全体から検索
元に戻す	Ctrl + Z	Cmd + Z	直前の操作を取り消し
やり直し	Ctrl + Y	Cmd + Shift + Z	取り消した操作をやり直し

よくあるトラブル

Q: 拡張機能の「Install」ボタンが押せない / インストールが進まない

A: インターネット接続を確認してください。VS Codeの拡張機能はインターネット経由でダウンロードされます。

Q: 保存しても自動整形が動かない

A: 以下を確認してください。

1. Prettier拡張機能がインストールされているか
2. settings.jsonの設定が正しく保存されているか
3. プロジェクトに **.prettierrc** などの設定ファイルがある場合、そちらの設定が優先されることがあります

理解度チェック

- ☐ VS Codeを起動できますか？
- ☐ 拡張機能を検索してインストールできますか？
- ☐ **Ctrl + S** (Mac: **Cmd + S**) でファイルを保存できますか？
- ☐ VS Code内でターミナルを開くことができますか？

1.5 Gitのインストール

このセクションで学ぶこと

- Gitとは何か、なぜ使うのか
- Gitのインストール手順
- 初期設定（ユーザー名・メールアドレス）
- GitHubアカウントの作成とSSHキーの設定

Gitとは？

Git（ギット）は**コードの変更履歴を管理するツール**です。ワードやエクセルの「元に戻す」機能の超強力版だと思ってください。

Gitとは何か？ Gitは「コードのタイムマシン」です。ゲームのセーブポイントのように、コードの状態を好きなタイミングで保存（コミット）でき、いつでも過去の状態に戻れます。

- **コミット（commit）**：コードのスナップショット（セーブポイント）
- **リポジトリ（repository）**：コミットの履歴を保管する場所
- **ローカルリポジトリ**：自分のPC上の履歴
- **リモートリポジトリ（GitHub）**：インターネット上の共有履歴

Gitを使うと以下のことができます：

- コードを変更するたびに「セーブポイント」を作れる（**コミット**）
- 過去の任意の時点に戻る
- 複数人で同じプロジェクトを同時に開発できる
- 「この変更をやっぱりやめたい」時に安全に戻せる

Gitの変更履歴のイメージ:

時間 →

[初期状態] → [ログイン画面追加] → [投稿機能追加] → [バグ修正] → [現在]

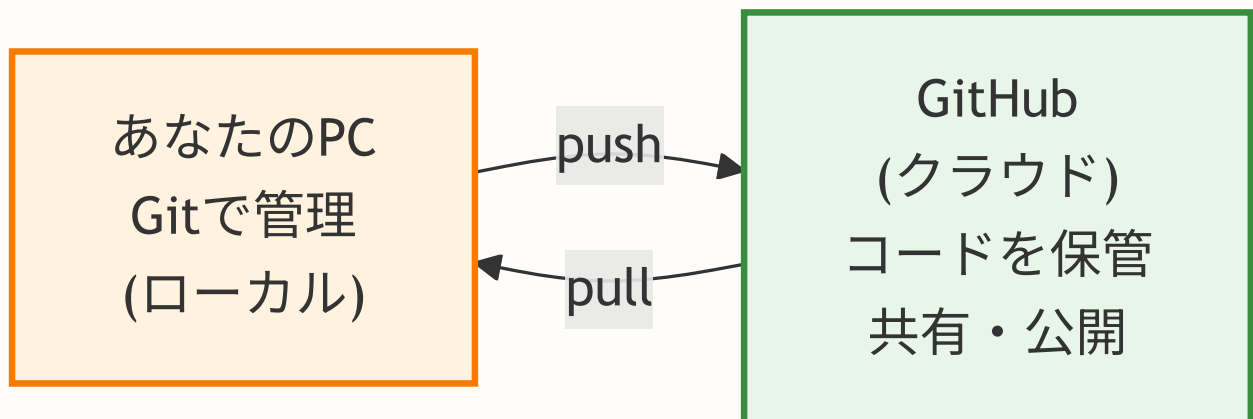
↑ ↑ ↑ ↑
コミット1 コミット2 コミット3 コミット4

- ・各コミットは「誰が」「いつ」「何を変更したか」を記録
- ・任意のコミットに戻ることができる
- ・コミット2で問題があれば、コミット1の状態に戻せる

GitHubとは？

GitHub（ギットハブ）は、Gitで管理しているコードをインターネット上に保存・共有するサービスです。GitとGitHubは別のものです。

Git と GitHub の関係:



- **push:** ローカルの変更をGitHubにアップロード
- **pull:** GitHubの変更をローカルにダウンロード

Windowsでのインストール手順

1. ブラウザで <https://git-scm.com/> にアクセス
2. 「Download for Windows」をクリック
3. ダウンロードされた `.exe` ファイルをダブルクリック
4. インストーラが起動します。多くの設定項目がありますが、**すべてデフォルトのまま「Next」を押し続けてOKです**
 - 特に以下は変更しないでください:

- 「Adjusting your PATH environment」 → 「Git from the command line and also from 3rd-party software」（デフォルト）
- 「Choosing the default editor」 → 「Use Visual Studio Code as Git's default editor」を選ぶとVS CodeがGitのエディタになります（任意）

5. 「Install」をクリック

6. 完了したら「Finish」をクリック

ここがポイント！ Gitのインストーラには設定項目が非常に多く、初心者は戸惑いがちです。基本的に**すべてデフォルトのまま**で問題ありません。慣れてきてから設定を変更すればOKです。

Macでのインストール手順

Macには最初からGitが入っている場合があります。まず確認しましょう。

```
# Gitがインストールされているか確認
git --version
```

「git version 2.x.x」のように表示されれば、すでにインストールされています。

表示されない場合、または「Xcode Command Line Tools」のインストールを促すダイアログが出た場合:

```
# Xcode Command Line Toolsをインストール（Gitが含まれている）
xcode-select --install
```

ダイアログが表示されたら「インストール」→「同意する」の順にクリックしてください。

インストールの確認と初期設定

```
# バージョン確認
git --version

# 実行結果の例（成功時）：
# git version 2.43.0
#
# 2.x.x（xは数字）の形式で表示されればOKです。
# バージョンが異なっても問題ありません。
```

次に、Gitの初期設定を行います。この設定は、コミット（セーブポイント）を作る際に「誰が変更したか」を記録するために使われます。**1回設定すればOK**で、毎回入力する必要はありません。

```
# ユーザー名を設定（GitHubのユーザー名と同じにすることを推奨）
git config --global user.name "あなたの名前"
# 例: git config --global user.name "yuya-tanaka"
# （何も表示されなければ成功です）

# メールアドレスを設定（GitHubに登録するメールアドレスと同じにする）
git config --global user.email "your-email@example.com"
# 例: git config --global user.email "yuya@example.com"
# （何も表示されなければ成功です）

# 設定の確認
git config --global --list

# 実行結果の例：
# user.name=yuya-tanaka
# user.email=yuya@example.com
# （他の設定項目も表示される場合がありますが、上記2つが含まれていればOK）
```

--global オプションとは？ **--global** は「このPC全体に適用する設定」を意味します。プロジェクトごとに異なるユーザー名を使いたい場合は、**--global** を外すと そのプロジェクトだけの設定になります。初心者のうちは **--global** で設定しておけばOKです。

注意！ `"あなたの名前"` と `"your-email@example.com"` は実際の値に置き換えてください。ダブルクォーテーション (`"`) は必要です。

この設定は、コミット（セーブポイント）を作る際に「誰が変更したか」を記録するために使われます。

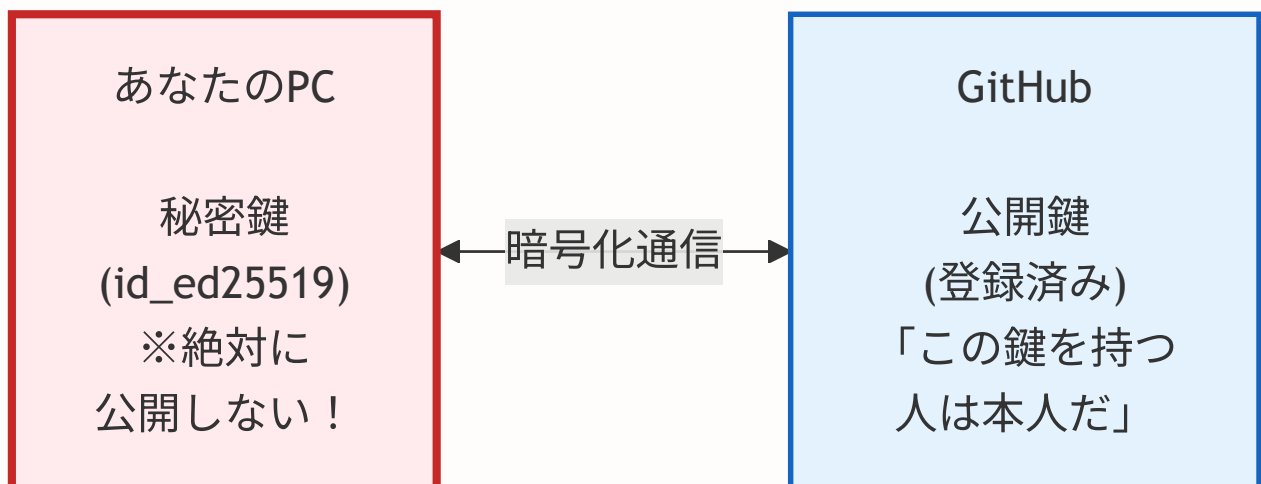
GitHubアカウントの作成

1. ブラウザで <https://github.com/> にアクセス
2. 「Sign up」をクリック
3. メールアドレス、パスワード、ユーザー名を入力して登録
4. メール認証を完了する

SSHキーの設定（GitHubとの安全な通信）

SSHキーは、パスワードを入力せずにGitHubと安全に通信するための仕組みです。鍵のペア（公開鍵と秘密鍵）を作成し、公開鍵をGitHubに登録します。

SSHキーのイメージ:



- **秘密鍵** = 家の鍵（自分だけが持つ）
- **公開鍵** = 鍵穴（GitHubに設置する）

```
# SSHキーを生成 (your-email@example.comは自分のメールアドレスに置き換え)
ssh-keygen -t ed25519 -C "your-email@example.com"

# コマンドの各部分の意味:
# ssh-keygen      : SSHキーを生成するコマンド
# -t ed25519      : 鍵の種類を「Ed25519」（現在推奨されている暗号方式）に指定
# -C "メール"     : 鍵にコメント（識別用のラベル）をつける

# 「Enter file in which to save the key」と聞かれたら
# → そのまま Enter を押す（デフォルトの場所に保存）

# 「Enter passphrase」と聞かれたら
# → そのまま Enter を押す（パスフレーズなし）
# → もう一度 Enter を押す（確認）

# 実行結果の例（成功時）:
# Generating public/private ed25519 key pair.
# Enter file in which to save the key (C:\Users\yuya/.ssh/id_ed25519): ← Enterを押す
# Enter passphrase (empty for no passphrase): ← Enterを押す
# Enter same passphrase again: ← Enterを押す
# Your identification has been saved in C:\Users\yuya/.ssh/id_ed25519
# Your public key has been saved in C:\Users\yuya/.ssh/id_ed25519.pub
# The key fingerprint is:
# SHA256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx your-email@example.com
# The key's randomart image is:
# +--[ED25519 256]--+
# |    .o..         |
# |    ..+o         |
# ...（アスキーアートが表示される）
```

次に、生成した公開鍵をコピーします。

```
# 公開鍵の内容を表示してコピー

# Windowsの場合 (PowerShell)
Get-Content ~/.ssh/id_ed25519.pub | Set-Clipboard

# Windows (Git Bash) の場合
cat ~/.ssh/id_ed25519.pub | clip

# Macの場合
cat ~/.ssh/id_ed25519.pub | pbcopy
```

注意！ コピーするのは `.pub` がついている方（公開鍵）です。 `.pub` がついていない方（秘密鍵）は絶対に公開しないでください。

GitHubに公開鍵を登録:

1. GitHubにログインする
2. 右上のアイコンをクリック → 「Settings」
3. 左メニューの「SSH and GPG keys」をクリック
4. 「New SSH key」 ボタンをクリック
5. 「Title」 に分かりやすい名前を入力（例: `My Windows PC`）
6. 「Key」 欄に先ほどコピーした公開鍵を貼り付け
7. 「Add SSH key」 をクリック

接続テスト:

```
ssh -T git@github.com
# 「Are you sure you want to continue connecting?」 → yes と入力して Enter
# 「Hi ユーザー名! You've successfully authenticated」と表示されれば成功
```

よくあるトラブル

Q: `ssh -T git@github.com` で「Permission denied」と表示される

A: SSHキーが正しく設定されていない可能性があります。

1. `ls ~/.ssh/` を実行して、`id_ed25519` と `id_ed25519.pub` が存在するか確認
2. 公開鍵の内容（`cat ~/.ssh/id_ed25519.pub`）をGitHubに正しく登録したか確認
3. キーを再生成して、再登録してみる

Q: Gitのコマンドが「command not found」になる（Mac）

A: `xcode-select --install` を実行して、Command Line Toolsをインストールしてください。

理解度チェック

- ☐ Gitとは何をするためのツールか説明できますか？
- ☐ GitとGitHubの違いを説明できますか？

- ☐ `git config` でユーザー名とメールアドレスを設定できましたか？
- ☐ SSHキーが何のためにあるか説明できますか？

1.6 Docker Desktopのインストール

このセクションで学ぶこと

- Dockerとは何か、なぜ使うのか
- Docker Desktopのインストール手順
- 基本的なDockerコマンド

Dockerとは？

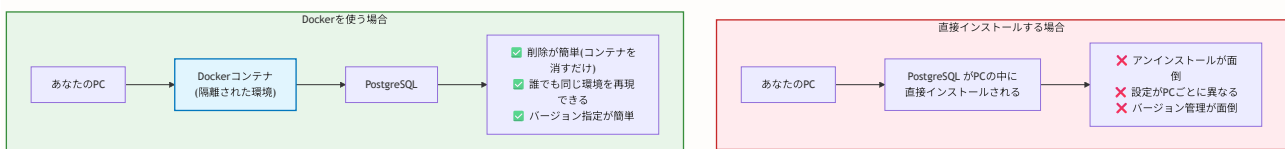
Docker（ドッカー）は、「コンテナ」と呼ばれる隔離された環境を作るツールです。

Dockerとは何か？ Dockerは「アプリの実行環境をまるごとパッケージする」ツールです。「自分のPCでは動くのに、他の人のPCでは動かない」問題を解決します。

- コンテナ:** 隔離された実行環境。他のアプリに影響しない
- Docker Compose:** 複数のコンテナ（例: PostgreSQL + Next.js）をまとめて管理するツール
- `-d` **フラグ:** バックグラウンド実行（ターミナルを占有しない）

BON-LOGでは、データベース（PostgreSQL）をDockerで動かします。PostgreSQLを直接PCにインストールすることもできますが、Dockerを使う方がメリットが多いです。

Dockerのイメージ:



コラム: なぜDockerを使うのか？

- 環境を汚さない: PostgreSQLが不要になったら、コンテナを削除するだけでOK
- チームで同じ環境: `docker-compose.yml` を共有すれば、全員同じ環境を構築できる
- バージョン管理が簡単: `postgres:16-alpine` と書くだけでPostgreSQL 16を指定できる
- 複数プロジェクト対応: プロジェクトごとに異なるバージョンのデータベースを使える

Windowsでのインストール手順

前提条件: WSL2の確認

Docker Desktop for Windowsは、WSL2（Windows Subsystem for Linux 2：ダブリュエスエルツー。Windows上でLinux環境を動作させるための仕組み）を使用します。WSL2を使うことで、DockerはLinuxのコンテナ技術をWindows上で効率よく実行できます。Windows 10 バージョン 2004以降であれば、WSL2がサポートされています。

```
# PowerShellを管理者として実行し、WSL2が有効か確認
wsl --status

# 有効でない場合は、以下でインストール
wsl --install

# PCの再起動が必要になる場合があります
```

Docker Desktopのインストール

1. ブラウザで <https://www.docker.com/products/docker-desktop/> にアクセス
2. 「Download for Windows」をクリック
3. ダウンロードされた `.exe` ファイルをダブルクリック
4. 「Use WSL 2 instead of Hyper-V」にチェックが入っていることを確認 → 「OK」をクリック
5. インストールが完了したら、PCを再起動
6. 再起動後、Docker Desktopが自動的に起動する
 - 利用規約が表示されたら「Accept」をクリック
 - アンケートはスキップしてOK

Macでのインストール手順

1. ブラウザで <https://www.docker.com/products/docker-desktop/> にアクセス

2. お使いのMacのチップに合わせてダウンロード

- Apple Silicon (M1/M2/M3/M4) : 「Mac with Apple Silicon」
- Intel: 「Mac with Intel Chip」
- 分からない場合: 画面左上のAppleマーク → 「このMacについて」で確認

3. ダウンロードされた `.dmg` ファイルをダブルクリック

4. Docker.app を Applications フォルダにドラッグ

5. Applications フォルダから Docker を起動

6. 利用規約に同意

インストールの確認

Docker Desktop が起動している状態で、ターミナルから確認します。

注意！ Docker コマンドは、Docker Desktop が起動している（タスクバー / メニューバーに Docker のクジラアイコン（鯨のマーク）が表示されている）状態でないと使えません。Docker Desktop の起動には 1〜2 分かかることがあります。クジラアイコンのアニメーションが止まったら起動完了です。

```
# Dockerのバージョン確認
docker --version

# 実行結果の例（成功時）：
# Docker version 25.0.3, build 4deb41
#
# バージョン番号は異なってもOKです。

# Docker Compose（ドッカーコンポーズ：複数のコンテナをまとめて管理するツール）の
# バージョン確認
docker compose version

# 実行結果の例（成功時）：
# Docker Compose version v2.24.5
```

もしエラーが出た場合のチェック手順:

```
# 1. 「docker: command not found」「認識されていません」の場合
#   → Docker Desktop が起動しているか確認してください
#   → タスクバー (Windows) / メニューバー (Mac) にクジラアイコンがあるか確認

# 2. 「Cannot connect to the Docker daemon」の場合
#   → Docker Desktop が完全に起動するまで待ってから再実行してください

# 3. 「docker compose」が認識されない場合 (古いDocker)
#   → 「docker-compose」 (ハイフンあり) を試してください
docker-compose version
```

Docker Compose とは？ Docker Compose は、複数のDockerコンテナ (例: データベース + アプリケーション) をまとめて定義・起動するためのツールです。 `docker-compose.yml` という設定ファイルに必要なコンテナの構成を記述します。Docker Desktopに同梱されているため、別途インストールする必要はありません。

よくあるトラブル

Q: Windowsで「WSL 2 installation is incomplete」と表示される

A: WSL2が正しくインストールされていません。PowerShellを**管理者として実行**し、 `ws2 --install` を実行してからPCを再起動してください。

Q: 「Cannot connect to the Docker daemon」と表示される

A: Docker Desktopが起動していません。スタートメニュー (Mac: アプリケーション) からDocker Desktopを起動してください。起動に1~2分かかることがあります。タスクバーのクジラアイコンが止まったら準備完了です。

Q: Dockerが起動するとPCが重くなる

A: DockerはPCのメモリを使用します。メモリが8GB以下のPCでは、Docker Desktopの設定 (Settings → Resources) からメモリ割り当てを減らしてみてください。開発時にDockerを使わない場合は、Docker Desktopを終了しておくとPCが軽くなります。

理解度チェック

- ☐ Dockerが何のためのツールか説明できますか？

- ☐ Docker Desktopを起動できますか？
- ☐ `docker --version` でバージョンを確認できましたか？
- ☐ Dockerを使うメリットを1つ以上挙げられますか？

1.7 プロジェクトの作成

このセクションで学ぶこと

- Next.jsプロジェクトの作成手順
- 各質問の意味と選択理由
- プロジェクトの初期ファイル構成
- 開発サーバーの起動方法

Next.jsプロジェクトの初期化

いよいよBON-LOGのプロジェクトを作成します。Next.js（ネクストジェイエス）は、React（リアクト：UIを効率よく構築するためのJavaScriptライブラリ）をベースにしたフルスタック（フロントエンドからバックエンドまで対応）型の Webアプリケーションフレームワーク（アプリ開発の土台となるソフトウェア）です。

ターミナルを開いて、以下のコマンドを順番に実行してください。

```
# プロジェクトを作成したいディレクトリに移動
# （例：デスクトップの中に Bonsai フォルダがある場合）
cd ~/Desktop/Bonsai

# もし Bonsai フォルダがなければ作成してから移動
# mkdir ~/Desktop/Bonsai
# cd ~/Desktop/Bonsai
```

```
# Next.jsプロジェクトを作成するコマンド
# npx は npm に同梱されているツールで、パッケージを一時的にダウンロードして実行する
# create-next-app は Next.js の公式プロジェクト作成ツール
# @latest は「最新版を使う」という意味
# bonsai-sns-project はプロジェクト名（フォルダ名になる）
npx create-next-app@latest bonsai-sns-project
```

コマンドを実行すると、いくつかの質問が表示されます。それぞれの質問の意味と、なぜその選択をするかを説明します。

質問1: Would you like to use TypeScript? → Yes
(TypeScript (タイプスクリプト) を使いますか?)

理由: TypeScript は JavaScript に「型」の概念を追加した言語です。
型があると「この変数は数値しか入れられない」「この関数は文字列を返す」といったルールをコードに書けるため、バグ (プログラムの誤り) をコードを書いている段階で検出できます。
BON-LOGでは全コードをTypeScriptで書きます。
(詳しくは第3章で学習します)

質問2: Would you like to use ESLint? → Yes
(ESLint (イーエスリント) を使いますか?)

理由: ESLintは JavaScript/TypeScript のコードの品質をチェックするツール (リンター: Linter) です。文法ミスや推奨されない書き方を自動的に検出して警告してくれます。

質問3: Would you like to use Tailwind CSS? → Yes
(Tailwind CSS (テイルウィンドシーエスエス) を使いますか?)

理由: Tailwind CSSはユーティリティファースト型のCSSフレームワーク (CSS: Cascading Style Sheets (カスケディングスタイルシート) = Webページの見た目を定義する言語) です。
HTMLのclass属性にクラス名を書くだけでスタイルを当てることができ、CSSファイルを別途作成する手間が大幅に減ります。
(詳しくは第6章で学習します)

質問4: Would you like your code inside a `src/` directory? → No
(srcディレクトリの中にコードを入れますか?)

理由: Next.js の App Router では、src/ なしの構成が一般的です。
シンプルなディレクトリ構成を維持するため No を選びます。

質問5: Would you like to use App Router? → Yes
(App Router (アップルーター) を使いますか?)

理由: App Router は Next.js 13以降で導入された新しいルーティング (ルーティング: URLとページの対応づけ) 方式です。
Server Components (サーバー上で実行されるコンポーネント) や Server Actions (サーバー上で実行されるデータ操作関数) などの最新機能が使えます。BON-LOGはApp Routerベースで開発します。
(以前の方式は「Pages Router」と呼ばれ、現在も使えますが、新規プロジェクトではApp Routerが推奨されています)

質問6: Would you like to use Turbopack for next dev? → Yes
(開発時にTurbopack (ターボパック) を使いますか?)

理由: Turbopack は Next.js の新しいバンドラー (バンドラー: 複数のJavaScript/CSSファイルを1つにまとめるツール) で、

従来のWebpack（ウェブバック）よりも開発サーバーの起動とリロード（ファイルを変更した時の画面更新）が大幅に高速になります。

質問7: Would you like to customize the import alias? → Yes → @/*
 （インポートエイリアス（import alias: ファイルの読み込み先の別名）をカスタマイズしますか？）

理由: @/* を設定すると、プロジェクトルートからの絶対パスでファイルをインポート（他のファイルから読み込む）できるようになります。

例: import { Button } from "@components/ui/Button"
 (@/ はプロジェクトルートを意味する。これがないと ../../ のような相対パスを書く必要があり、フォルダ階層が深くなるほど面倒になる)

注意！ `npx create-next-app` の実行には数分かかることがあります。途中で「Need to install the following packages:」と表示されたら、`y` を入力して Enter を押してください。

```
# 実行結果の例:

# Need to install the following packages:
#   create-next-app@16.x.x
# Ok to proceed? (y)          ← y を入力して Enter

# ✓ Would you like to use TypeScript? ... No / Yes          ← Yes を選択
# ✓ Would you like to use ESLint? ... No / Yes              ← Yes を選択
# ✓ Would you like to use Tailwind CSS? ... No / Yes        ← Yes を選択
# ✓ Would you like your code inside a `src/` directory? ... No / Yes ← No を選択
# ✓ Would you like to use App Router? ... No / Yes          ← Yes を選択
# ✓ Would you like to use Turbopack for `next dev`? ... No / Yes ← Yes を選択
# ✓ Would you like to customize the import alias? ... No / Yes ← Yes を選択
# ✓ What import alias would you like configured? ... @/*    ← そのまま Enter

# Creating a new Next.js app in /Users/yuya/Desktop/Bonsai/bonsai-sns-project.
# （パッケージのインストールが始まります。数分かかります）
#
# Installing dependencies:
# - react
# - react-dom
# - next
# ...
#
# Initialized a git repository.
# Success! Created bonsai-sns-project at ...
```


選択操作のやり方 質問では矢印キー（↑↓）で選択肢を切り替え、Enterキーで決定します。「Yes / No」の質問では、Yes がハイライトされた状態で Enter を押すと Yes、矢印キーで No に移動して Enter を押すと No が選択されます。

プロジェクトのディレクトリに移動

```
# 作成されたプロジェクトのフォルダに移動
cd bonsai-sns-project
```

作成されたファイルの確認

プロジェクトが作成されると、以下のようなファイル・フォルダが自動生成されます。

```
bonsai-sns-project/
├─ app/                # アプリケーションのメインフォルダ
│  ├─ favicon.ico      # ブラウザのタブに表示されるアイコン
│  ├─ globals.css      # 全体に適用されるCSS
│  ├─ layout.tsx       # 全ページ共通のレイアウト
│  └─ page.tsx         # トップページ (http://localhost:3000 で表示)
├─ public/            # 静的ファイル (画像など) を置くフォルダ
│  ├─ next.svg         # Next.jsのロゴ画像
│  └─ vercel.svg       # Vercelのロゴ画像
├─ .eslintrc.json     # ESLintの設定ファイル
├─ .gitignore         # Gitで管理しないファイルのリスト
├─ next.config.ts     # Next.jsの設定ファイル
├─ package.json       # プロジェクトの情報と依存パッケージの一覧
├─ package-lock.json  # パッケージのバージョンを固定するファイル
├─ postcss.config.mjs  # PostCSS (Tailwind CSS用) の設定
├─ tailwind.config.ts  # Tailwind CSSの設定
├─ tsconfig.json      # TypeScriptの設定
└─ README.md         # プロジェクトの説明ファイル
```

ここがポイント！ package.json とは？ `package.json` はプロジェクトの「設計書」のようなファイルです。

- プロジェクト名やバージョン
- 使用しているパッケージ（ライブラリ）の一覧
- よく使うコマンドのショートカット（scripts）が記録されています。

`npm install` コマンドを実行すると、`package.json` に書かれた パッケージが自動的にダウンロードされて `node_modules` フォルダに保存されます。

Gitリポジトリの初期化

プロジェクトのバージョン管理を始めます。 リポジトリ（repository: ソースコードとその変更履歴を保管する場所）を作成しましょう。

注意: `npm create-next-app` が自動的にGitリポジトリを初期化してくれる場合があります。 その場合、`git init` はスキップしてOKです。

```
# Gitリポジトリを初期化（プロジェクトフォルダで実行）
# これにより、このフォルダがGitで管理されるようになる
git init

# 実行結果の例：
# Initialized empty Git repository in C:/Users/yuya/Desktop/Bonsai/bonsai-sns-project/.git
# （すでに初期化済みの場合: Reinitialized existing Git repository in ...）

# 現在のファイルをすべてステージング（コミットの準備）する
# 「.」は「現在のフォルダのすべてのファイル」を意味する
# ステージング（staging）= 「次のコミットに含めるファイルを選択すること」
git add .
# （何も表示されなければ成功です）

# 最初のコミット（セーブポイント）を作成する
# -m はメッセージ（変更内容の説明）を指定するオプション
git commit -m "Initial commit: Next.js project setup"

# 実行結果の例：
# [main (root-commit) abc1234] Initial commit: Next.js project setup
# 15 files changed, 500 insertions(+)
# create mode 100644 .eslintrc.json
# create mode 100644 .gitignore
# create mode 100644 README.md
# ...
```

開発サーバーの起動

```
# 開発サーバーを起動する
# 「npm run dev」は package.json の scripts に定義された
# 「dev」コマンド（= next dev）を実行するショートカット
npm run dev
```

- **実行するとこうなる**（ターミナル）：次のような表示が出ます。この表示が出たら、開発サーバーが正常に起動しています。

```
▲ Next.js 16.x.x (Turbopack)
- Local:   http://localhost:3000

✓ Starting ...
✓ Ready in xxxms
```

各行の意味:

- `Next.js 16.x.x (Turbopack)` -- 使用しているNext.jsのバージョンとバンドラー
- `Local: http://localhost:3000` -- ブラウザでアクセスするURL
- `Ready in xxxms` -- サーバーの起動にかかった時間 (ms = ミリ秒。1000ms = 1秒)

この状態でターミナルは「サーバー実行中」になっており、新しいコマンドを入力できません。ターミナルが入力を受け付けない（プロンプトが表示されない）のは正常な状態です。

- **実行するとうなる**（ブラウザ）：ブラウザ（Chrome推奨）で <http://localhost:3000> を開いてください。Next.js のウェルカムページ（「Welcome to Next.js」などの見出しとリンク一覧）が表示されれば成功です。画面は Next.js のバージョンにより多少異なりますが、英語の説明と「Get started」などのボタンが見えていれば問題ありません。

ブラウザのアドレスバーに以下を入力:

`http://localhost:3000`

- **localhost** とは「自分のPC」を指す特別な名前
- **3000** はポート番号（サーバーの窓口番号）

注意！ 開発サーバーを停止するには、ターミナルで `Ctrl + C` を押します。サーバーが動いている間は、ターミナルに別のコマンドを入力できません。新しいコマンドを実行したい場合は、VS Codeで新しいターミナルを開くか、サーバーを一度停止してください。

よくあるトラブル

Q: `npm run dev` で「port 3000 is already in use」と表示される

A: ポート3000が他のプログラムに使われています。

- 別のターミナルで開発サーバーが動いていないか確認してください
- 他のプログラムが3000番ポートを使っている場合、そのプログラムを終了するか、`npm run dev -- -p 3001` で別のポート番号を指定してください

Q: ブラウザに「This site can't be reached」と表示される

A: 開発サーバーが起動していない可能性があります。

- ターミナルで `npm run dev` が実行中であることを確認してください
- URLが `http://localhost:3000` (`http`の後にsがない) であることを確認してください

Q: `npx create-next-app` が途中で止まる / エラーになる

A: ネットワーク接続を確認してください。プロキシ環境（会社や学校のネットワーク）の場合、npm のプロキシ設定が必要な場合があります。

理解度チェック

- ☐ `npx create-next-app` コマンドで新しいプロジェクトを作成できましたか？
- ☐ `npm run dev` で開発サーバーを起動できましたか？
- ☐ ブラウザで `http://localhost:3000` にアクセスできましたか？
- ☐ `package.json` の役割を説明できますか？
- ☐ `Ctrl + C` で開発サーバーを停止できますか？

1.8 PostgreSQLの起動（Docker）

このセクションで学ぶこと

- `docker-compose.yml`の書き方と各項目の意味
- PostgreSQLコンテナの起動・停止方法
- コンテナの状態確認方法

`docker-compose.yml`の作成

プロジェクトルート（`bonsai-sns-project` フォルダ直下）に `docker-compose.yml` を作成します。

docker-compose.yml とは？ Docker Compose（ドッカーコンポーズ）のコンテナ設定を記述するファイルです。「どんなソフトウェアを」「どんな設定で」「どのように起動するか」を定義します。YAML（ヤムル：Yet Another Markup Language）形式で書かれ、インデント（字下げ）が重要です。**スペース2つ**でインデントしてください。タブ文字は使えません。

YAML の基本ルール:

- **キー：値** の形式で設定を書く
- 階層構造はスペース（半角2つ）のインデントで表現する
- **#** 以降はコメント（メモ）として無視される
- 文字列の値はクォーテーション（**"**）で囲んでも囲まなくてもよい

VS Code でプロジェクトを開き、プロジェクトルートに **docker-compose.yml** を新規作成します。

```

# docker-compose.yml
# Docker Compose の設定ファイル

services:
  # "postgres" という名前のサービス（コンテナ）を定義
  postgres:
    # 使用するDockerイメージ（イメージ：コンテナの設計図・テンプレート）
    # PostgreSQL 16のAlpine Linux版
    # Alpine（アルパイン）は軽量なLinuxディストリビューション（配布版）で、
    # 通常のイメージ（約400MB）に比べてサイズが小さい（約80MB）ため起動が速い
    image: postgres:16-alpine

    # コンテナの名前（docker compose ps などに表示される名前）
    container_name: bonsai-postgres

    # コンテナが異常終了した場合に自動的に再起動する
    # Docker Desktopを手動で停止した場合は再起動しない
    restart: unless-stopped

    # PostgreSQL の環境変数（初期設定）
    environment:
      # データベースの管理者ユーザー名
      POSTGRES_USER: postgres
      # データベースの管理者パスワード（開発用なので簡単なもの）
      POSTGRES_PASSWORD: postgres
      # 初期作成するデータベースの名前
      POSTGRES_DB: bonsai_sns

    # ポートのマッピング（ホスト:コンテナ）
    # PCの5432番ポートをコンテナの5432番ポートに接続
    # これにより、localhost:5432 でPostgreSQLに接続できる
    ports:
      - "5432:5432"

    # データの永続化（コンテナを削除してもデータが残る）
    # postgres_data という名前のボリュームにデータを保存
    volumes:
      - postgres_data:/var/lib/postgresql/data

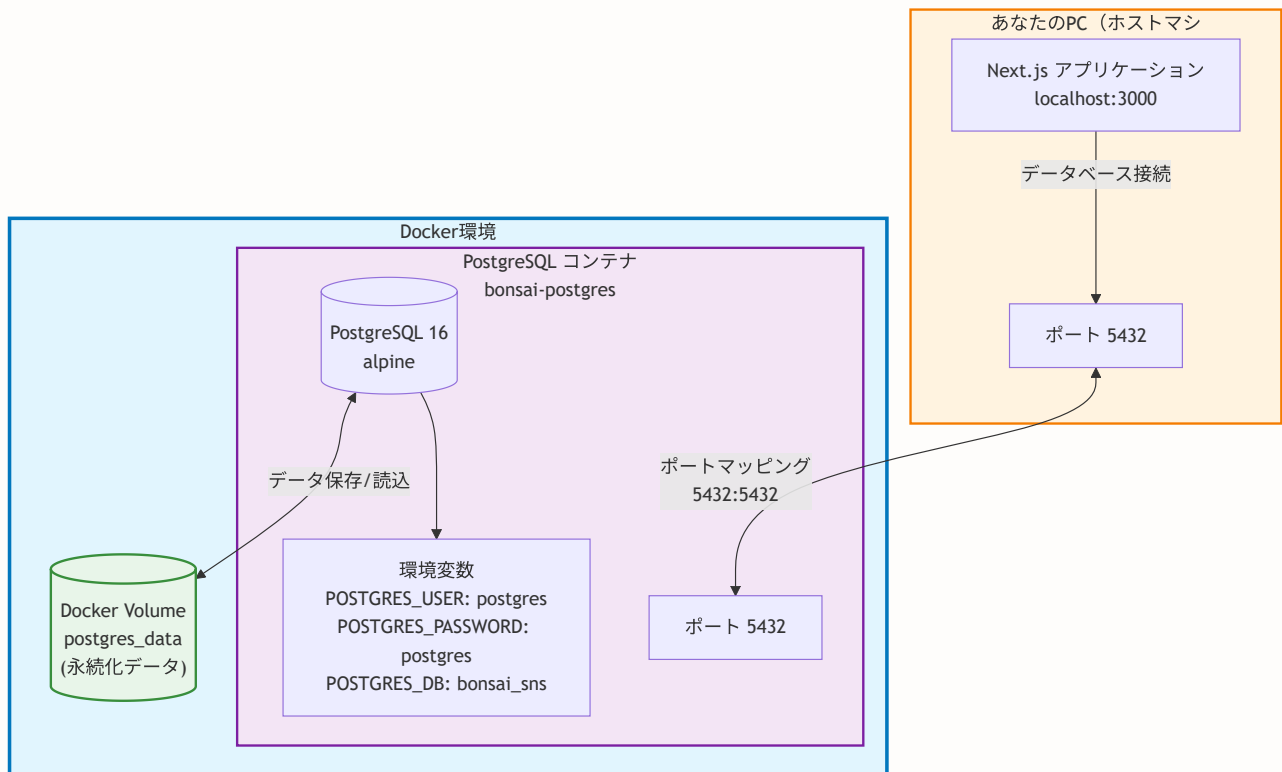
    # ヘルスチェック：PostgreSQLが接続受け付け可能か定期的に確認
    # 他のコンテナ（app）が「起動完了まで待つ」のに使用される
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d bonsai_sns"]
      interval: 10s    # 10秒ごとにチェック
      timeout: 5s      # 5秒以内に応答がなければ失敗
      retries: 5        # 5回失敗でunhealthy判定

# ボリューム（データの保存先）の定義
volumes:
  postgres_data:

```

Docker Composeサービス構成図

上記の `docker-compose.yml` で定義されているサービスとその関係を図で表すと以下のようになります。



図の説明:

- **Next.jsアプリ:** ホストPC上で動作し、`localhost:5432` でデータベースに接続
- **ポートマッピング:** ホストの5432番ポートとコンテナの5432番ポートを接続
- **PostgreSQLコンテナ:** 隔離された環境でPostgreSQLが動作
- **Docker Volume:** コンテナを削除してもデータが残る永続化ストレージ

PostgreSQLの起動と確認

```
# Docker Desktopが起動していることを確認してから実行

# PostgreSQLコンテナを起動
# -d はバックグラウンド（デタッチドモード：ターミナルを占有せずに実行する）オプション
docker compose up -d postgres

# 初回は PostgreSQL のイメージ（コンテナの元となるテンプレート）を
# ダウンロードするため、数分かかります

# 実行結果の例（初回）：
# [+] Running 1/1
#   ✓ postgres Pulled                                15.2s
#   ✓ 4abcdefg Pull complete                          5.0s
#   ✓ 1234abcd Pull complete                          7.0s
# [+] Running 2/2
#   ✓ Network bonsai-sns-project_default Created      0.0s
#   ✓ Container bonsai-postgres Started               0.3s

# 実行結果の例（2回目以降）：
# [+] Running 1/1
#   ✓ Container bonsai-postgres Started               0.3s
```

起動が完了したら、状態を確認しましょう。

```
# コンテナの状態を確認
docker compose ps

# 期待する出力：
# NAME                IMAGE                COMMAND                SERVICE    CREATED
# bonsai-postgres     postgres:16-alpine  "docker-entrypoint.s..." postgres   10 seconds ago
```

STATUS が「Up」と表示されていれば、PostgreSQLが正常に起動しています。

```
# ログを確認して、PostgreSQLが接続を受け付ける準備ができているか確認
docker compose logs -f postgres

# 「database system is ready to accept connections」と表示されれば成功
# ログの表示を止めるには Ctrl + C を押す
```

起動・停止・削除のコマンドまとめ

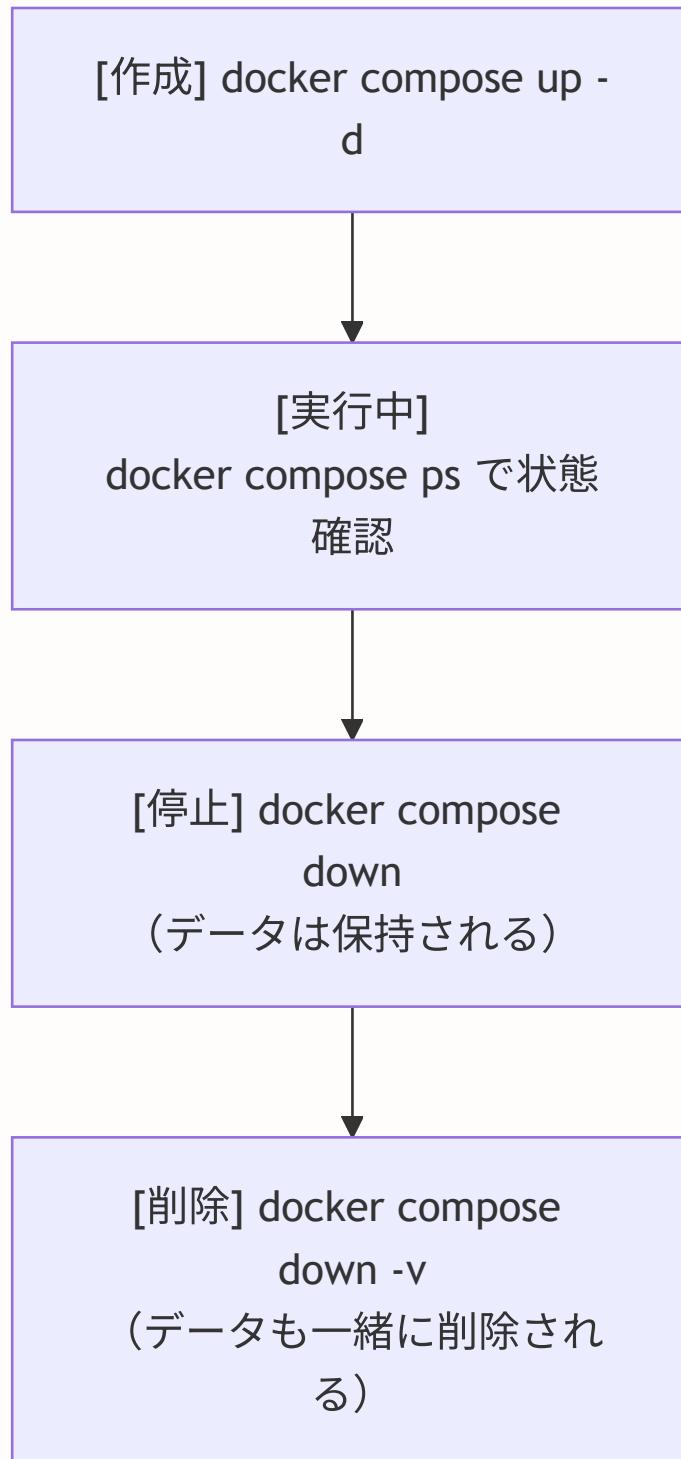
```
# 起動
docker compose up -d postgres

# 状態確認
docker compose ps

# ログ確認 (Ctrl+Cで終了)
docker compose logs -f postgres

# 停止 (データは保持)
docker compose down

# 停止 + データ削除 (完全にリセットしたい時)
docker compose down -v
```



よくあるトラブル

Q: 「port 5432 is already allocated」と表示される

A: 5432番ポートが他のプログラムに使われています。

- ローカルにPostgreSQLがインストールされている場合は、それを停止してください

- 他のDockerコンテナが5432番を使っている場合は、`docker ps` で確認して停止してください

Q: 「Cannot connect to the Docker daemon」と表示される

A: Docker Desktopが起動していません。Docker Desktopアプリケーションを起動してください。

Q: ダウンロードが途中で失敗する

A: ネットワーク接続を確認してください。再度 `docker compose up -d postgres` を実行すれば、途中から再開されます。

理解度チェック

- ☐ `docker-compose.yml` を作成できましたか？
- ☐ PostgreSQLコンテナを起動できましたか？
- ☐ `docker compose ps` でコンテナの状態を確認できましたか？
- ☐ コンテナの停止方法を知っていますか？
- ☐ `-d` オプションの意味を説明できますか？

1.9 環境変数の設定

このセクションで学ぶこと

- 環境変数ファイル（.env.local）とは何か
- なぜ環境変数を使うのか
- .env.local の作成と設定内容の意味
- 全環境変数の詳細解説（DATABASE_URL, NEXTAUTH_*, R2_*, STRIPE_*, SENTRY_* 等）
- `NEXT_PUBLIC_` プレフィックスの仕組みとセキュリティ
- 環境変数をGitにコミットしてはいけない理由
- 環境変数の安全な管理方法

環境変数とは？

環境変数とは？ 環境変数は「アプリの外部設定」です。パスワードやAPIキーなど、コードに直接書きたくない情報を安全に渡す仕組みです。

- `.env.local` : ローカル開発用の設定ファイル（Gitに含めない）
- `NEXT_PUBLIC_` **プレフィックス**: この接頭辞が付くとブラウザ側でも参照可能。付かない変数はサーバー側のみ（セキュリティのため）
- `localhost` : 自分のPC自身を指す特別なアドレス
- **ポート番号（3000）** : 同じPCで複数のサーバーを区別するための番号（マンションの部屋番号のようなもの）

環境変数は、アプリケーションの設定値を外部ファイルに保存する仕組みです。「データベースの接続先はどこか？」「秘密の鍵は何か？」といった情報を、コードの中に直接書くのではなく、別のファイルに分離します。

なぜ環境変数を使うのか：

❌ コードに直接書く場合（危険！）：

```
const password = "super-secret-password"
```

→ GitHubにプッシュすると、世界中に公開されてしまう！

✅ 環境変数を使う場合（安全）：

```
const password = process.env.DATABASE_PASSWORD
```

→ 値は `.env.local` ファイルにあり、Gitには含まれない

環境（開発/本番）によって値を変えられる：

開発： `DATABASE_URL = "localhost:5432"` （自分のPC）

本番： `DATABASE_URL = "xxx.supabase.co"` （クラウドサーバー）

環境変数の基本的な仕組み

Next.js では、環境変数ファイルを複数のレベルで読み込むことができます。優先順位は以下の通りです（上が最優先）。

環境変数ファイルの優先順位:

1. `.env.local` ← 最優先 (ローカルの上書き設定)
2. `.env.development` ← 開発モード(`npm run dev`)時のみ読み込み
3. `.env.production` ← 本番モード(`npm run build`)時のみ読み込み
4. `.env` ← 全環境共通のデフォルト値

※ BON-LOG では主に `.env.local` を使用します

環境変数は、コード内で `process.env.変数名` という形式でアクセスできます。

```
// サーバーサイドでの使用例 (Server Component, Server Action, API Route)
const dbUrl = process.env.DATABASE_URL
const secret = process.env.NEXTAUTH_SECRET

// クライアントサイドでの使用例 (Client Component)
// NEXT_PUBLIC_ プレフィックスがついた変数のみアクセス可能
const appUrl = process.env.NEXT_PUBLIC_APP_URL
```

.env.localファイルの作成

プロジェクトルート (`bonsai-sns-project` フォルダ直下) に `.env.local` ファイルを作成します。

注意! ファイル名の先頭にドット (`.`) が付いています。これは「隠しファイル」を意味し、通常のエクスプローラー / Finder では表示されないことがあります。VS Code のファイルエクスプローラーでは表示されるので、VS Code 上で作成するのが確実です。

プロジェクトには `.env.local.example` というサンプルファイルが用意されています。このファイルをコピーして `.env.local` を作成するのが最も確実な方法です。

```
# サンプルファイルから .env.local を作成
cp .env.local.example .env.local

# Windows PowerShell の場合
Copy-Item .env.local.example .env.local
```

あるいは、VS Code で `.env.local.example` を開いて内容を確認しながら、新規ファイル `.env.local` を作成してもOKです。

全環境変数の詳細解説

以下で、BON-LOG が使用する全環境変数を **カテゴリ別** に解説します。 まず開発環境で最低限必要なものを設定し、その後、本番デプロイ時に追加の設定を行います。

カテゴリ1: データベース接続（必須）

```
# === データベース接続 ===

# PostgreSQL への接続URL
# 形式: postgresql://ユーザー名:パスワード@ホスト:ポート/データベース名
# Docker Compose で設定した値と一致させる必要がある
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/bonsai_sns"

# Prisma が直接接続するためのURL（コネクションプーリング時に必要）
# 開発環境では DATABASE_URL と同じでOK
DIRECT_URL="postgresql://postgres:postgres@localhost:5432/bonsai_sns"
```

DATABASE_URL の各部分の解説:

部分	値	説明
postgresql://	プロトコル	PostgreSQLへの接続を意味する
postgres	ユーザー名	docker-compose.ymlのPOSTGRES_USERと一致
postgres	パスワード	docker-compose.ymlのPOSTGRES_PASSWORDと一致
localhost	ホスト名	localhostは自分のPC
5432	ポート番号	PostgreSQLのデフォルトは5432
bonsai_sns	データベース名	接続先のデータベース

開発環境と本番環境の違い

- 開発環境: `localhost:5432` (ローカルのDockerコンテナ)
- 本番環境 (Supabase): `aws-0-[region].pooler.supabase.com:6543` (クラウドサーバー)

本番環境では Supabase の接続プール (PgBouncer) を経由するため、`DATABASE_URL` (プール経由) と `DIRECT_URL` (直接接続) を分ける必要があります。開発環境では両方同じ値でOKです。

開発環境と本番環境のDB接続:

開発環境:

アプリ → `DATABASE_URL` → `localhost:5432` → Docker内PostgreSQL

本番環境:

アプリ → `DATABASE_URL` → PgBouncer(6543) → Supabase PostgreSQL

Prisma Migrate → `DIRECT_URL` → 直接接続(5432) → Supabase PostgreSQL

カテゴリ2: NextAuth.js 認証設定 (必須)

```
# === NextAuth.js (認証) ===

# アプリケーションのベースURL
NEXTAUTH_URL=http://localhost:3000

# セッション暗号化に使う秘密鍵
# 本番環境では必ずランダムな文字列に変更すること！
# 生成方法: openssl rand -base64 32
NEXTAUTH_SECRET=development-secret-key-change-in-production
```

変数名	用途	開発環境の値	本番環境の値
<code>NEXTAUTH_URL</code>	認証コールバックのベースURL	<code>http://localhost:3000</code>	<code>https://your-domain.com</code>
<code>NEXTAUTH_SECRET</code>	JWT トークンの暗号化キー	任意の文字列	<code>openssl rand -base64 32</code> で生成した強力な文字列

NEXTAUTH_SECRET のセキュリティ

`NEXTAUTH_SECRET` はユーザーのセッション（ログイン状態）を暗号化するための鍵です。この値が漏洩したり弱い値だと、以下のリスクがあります：

- **JWTトークンの偽造**: 攻撃者が任意のユーザーになりすませる
- **セッションハイジャック**: 他人のログイン状態を乗っ取れる
- **アカウント乗っ取り**: 管理者権限での不正操作が可能に

本番環境では必ず以下のコマンドで生成してください：

```
# Mac / Linux / Git Bash
openssl rand -base64 32
# 出力例: K7gNj3LBvQx8rPmT2sFw1dZcYhA5oRnE6qUiX9kJ0M=
```

カテゴリ3: アプリケーション設定（必須）

```
# ═══ アプリケーション ═══

# フロントエンドからアクセスするURL
# NEXT_PUBLIC_ プレフィックスがついた変数はブラウザ側でも使える
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

`NEXT_PUBLIC_APP_URL` は、アプリケーション内でURLを生成する際に使用されます。例えば、メール内のリンクやOGP画像のURLなどに利用されます。

カテゴリ4: ストレージ設定（画像アップロード）

BON-LOG では投稿画像やプロフィール画像のアップロード先を選択できます。

```
# ─── ストレージ設定 ───

# STORAGE_PROVIDER: 使用するストレージプロバイダー
#   - local: ローカルファイル保存（開発用・デフォルト）
#   - r2: Cloudflare R2（本番環境推奨・低コスト）
#   - supabase: Supabase Storage
STORAGE_PROVIDER=local
```

開発環境では `STORAGE_PROVIDER=local` を使います。画像は `public/uploads/` フォルダに保存されます。

本番環境では **Cloudflare R2** を推奨します。その場合、以下の追加設定が必要です:

```
# Cloudflare R2設定（STORAGE_PROVIDER=r2 の場合に必要）
STORAGE_PROVIDER=r2
R2_ACCOUNT_ID=your-account-id           # CloudflareアカウントID
R2_ACCESS_KEY_ID=your-access-key-id      # R2 APIトークンのアクセスキーID
R2_SECRET_ACCESS_KEY=your-secret-key     # R2 APIトークンのシークレットキー
R2_BUCKET_NAME=bonsai-uploads            # R2バケット名
R2_PUBLIC_URL=https://pub-xxx.r2.dev     # R2バケットの公開URL
```

変数名	取得場所	説明
<code>R2_ACCOUNT_ID</code>	Cloudflare ダッシュボード > アカウントID	Cloudflareアカウントの固有ID
<code>R2_ACCESS_KEY_ID</code>	Cloudflare > R2 > API トークン	S3互換APIのアクセスキー
<code>R2_SECRET_ACCESS_KEY</code>	Cloudflare > R2 > API トークン	S3互換APIのシークレットキー
<code>R2_BUCKET_NAME</code>	Cloudflare > R2 > バケット一覧	ファイルを保存するバケットの名前
<code>R2_PUBLIC_URL</code>	Cloudflare > R2 > バケット設定 > 公開アクセス	画像の公開配信URL

ストレージの動作イメージ:

開発環境 (local) :

画像アップロード → `public/uploads/post-images/xxx.jpg` → `localhost:3000/uploads/ ...`

本番環境 (Cloudflare R2) :

画像アップロード → Cloudflare R2 バケット → `https://pub-xxx.r2.dev/ ...`

カテゴリ5: Redis キャッシュ・レート制限

```
# ===== Redis (セッション・キャッシュ・レート制限) =====  
  
# Upstash Redis REST API (本番環境推奨)  
# https://upstash.com でデータベース作成後、REST URL/トークンを取得  
# UPSTASH_REDIS_REST_URL=https://xxx.upstash.io  
# UPSTASH_REDIS_REST_TOKEN=xxx
```

Redis は、BON-LOG で以下の用途に使用されます：

- **レート制限**: スпам防止（例: 1日20投稿まで）
- **キャッシュ**: よく使うデータの高速配信
- **セッション管理**: ログイン状態の管理補助

変数名	取得場所	説明
UPSTASH_REDIS_REST_URL	Upstash ダッシュボード > Database > REST API	Redis サーバーのHTTPエンドポイント
UPSTASH_REDIS_REST_TOKEN	Upstash ダッシュボード > Database > REST API	APIアクセス用トークン

開発環境では Redis は必須ではありません。 設定しなくても基本的な機能は動作します。レート制限などの一部機能が無効になりますが、開発には影響しません。

カテゴリ6: メール送信設定

```
# ===== メール送信設定 =====

# EMAIL_PROVIDER: 使用するメールプロバイダー
#   - console: コンソール出力 (開発用・デフォルト)
#   - resend: Resend (本番環境推奨・無料枠あり)
#   - azure: Azure Communication Services
EMAIL_PROVIDER=console

# Resend設定 (EMAIL_PROVIDER=resend の場合に必要)
# RESEND_API_KEY=re_XXXXXXXXXXXX
# EMAIL_FROM=BON-LOG <noreply@resend.dev>
```

開発環境では `EMAIL_PROVIDER=console` がおすすめです。メールの内容がターミナルのコンソールに出力されるため、実際にメールを送信せずに動作確認できます。

変数名	用途	開発環境	本番環境
<code>EMAIL_PROVIDER</code>	メール送信方法の選択	<code>console</code>	<code>resend</code>
<code>RESEND_API_KEY</code>	Resend の API キー	不要	<code>re_xxx ...</code>
<code>EMAIL_FROM</code>	送信元メールアドレス	不要	<code>BON-LOG <noreply@your-domain.com></code>

メール送信の用途 BON-LOG では以下の場面でメールを送信します:

- メールアドレスの確認 (新規登録時)
- パスワードリセット
- 重要な通知 (アカウントに関するセキュリティ通知等)

カテゴリ7: 全文検索設定

```
# ===== 検索設定 (PostgreSQL全文検索) =====  
  
# SEARCH_MODE: 使用する全文検索モード  
#   - like: LIKE検索 (デフォルト・設定不要)  
#   - trgm: pg_trgm使用 (Supabase等のクラウドDB対応・推奨)  
#   - bigm: pg_bigm使用 (日本語最適・要インストール)  
SEARCH_MODE=like
```

日本語の投稿を検索するための設定です。開発環境では `like` (部分一致検索) で十分ですが、本番環境では `trgm` (トリグラム検索) を使うと検索精度が向上します。

カテゴリ8: Stripe 決済設定 (有料会員機能)

```
# ===== Stripe (有料会員機能) =====  
  
# Stripeダッシュボードで取得: https://dashboard.stripe.com/apikeys  
# STRIPE_SECRET_KEY=sk_test_xxx  
# STRIPE_WEBHOOK_SECRET=whsec_xxx  
# NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_xxx  
  
# Stripe Price ID (商品の価格ID)  
# Stripeダッシュボード > 商品 > 価格IDをコピー  
# STRIPE_PRICE_ID_MONTHLY=price_xxx  
# STRIPE_PRICE_ID_YEARLY=price_xxx
```

BON-LOG のプレミアム会員機能で使用する決済設定です。開発環境では設定不要です。有料会員機能を実装する段階で設定します。

変数名	プレフィックス	用途	注意点
STRIPE_SECRET_KEY	sk_test_ / sk_live_	サーバー側で決済処理に使用	絶対に公開しない
STRIPE_WEBHOOK_SECRET	whsec_	Stripe からの通知を検証	絶対に公開しない
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	pk_test_ / pk_live_	クライアント側で決済フォーム表示	公開可能
STRIPE_PRICE_ID_MONTHLY	price_	月額プランの価格ID	Stripeダッシュボードで確認
STRIPE_PRICE_ID_YEARLY	price_	年額プランの価格ID	Stripeダッシュボードで確認

テストモードと本番モードの違い

Stripe にはテストモードと本番モードがあります。

- **テストモード** (sk_test_): 実際の課金は行われず。開発・テスト用。
- **本番モード** (sk_live_): 実際に課金される。本番環境用。

開発中は必ずテストモードのキーを使ってください。テストモードでは、テスト用のクレジットカード番号 (4242 4242 4242 4242) で決済をテストできます。

カテゴリ9: Sentry エラー監視

```
# === エラー監視 (Sentry) ===

# https://sentry.io でプロジェクト作成後、DSNを取得
# SENTRY_DSN=https://xxx@xxx.ingest.sentry.io/xxx
# NEXT_PUBLIC_SENTRY_DSN=https://xxx@xxx.ingest.sentry.io/xxx
# SENTRY_AUTH_TOKEN=sentrys_xxx
```

Sentry はアプリケーションで発生したエラーを自動的に収集・通知するサービスです。開発環境では設定不要ですが、本番環境では強く推奨します。

変数名	用途	注意点
<code>SENTRY_DSN</code>	サーバー側のエラー送信先	サーバーサイドでのみ使用
<code>NEXT_PUBLIC_SENTRY_DSN</code>	クライアント側のエラー送信先	ブラウザで発生したエラーの送信用
<code>SENTRY_AUTH_TOKEN</code>	ソースマップのアップロード用	ビルド時に使用。 絶対に公開しない

なぜ SENTRY_DSN と NEXT_PUBLIC_SENTRY_DSN の2つがあるのか？

`SENTRY_DSN` はサーバー側でのみ使え、`NEXT_PUBLIC_SENTRY_DSN` はブラウザ側でも使えます。同じ値を設定しますが、ブラウザでもエラーを監視するために `NEXT_PUBLIC_` 付きの変数が必要です。DSN (Data Source Name) 自体は公開しても大きなリスクはありません (不正にイベントを送信されるリスクはありますが、データの読み取りはできません)。

カテゴリ10: 広告設定

```
# ===== 広告設定 =====

# NEXT_PUBLIC_AD_PROVIDER: 使用する広告プロバイダー
#   - ninja: 忍者AdMax (デフォルト・審査不要)
#   - adsense: Google AdSense (審査通過後に切り替え)
NEXT_PUBLIC_AD_PROVIDER=ninja

# --- 忍者AdMax ---
NEXT_PUBLIC_NINJA_AD_ID_SIDEBar=
NEXT_PUBLIC_NINJA_AD_ID_INFEEd=
NEXT_PUBLIC_NINJA_AD_ID_POST_DETAIL=

# --- Google AdSense ---
# NEXT_PUBLIC_ADSENSE_CLIENT_ID=ca-pub-xxxxxxxxxxxxxxxxxx
# NEXT_PUBLIC_ADSENSE_SLOT_SIDEBar=xxxxxxxxxx
# NEXT_PUBLIC_ADSENSE_SLOT_INFEEd=xxxxxxxxxx
# NEXT_PUBLIC_ADSENSE_SLOT_POST_DETAIL=xxxxxxxxxx
```

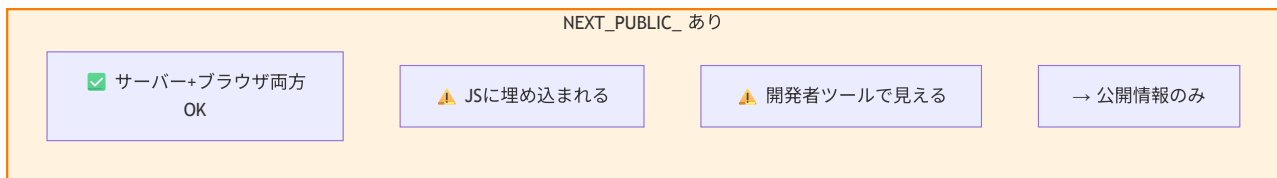
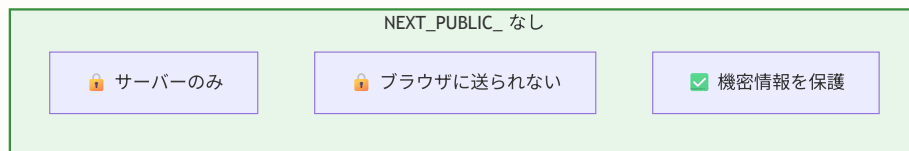
広告表示の設定です。**開発環境では空のままでも動作します。** 広告を表示したい場合に、各広告プロバイダーで広告枠を作成して ID を設定します。

広告変数がすべて `NEXT_PUBLIC_` なのはなぜ？

広告の表示はブラウザ側（クライアントサイド）で行われるため、すべて `NEXT_PUBLIC_` プレフィックスが必要です。広告IDは公開情報であるため、ブラウザに含まれても問題ありません。

NEXT_PUBLIC_ プレフィックスの仕組み（重要）

Next.js では、環境変数のセキュリティを保つために**重要なルール**があります。



BON-LOG の全環境変数を分類すると：

レベル	変数名	理由
サーバーのみ	DATABASE_URL	データベースのパスワードを含む
サーバーのみ	DIRECT_URL	データベースのパスワードを含む
サーバーのみ	NEXTAUTH_SECRET	セッション暗号化の鍵
サーバーのみ	R2_SECRET_ACCESS_KEY	ストレージの秘密鍵
サーバーのみ	R2_ACCESS_KEY_ID	ストレージのアクセスキー
サーバーのみ	STRIPE_SECRET_KEY	決済処理の秘密鍵
サーバーのみ	STRIPE_WEBHOOK_SECRET	Webhook検証用
サーバーのみ	UPSTASH_REDIS_REST_TOKEN	Redisのアクセストークン
サーバーのみ	RESEND_API_KEY	メール送信のAPIキー
サーバーのみ	SENTRY_AUTH_TOKEN	ソースマップアップロード用
公開可能	NEXT_PUBLIC_APP_URL	アプリのURL（公開情報）
公開可能	NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	決済フォーム表示用（公開前提）
公開可能	NEXT_PUBLIC_SENTRY_DSN	エラー送信先（公開しても安全）
公開可能	NEXT_PUBLIC_AD_PROVIDER	広告設定（公開情報）
公開可能	NEXT_PUBLIC_NINJA_AD_ID_*	広告枠ID（公開情報）
公開可能	NEXT_PUBLIC_ADSENSE_*	広告設定（公開情報）

ここが重要！

間違った例: データベースのパスワードに `NEXT_PUBLIC_` をつけてしまう

```
# 絶対にやってはいけない！
NEXT_PUBLIC_DATABASE_URL="postgresql://postgres:password@ ... "
# → パスワードがブラウザのソースコードから誰でも見られてしまう！
```

正しい例: データベース接続情報は `NEXT_PUBLIC_` なしで定義する

```
# 正しい（サーバーサイドでのみアクセス可能）
DATABASE_URL="postgresql://postgres:password@ ... "
```

開発環境の最小構成

開発を始めるにあたって、最低限必要な環境変数は以下の通りです。

```
# === 開発環境 最小構成 ===
# この3つだけあれば開発を始められます

DATABASE_URL="postgresql://postgres:postgres@localhost:5432/bonsai_sns"
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=development-secret-key-change-in-production
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

残りの環境変数は、各機能を実装・利用する段階で追加していけば問題ありません。

`.env.local`はGitに含めない

`.env.local` には機密情報が含まれるため、**絶対にGitにコミットしてはいけません**。

Next.jsのプロジェクトでは、`.gitignore` ファイルに `.env*.local` が最初から含まれているため、自動的にGitの管理対象外になります。

```
# .gitignore の中身を確認すると、以下のような行がある
# .env*.local
# ↑ これにより .env.local は自動的にGitに含まれない
```

注意！ もし誤って `.env.local` をコミットしてしまったら すぐに以下を実行してください:

1. `.env.local` 中の秘密鍵やパスワードをすべて変更する
2. Gitの履歴からファイルを削除する
3. 一度GitHubにプッシュした機密情報は「漏洩した」とみなし、必ず変更すること

環境変数の安全管理方法

環境変数を安全に管理するためのベストプラクティスを紹介します。

1. `.env.local.example` をチームで共有する

`.env.local` は Git に含めませんが、`.env.local.example` (サンプルファイル) は Git に含めて共有します。これにより、チームメンバーが必要な環境変数を把握できます。

```
# .env.local.example には実際の値ではなくプレースホルダーを記載
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/bonsai_sns"
NEXTAUTH_SECRET=your-development-secret-key-change-in-production
```

2. 本番環境ではホスティングサービスの環境変数設定を使う

本番環境 (Vercel等) では、`.env.local` ファイルではなく、ホスティングサービスのダッシュボードで環境変数を設定します。

Vercel での環境変数設定:

Vercel ダッシュボード > プロジェクト > Settings > Environment Variables

名前: DATABASE_URL

値: postgresql://postgres.[ref]:[pass]@aws-0-...

環境: Production, Preview

→ サーバー上のファイルに書く必要がなく、暗号化されて管理される

3. シークレットのローテーション (定期的な変更)

本番環境のシークレット（`NEXTAUTH_SECRET` や API キー）は、セキュリティのために定期的に変更することが推奨されます。

推奨される変更頻度：

`NEXTAUTH_SECRET` → 半年～1年ごと（変更時に全ユーザーが再ログイン必要）
`STRIPE_SECRET_KEY` → Stripe ダッシュボードでローテーション
`RESEND_API_KEY` → 漏洩の疑いがある場合に即座に変更
`R2_SECRET_ACCESS_KEY` → 半年～1年ごと

4. 環境変数の命名規則

環境変数の命名規則：

`UPPER_SNAKE_CASE` を使用 （例：`DATABASE_URL`、`NEXTAUTH_SECRET`）
 わかりやすい名前にする （例：`STRIPE_SECRET_KEY`、`EMAIL_PROVIDER`）
`NEXT_PUBLIC_` は公開情報のみ（例：`NEXT_PUBLIC_APP_URL`）

本番デプロイ時のチェックリスト

本番環境にデプロイする際は、以下の環境変数が正しく設定されているか確認してください。

必須：

- [] `DATABASE_URL` (Supabase の接続URL)
- [] `DIRECT_URL` (Supabase の直接接続URL)
- [] `NEXTAUTH_URL` (本番URL: `https://your-domain.com`)
- [] `NEXTAUTH_SECRET` (`openssl rand -base64 32` で生成した強力な値)
- [] `NEXT_PUBLIC_APP_URL` (本番URL: `https://your-domain.com`)

推奨：

- [] `STORAGE_PROVIDER=r2` + `R2_*` 変数 (画像アップロード)
- [] `UPSTASH_REDIS_REST_URL` + `TOKEN` (レート制限)
- [] `EMAIL_PROVIDER=resend` + `RESEND_API_KEY` (メール送信)
- [] `SENTRY_DSN` + `NEXT_PUBLIC_SENTRY_DSN` (エラー監視)

任意：

- [] `STRIPE_*` 変数 (有料会員機能を使う場合)
- [] `NEXT_PUBLIC_AD_*` (広告を表示する場合)

理解度チェック

- ☐ `.env.local` ファイルを作成できましたか？

- ☐ `DATABASE_URL` の各部分（ユーザー名、パスワード、ホスト、ポート、データベース名）が何を指しているか分かりますか？
- ☐ `.env.local` をGitにコミットしてはいけない理由を説明できますか？
- ☐ `NEXT_PUBLIC_` プレフィックスの意味を説明できますか？
- ☐ `NEXTAUTH_SECRET` が弱いと何が起こるか説明できますか？
- ☐ 開発環境で最低限必要な環境変数を4つ言えますか？
- ☐ `STRIPE_SECRET_KEY` に `NEXT_PUBLIC_` をつけてはいけない理由を説明できますか？

1.10 ディレクトリ構成の準備

このセクションで学ぶこと

- BON-LOGのディレクトリ構成の全体像
- 各フォルダの役割
- ディレクトリの作成方法

ディレクトリ構成の作成

BON-LOGの開発で使用するフォルダを作成します。

```

# 認証関連のページ用フォルダ
mkdir -p app/(auth)/login      # ログインページ
mkdir -p app/(auth)/register   # 新規登録ページ

# メインアプリのページ用フォルダ
mkdir -p app/(main)/feed      # タイムライン（投稿一覧）
mkdir -p app/(main)/posts     # 投稿詳細
mkdir -p app/(main)/users     # ユーザープロフィール

# API用フォルダ
mkdir -p app/api/auth         # 認証API

# コンポーネント用フォルダ
mkdir -p components/ui        # 基本UIコンポーネント（ボタン、カードなど）
mkdir -p components/auth      # 認証関連コンポーネント（ログインフォームなど）
mkdir -p components/post      # 投稿関連コンポーネント（投稿カードなど）
mkdir -p components/user      # ユーザー関連コンポーネント（プロフィールカードなど）
mkdir -p components/layout    # レイアウトコンポーネント（ヘッダー、サイドバーなど）
mkdir -p components/common    # 共通コンポーネント（ローディング、エラー表示など）

# ビジネスロジック用フォルダ
mkdir -p lib/actions          # Server Actions（サーバー側の処理）

# その他
mkdir -p types                # TypeScript型定義ファイル
mkdir -p hooks                # カスタムフック
mkdir -p public                # 静的ファイル（画像、アイコンなど）

```

Windowsで `mkdir -p` がエラーになる場合 PowerShellでは `mkdir -p` の代わりに `New-Item -ItemType Directory -Path` を使うか、Git Bash を使ってください。VS Code のターミナルを Git Bash に変更する方法:

1. VS Code のターミナルで、右上の「+」の横にあるドロップダウン（`v` マーク）をクリック
2. 「Select Default Profile」を選択
3. 「Git Bash」を選択

ディレクトリ構成の全体像

```

bonsai-sns-project/
├─ app/                                # ページとAPIルート
│  ├─ (auth)/                          # 認証ページ (ログイン・登録)
│  │  ├─ login/                        # /login でアクセスできるページ
│  │  │  └─ page.tsx                  # ログインページの本体
│  │  └─ register/                    # /register でアクセスできるページ
│  │     └─ page.tsx                  # 新規登録ページの本体
│  │
│  ├─ (main)/                          # メインアプリ (認証必須のページ)
│  │  ├─ feed/                        # /feed タイムライン
│  │  ├─ posts/                       # /posts 投稿関連
│  │  │  └─ [id]/                     # /posts/xxx 投稿詳細 (動的ルート)
│  │  └─ users/                       # /users ユーザー関連
│  │     └─ [id]/                     # /users/xxx ユーザープロフィール
│  │
│  ├─ admin/                          # 管理者ダッシュボード
│  ├─ api/                            # APIエンドポイント
│  │  └─ auth/                        # 認証API
│  │     └─ [...nextauth]/           # NextAuth.js のルート
│  │
│  └─ layout.tsx                      # 全ページ共通のレイアウト
│  └─ page.tsx                        # トップページ
│  └─ globals.css                     # グローバルCSS
│
├─ components/                        # 再利用可能なUIコンポーネント
│  ├─ ui/                            # 基本コンポーネント (Button, Card, Input ...)
│  ├─ auth/                          # 認証関連 (LoginForm, RegisterForm ...)
│  ├─ post/                          # 投稿関連 (PostCard, PostForm, LikeButton ...)
│  ├─ user/                          # ユーザー関連 (UserCard, FollowButton ...)
│  ├─ layout/                        # レイアウト (Header, Sidebar, BottomNav ...)
│  └─ common/                        # 共通 (Loading, ErrorMessage, Avatar ...)
│
├─ lib/                              # ビジネスロジック・ユーティリティ
│  ├─ db.ts                          # Prismaクライアントの設定
│  ├─ auth.ts                        # NextAuth.jsの設定
│  ├─ actions/                       # Server Actions
│  │  ├─ post.ts                     # 投稿関連の処理
│  │  ├─ user.ts                     # ユーザー関連の処理
│  │  └─ auth.ts                     # 認証関連の処理
│  └─ utils/                          # ユーティリティ関数
│
├─ prisma/                           # データベース
│  ├─ schema.prisma                  # DBスキーマ定義
│  └─ migrations/                    # マイグレーションファイル
│
└─ types/                            # TypeScript型定義

```

```

├── next-auth.d.ts          # NextAuth.jsの型拡張
├── hooks/                  # カスタムフック
├── public/                 # 静的ファイル（画像、フォントなど）
├── __tests__/              # テストファイル
├── .env.local              # 環境変数（Gitに含めない）
├── docker-compose.yml      # Docker設定
├── package.json            # プロジェクト設定
├── tsconfig.json           # TypeScript設定
└── tailwind.config.ts      # Tailwind CSS設定

```

コラム: Next.js の (フォルダ名) とは？ フォルダ名を丸括弧 () で囲むと、**Route Group**（ルートグループ）になります。これはURLには影響せず、ファイルを整理するためだけに使います。

例: `app/(auth)/login/page.tsx` のURLは `/login` (`/(auth)` は含まれない)

BON-LOGでは以下のように使い分けます:

- `(auth)` - 認証ページ専用のレイアウト（シンプルなデザイン）
- `(main)` - メインアプリ専用のレイアウト（3カラムデザイン）

理解度チェック

- ☐ `app/` フォルダの役割を説明できますか？
- ☐ `components/` フォルダの役割を説明できますか？
- ☐ `lib/` フォルダの役割を説明できますか？
- ☐ Route Group（丸括弧のフォルダ名）の意味を説明できますか？

1.11 確認チェックリスト

以下がすべて完了していることを確認してください。チェックが全部つけば、開発を始める準備は万全です！

- ☐ Node.js v20以上がインストールされている (`node --version` で確認)
- ☐ npm がインストールされている (`npm --version` で確認)

- ☐ VS Codeがインストールされ、推奨拡張機能が入っている
- ☐ VS Codeの設定（settings.json）を編集した
- ☐ Gitがインストールされ、ユーザー設定が完了している（`git --version` で確認）
- ☐ GitHubアカウントを作成した
- ☐ SSHキーをGitHubに登録した
- ☐ Docker Desktopがインストールされている（`docker --version` で確認）
- ☐ Next.jsプロジェクトが作成され、`npm run dev` で起動できる
- ☐ ブラウザで <http://localhost:3000> にアクセスできる
- ☐ `docker-compose.yml` が作成されている
- ☐ PostgreSQLがDockerで起動できる（`docker compose up -d postgres`）
- ☐ `.env.local` ファイルが作成されている
- ☐ ディレクトリ構成が準備されている

よくあるトラブルと解決法

エラー	原因	解決法
<code>EADDRINUSE: port 3000</code>	ポート3000が既に使用中	別のターミナルでサーバーが動いていないか確認。 <code>npx kill-port 3000</code> で強制終了
<code>npm install</code> が失敗	Node.jsバージョン不一致 やネットワーク問題	<code>node -v</code> でバージョン確認（18以上推奨）。 <code>npm cache clean --force</code> 後に再実行
<code>prisma generate</code> が失敗	データベースに接続できない	<code>.env.local</code> の <code>DATABASE_URL</code> を確認。Dockerが起動しているか <code>docker ps</code> で確認
サーバーが止まらない	Ctrl+Cが効かない	<code>Ctrl+C</code> を2回押す。それでもダメなら新しいターミナルで <code>npx kill-port 3000</code>

サーバーの停止方法: ターミナルで `Ctrl+C`（Macは `Control+C`）を押すとサーバーが停止します。

1.12 よくあるトラブル（まとめ）

ここでは、この章全体でよくある問題とその解決方法をまとめます。

全般

Q: コマンドを実行しても「command not found」「認識されていません」と表示される

A: そのツールがインストールされていないか、PATHが通っていません。

1. ツールが正しくインストールされているか確認
2. ターミナルを閉じて開き直す（環境変数の再読み込み）
3. PCを再起動する
4. それでもダメなら、ツールを再インストール

Q: エラーメッセージが英語で読めない

A: エラーメッセージはGoogle検索やDeepLで翻訳してみましょう。多くの場合、エラーメッセージの最初の1〜2行に原因が書かれています。「Error:」や「ERR!」の直後の文章が特に重要です。

Q: セキュリティソフトがインストールをブロックする

A: 開発ツール（Node.js、Git、Docker）は安全なソフトウェアです。セキュリティソフトの一時的な除外設定や、「許可」をクリックして進めてください。

環境別の注意点

Windows特有の問題

- PowerShellの実行ポリシーでスクリプトがブロックされる場合: PowerShellを管理者として起動し、`Set-ExecutionPolicy RemoteSigned` を実行
- パスの区切り文字が `\` （バックスラッシュ）であることに注意
 - ただしターミナルでは `/` も使える場合が多い

Mac特有の問題

- 「開発元を検証できないため開けません」と表示される場合: システム設定 → プライバシーとセキュリティ → 「このまま開く」をクリック
- M1/M2/M3/M4 Mac の場合、一部のツールは Rosetta 2 が必要な場合がある

演習問題

基礎問題

問題1: ターミナルの基本操作

ターミナルで以下の操作を実行してください。

1. 現在いるフォルダのパスを表示する
2. デスクトップに移動する
3. デスクトップの中身を一覧表示する
4. 元のフォルダに戻る

問題2: Node.jsの動作確認

ターミナルで `node -e "console.log('Hello, BON-LOG!')"` を実行し、出力を確認してください。

コマンドの解説

- `node` - Node.jsを実行するコマンド
- `-e` - 続く文字列をJavaScriptとして実行するオプション
- `"console.log('Hello, BON-LOG!')"` - 実行するJavaScriptのコード

問題3: Dockerコンテナの確認

`docker compose up -d postgres` でPostgreSQLを起動した後、`docker compose ps` で状態を確認してください。どのような出力が表示されますか？STATUSの欄に何と書かれていますか？

応用問題

問題4: 環境変数の理解

`.env.local` ファイルはなぜGitにコミットしてはいけないのでしょうか？ 3つ以上の理由を考えてみましょう。

▶ 回答例

問題5: package.jsonの理解

プロジェクトルートにある `package.json` ファイルを VS Code で開いて、以下の質問に教えてください。

1. プロジェクト名は何ですか？
2. `scripts` セクションにはどんなコマンドが定義されていますか？
3. `dependencies` セクションにはどんなパッケージが含まれていますか？

チャレンジ問題

問題6: GitHubリポジトリの作成

GitHubにリモートリポジトリを作成し、ローカルプロジェクトと連携させてください。

ヒント:

1. GitHubで「New repository」をクリック
2. リポジトリ名を入力して作成
3. ターミナルで以下を実行:

```
git remote add origin git@github.com:あなたのユーザー名/bonsai-sns-project.git
git push -u origin main
```

問題7: 開発サーバーでの編集

1. `npm run dev` で開発サーバーを起動
2. `app/page.tsx` を VS Code で開く
3. ページの内容を「BON-LOG へようこそ！」に変更して保存
4. ブラウザで変更が反映されることを確認

ヒント: Next.js の開発サーバーは「ホットリロード」機能があり、 ファイルを保存するだけで自動的にブラウザの表示が更新されます。

1.13 環境変数設定 - 実践ガイド

このセクションで学ぶこと

- `.env.local.example` ファイルの全体構成と読み方
- 各環境変数カテゴリの実践的な設定手順
- 開発フェーズごとに必要な環境変数の段階的な追加方法
- トラブルシューティング: 環境変数が反映されない場合の対処法

`.env.local.example` の全体構成

プロジェクトルートにある `.env.local.example` は、BON-LOG が使用する全環境変数のテンプレートです。このファイルは大きく **10のカテゴリ** に分かれています。以下に、各カテゴリの概要と、どのタイミングで設定が必要になるかを一覧にします。

`.env.local.example` の構成マップ:

カテゴリ	開発初期	機能実装時	本番デプロイ
1. データベース (PostgreSQL)	✅ 必須	-	✅ 必須
2. NextAuth.js (認証)	✅ 必須	-	✅ 必須
3. アプリケーション	✅ 必須	-	✅ 必須
4. ストレージ (画像)	-	画像機能	✅ 必須
5. Redis (キャッシュ)	-	制限機能	★ 推奨
6. メール送信	-	認証機能	✅ 必須
7. 全文検索	-	検索機能	★ 推奨
8. Stripe (決済)	-	課金機能	💡 任意
9. Sentry (エラー監視)	-	-	★ 推奨
10. 広告設定	-	収益化	💡 任意

- **✅ 必須** = その段階で設定しないと動作しない
- **★ 推奨** = 設定しなくても動くが、設定を強く推奨
- **💡 任意** = その機能を使う場合のみ必要

DATABASE_URL の実践的な設定

DATABASE_URL は、アプリケーションがデータベースに接続するための最も重要な環境変数です。前のセクション（1.8 PostgreSQLの起動）で作成した Docker コンテナに接続するための URL を設定します。

```
# 開発環境: Docker Compose で起動した PostgreSQL に接続
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/bonsai_sns"
```

接続テスト方法:

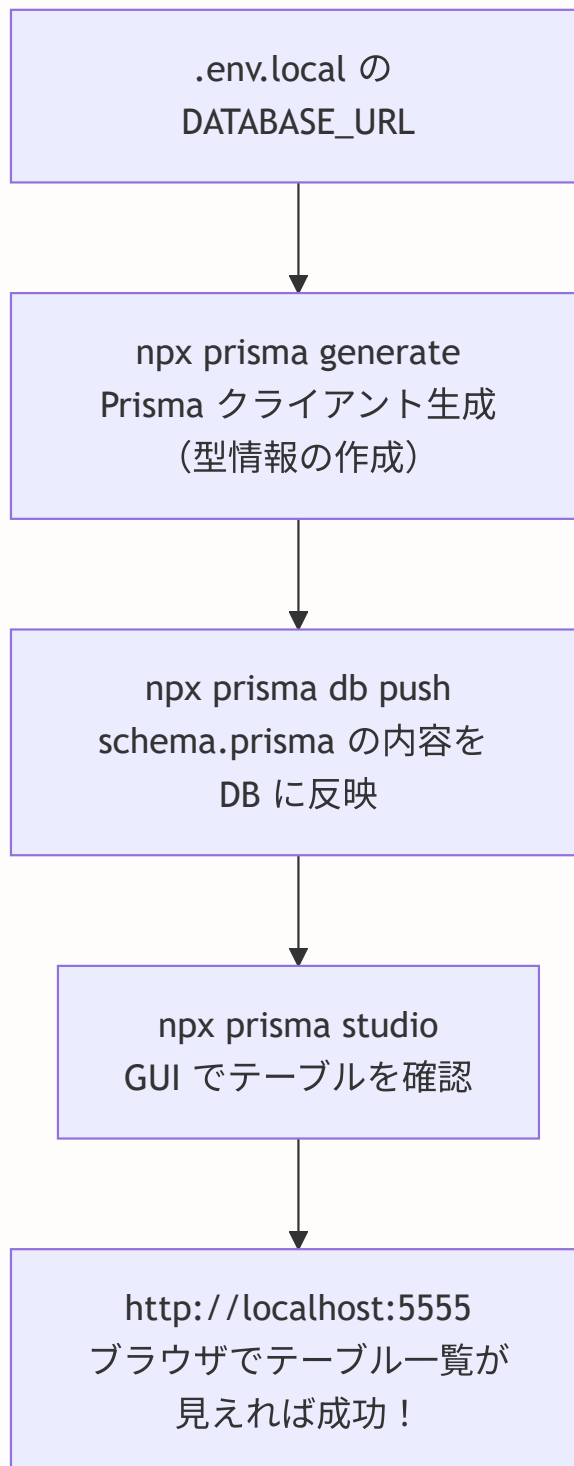
環境変数を設定した後、実際にデータベースに接続できるか確認しましょう。

```
# 1. PostgreSQL コンテナが起動していることを確認
docker compose ps
# STATUS が "Up" であること

# 2. Prisma クライアントを生成
npx prisma generate

# 3. データベースにスキーマを反映（テーブルを作成）
npx prisma db push

# 4. Prisma Studio でデータベースの中身を確認
npx prisma studio
# ブラウザで http://localhost:5555 が開き、テーブル一覧が表示されれば成功
```



よくあるエラー: P1001: Can't reach database server

このエラーは、PostgreSQL に接続できないことを意味します。以下を確認してください:

1. Docker コンテナが起動しているか (`docker compose ps`)
2. DATABASE_URL のポート番号が正しいか (デフォルトは 5432)
3. DATABASE_URL のユーザー名・パスワードが docker-compose.yml と一致しているか
4. Docker Desktop が起動しているか

NEXTAUTH_URL と NEXTAUTH_SECRET の設定

認証機能の設定は、ユーザーのログイン・登録を実装する際に必要です。開発環境では以下のように設定してください。

```
# ベースURL: 開発サーバーのアドレス
NEXTAUTH_URL=http://localhost:3000

# 秘密鍵: 開発環境では任意の文字列でOK
NEXTAUTH_SECRET=development-secret-key-change-in-production
```

本番環境用の NEXTAUTH_SECRET を生成する:

```
# Mac / Linux / Git Bash の場合
openssl rand -base64 32
# 出力例: K7gNj3LBvQx8rPmT2sFw1dZcYhA5oRnE6qUiX9kJ0M=

# Windows PowerShell の場合 (openssl が無い場合)
[Convert]::ToBase64String((1..32 | ForEach-Object { Get-Random -Maximum 256 }) -as [byte[]])
# 出力例: aB3kLmNoPqRsTuVwXyZ1234567890+/-

# Node.js を使う方法 (OS問わず)
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```


注意！ `NEXTAUTH_URL` は末尾にスラッシュ (`/`) をつけないでください。

```
# 正しい
NEXTAUTH_URL=http://localhost:3000

# 間違い（末尾のスラッシュが問題を起こす場合がある）
NEXTAUTH_URL=http://localhost:3000/
```

NEXT_PUBLIC_APP_URL の設定

```
# アプリケーションの公開URL
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

この変数は、アプリケーション内でフルURLを組み立てる必要がある場面で使用されます。

```
// 使用例：メール内のリンクURL
const resetUrl = `${process.env.NEXT_PUBLIC_APP_URL}/password-reset?token=${token}`

// 使用例：OGP画像のURL
const ogImageUrl = `${process.env.NEXT_PUBLIC_APP_URL}/api/og?title=${title}`

// 使用例：共有URL
const shareUrl = `${process.env.NEXT_PUBLIC_APP_URL}/posts/${postId}`
```

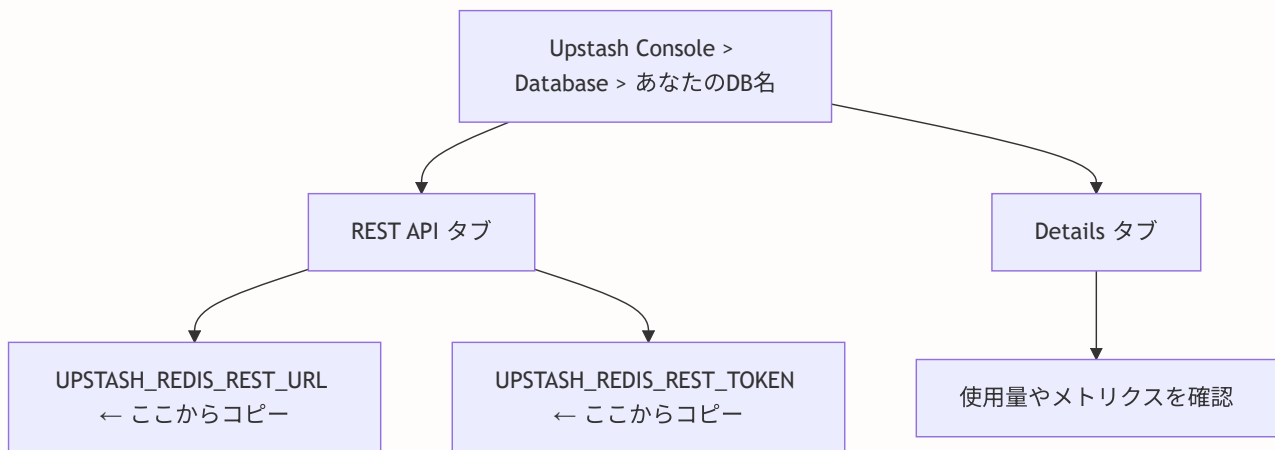
Upstash Redis の設定

Redis はレート制限（スパム防止）やキャッシュに使用されます。開発初期は設定しなくても基本機能は動作しますが、以下の機能を使う際に必要になります。

```
# Upstash Redis REST API
# https://upstash.com で無料アカウントを作成し、データベースを作成後に取得
UPSTASH_REDIS_REST_URL=https://xxx.upstash.io
UPSTASH_REDIS_REST_TOKEN=AXxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Upstash アカウントの作成手順:

1. <https://upstash.com> にアクセスし、アカウントを作成（GitHub連携推奨）
2. ダッシュボードで「Create Database」をクリック
3. リージョンを選択（日本に近い `ap-northeast-1` 推奨）
4. 作成後、「REST API」タブで URL と Token をコピー



Redis が未設定の場合の動作:

Redis 関連の環境変数を設定しない場合でも、アプリケーション自体は起動・動作します。ただし以下の機能が制限されます:

- レート制限が無効（1日の投稿数制限などが機能しない）
- サーバーサイドキャッシュが使えない
- 一部のセキュリティ機能（ブルートフォース保護等）が動作しない

Cloudflare R2 ストレージの設定

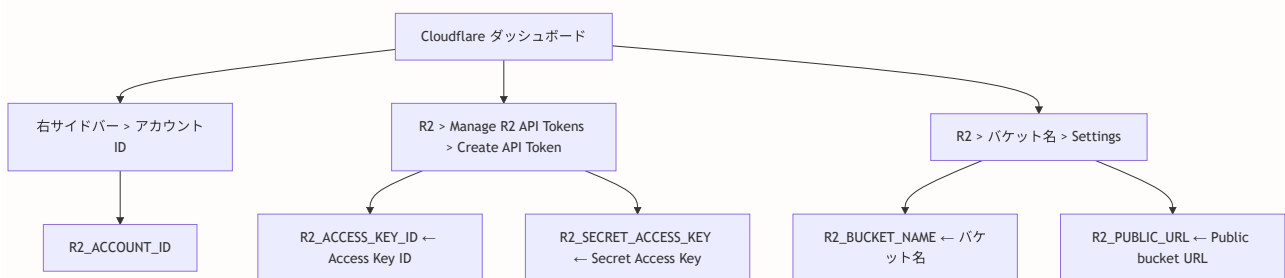
画像アップロード機能を使う際のストレージ設定です。開発環境ではローカルストレージ（`STORAGE_PROVIDER=local`）を使えるため、R2 の設定は本番デプロイ時に必要になります。

```
# 開発環境: ローカルファイルシステムに保存 (設定不要)
STORAGE_PROVIDER=local
```

```
# 本番環境: Cloudflare R2 に保存
STORAGE_PROVIDER=r2
R2_ACCOUNT_ID=your-account-id
R2_ACCESS_KEY_ID=your-access-key-id
R2_SECRET_ACCESS_KEY=your-secret-access-key
R2_BUCKET_NAME=bonsai-uploads
R2_PUBLIC_URL=https://pub-xxx.r2.dev
```

Cloudflare R2 の設定手順 (本番環境用) :

1. <https://dash.cloudflare.com> にログイン
2. 左メニューの「R2」をクリック
3. 「Create bucket」でバケットを作成 (名前: `bonsai-uploads` 等)
4. 「Settings」タブで公開アクセスを有効化
5. 「Manage R2 API Tokens」で API トークンを作成
6. 取得した情報を環境変数に設定



Resend メール送信の設定

メール送信は、ユーザー登録時のメールアドレス確認やパスワードリセットで使用されます。

```
# 開発環境: コンソール出力 (実際のメール送信なし)
EMAIL_PROVIDER=console
```

```
# 本番環境: Resend を使用
EMAIL_PROVIDER=resend
RESEND_API_KEY=re_xxxxxxxxxxxxx
EMAIL_FROM=BON-LOG <noreply@your-domain.com>
```

開発環境で `EMAIL_PROVIDER=console` にすると：メール送信の代わりに、ターミナルのコンソールにメールの内容が表示されます。実際にメールが送信されないため、開発中に何度もテストする際に便利です。

```
[Console Email] To: user@example.com
Subject: メールアドレスの確認
Body: 以下のリンクをクリックして、メールアドレスを確認してください ...
```

Stripe 決済の設定

プレミアム会員機能（有料プラン）を実装する際に必要です。開発初期は設定不要で、課金機能の実装段階で追加します。

```
# テストモードのキー（開発・テスト用）
STRIPE_SECRET_KEY=sk_test_XXXXXXXXXXXXXXXXXXXX
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXXXXXXXXXXXXXXX
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_XXXXXXXXXXXXXXXXXXXX

# 商品の価格ID
STRIPE_PRICE_ID_MONTHLY=price_XXXXXXXXXXXXXXXXXXXX
STRIPE_PRICE_ID_YEARLY=price_XXXXXXXXXXXXXXXXXXXX
```

テストキーと本番キーの見分け方:

- テストキー: `sk_test_`、`pk_test_` で始まる
- 本番キー: `sk_live_`、`pk_live_` で始まる

開発中は**必ずテストキー**を使用してください。テストモードでは、テスト用カード番号 `4242 4242 4242 4242` で決済テストができます。

Sentry エラー監視の設定

本番環境でのエラーを自動検知・通知するためのサービスです。開発環境では設定不要ですが、本番運用時には強く推奨します。

```
# サーバーサイドのエラー送信先
SENTRY_DSN=https://xxx@xxx.ingest.sentry.io/xxx

# クライアントサイドのエラー送信先（同じ値を設定）
NEXT_PUBLIC_SENTRY_DSN=https://xxx@xxx.ingest.sentry.io/xxx

# ソースマップアップロード用トークン
SENTRY_AUTH_TOKEN=sentrys_XXXXXXXXXXXXXXXXXXXX
```

SENTRY_DSN と NEXT_PUBLIC_SENTRY_DSN に同じ値を設定する理由:

Sentry DSN はエラーの送信先を示す URL です。同じ値ですが、2つの環境変数が必要な理由は:

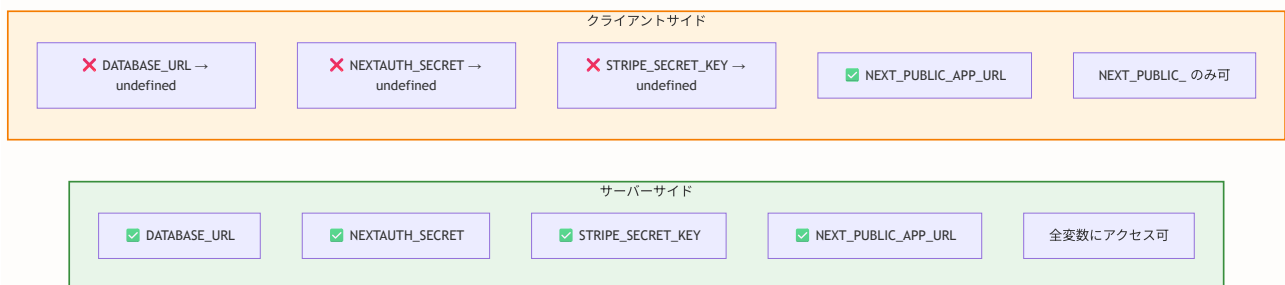
- **SENTRY_DSN**: サーバーサイド (Server Components、API Routes) で使用
- **NEXT_PUBLIC_SENTRY_DSN**: ブラウザ側 (Client Components) で使用

ブラウザで使う環境変数には **NEXT_PUBLIC_** プレフィックスが必須なため、同じ値を2つの変数名で設定する必要があります。

NEXT_PUBLIC_ プレフィックスの仕組み (補足)

Next.js の環境変数には重要なセキュリティルールがあります。この仕組みを正しく理解することは、本番運用において非常に重要です。

環境変数のアクセス制御:



BON-LOG の環境変数を安全性で整理すると:

- 絶対に `NEXT_PUBLIC_` をつけてはいけない変数（機密情報）：
 - `DATABASE_URL` - DBのパスワードを含む
 - `NEXTAUTH_SECRET` - セッション暗号化の鍵
 - `STRIPE_SECRET_KEY` - 決済処理の秘密鍵
 - `STRIPE_WEBHOOK_SECRET` - Webhook検証用
 - `R2_SECRET_ACCESS_KEY` - ストレージの秘密鍵
 - `RESEND_API_KEY` - メール送信のAPIキー
 - `UPSTASH_REDIS_REST_TOKEN` - Redisのアクセストークン
 - `SENTRY_AUTH_TOKEN` - ソースマップアップロード用
- `NEXT_PUBLIC_` をつけてよい変数（公開情報）：
 - `NEXT_PUBLIC_APP_URL` - アプリのURL
 - `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` - 決済フォーム表示用
 - `NEXT_PUBLIC_SENTRY_DSN` - エラー送信先
 - `NEXT_PUBLIC_AD_PROVIDER` - 広告設定
 - `NEXT_PUBLIC_NINJA_AD_ID_*` - 広告枠ID
 - `NEXT_PUBLIC_ADSENSE_*` - AdSense設定

環境変数が反映されない場合のトラブルシューティング

環境変数を変更しても反映されない場合、以下の手順で確認してください。

```
# 1. 開発サーバーを再起動する（必須！）
# 環境変数はサーバー起動時に読み込まれるため、変更後は再起動が必要
# Ctrl + C でサーバーを停止してから：
npm run dev

# 2. .env.local ファイルの配置場所を確認
# プロジェクトルート直下にあること（app/ の中ではない！）
# 正しい： bonsai-sns-project/.env.local
# 間違い： bonsai-sns-project/app/.env.local

# 3. ファイル名を確認
# 正しい： .env.local（ドットで始まる）
# 間違い： env.local（ドットがない）
# 間違い： .env.local.txt（拡張子がついている）

# 4. 環境変数の値を確認（デバッグ用）
# Server Component や API Route で以下を一時的に追加して確認：
# console.log('DB URL:', process.env.DATABASE_URL)
```

重要: `NEXT_PUBLIC_` 変数を変更した場合

`NEXT_PUBLIC_` プレフィックスの変数は**ビルド時**にJavaScript に埋め込まれます。そのため、変更後は必ず開発サーバーの再起動が必要です。本番環境では、環境変数を変更した後に `npm run build` を再実行する必要があります。

理解度チェック

- ☐ `.env.local.example` から `.env.local` を作成できましたか？
- ☐ `DATABASE_URL` を設定して、`npx prisma db push` が成功しましたか？
- ☐ `NEXT_PUBLIC_` プレフィックスがついた変数とついていない変数の違いを説明できますか？
- ☐ 環境変数を変更した後に何をやる必要があるか分かりますか？
- ☐ 開発フェーズごとに、どの環境変数が必要になるか把握できましたか？

1.14 プロジェクト構造概要

このセクションで学ぶこと

- BON-LOG の完成時のプロジェクト構造の全体像
- `app/` ディレクトリの Route Group 構成と各ページの役割
- `components/` ディレクトリの分類方針
- `lib/` ディレクトリの役割とモジュール構成
- `hooks/`、`prisma/`、`types/`、`public/`、`__tests__/` の役割
- プロジェクトルートの設定ファイル群の一覧と用途

プロジェクト全体の鳥瞰図

BON-LOG の完成時のプロジェクトは、以下のような構造になっています。最初はシンプルですが、機能を追加するごとにフォルダが増えていきます。ここでは **完成時の全体像** を先に把握して、各部分の役割を理解しましょう。

```

bonsai-sns-project/      # プロジェクトルート
|
├─ app/                  # [1] ページとAPIルート
├─ components/           # [2] 再利用可能なUIコンポーネント
├─ lib/                  # [3] ビジネスロジック・ユーティリティ
├─ hooks/                # [4] カスタム React フック
├─ prisma/               # [5] データベーススキーマ・マイグレーション
├─ types/                # [6] TypeScript型定義
├─ public/               # [7] 静的ファイル（画像・アイコン等）
├─ __tests__/            # [8] テストファイル
├─ e2e/                  # [9] E2E（エンドツーエンド）テスト
├─ scripts/              # [10] ユーティリティスクリプト
├─ docs/                 # [11] ドキュメント
|
├─ .env.local             # 環境変数（Git管理対象外）
├─ .env.local.example     # 環境変数のテンプレート
├─ package.json           # プロジェクト設定・依存パッケージ
├─ tsconfig.json          # TypeScript設定
├─ next.config.ts         # Next.js設定
├─ proxy.ts               # リクエスト前処理（認証チェック等）
├─ docker-compose.yml     # Docker設定
├─ vitest.config.ts       # テスト設定
├─ playwright.config.ts   # E2Eテスト設定
└─ eslint.config.mjs      # ESLint設定

```

[1] app/ ディレクトリ - ページとAPIルート

app/ ディレクトリは、Next.js の **App Router** によるページ定義の中心です。フォルダ構造がそのままURLのパスに対応します（Route Group を除く）。


```

app/
├─ (auth)/                                # Route Group: 認証ページ (URLに含まれない)
│   ├── layout.tsx                       # 認証ページ専用レイアウト (シンプルなデザイン)
│   ├── loading.tsx                     # ローディング表示
│   ├── login/                          # /login → ログインページ
│   ├── register/                      # /register → 新規登録ページ
│   ├── password-reset/                # /password-reset → パスワードリセット
│   └─ verify-email/                   # /verify-email → メールアドレス確認
├─ (main)/                               # Route Group: メインアプリ (認証必須ページ)
│   ├── layout.tsx                     # 3カラムレイアウト (ナビ + コンテンツ + サイドバー)
│   ├── loading.tsx                   # ローディング表示
│   ├── feed/                         # /feed → タイムライン
│   ├── posts/                       # /posts → 投稿詳細
│   ├── users/                      # /users → ユーザープロフィール
│   ├── search/                     # /search → 検索
│   ├── shops/                      # /shops → 盆栽園マップ
│   ├── events/                     # /events → イベント
│   ├── notifications/              # /notifications → 通知
│   ├── bookmarks/                 # /bookmarks → ブックマーク
│   ├── messages/                  # /messages → メッセージ
│   ├── settings/                  # /settings → 設定
│   ├── drafts/                    # /drafts → 下書き
│   ├── analytics/                 # /analytics → アナリティクス
│   └─ bonsai/                     # /bonsai → 盆栽コレクション
├─ (public)/                            # Route Group: 公開ページ (認証不要)
│   ├── layout.tsx                  # 公開ページ用レイアウト
│   ├── about/                     # /about → BON-LOGについて
│   ├── contact/                   # /contact → お問い合わせ
│   └─ help/                       # /help → ヘルプ
├─ (legal)/                            # Route Group: 法的ページ
│   ├── layout.tsx                 # 法的ページ用レイアウト
│   ├── terms/                     # /terms → 利用規約
│   ├── privacy/                   # /privacy → プライバシーポリシー
│   └─ tokushoho/                  # /tokushoho → 特定商取引法に基づく表記
└─ admin/                              # 管理者ダッシュボード
    ├── layout.tsx                 # 管理者用レイアウト
    ├── page.tsx                   # /admin → ダッシュボードトップ
    ├── users/                     # /admin/users → ユーザー管理
    ├── posts/                     # /admin/posts → 投稿管理
    ├── reports/                   # /admin/reports → 通報管理
    ├── shops/                     # /admin/shops → 盆栽園管理
    ├── events/                    # /admin/events → イベント管理
    ├── stats/                     # /admin/stats → 統計情報
    ├── reviews/                  # /admin/reviews → レビュー管理
    └─ contact/                    # /admin/contact → お問い合わせ管理

```

```

|   ├── logs/                # /admin/logs → ログ管理
|   ├── maintenance/        # /admin/maintenance → メンテナンス
|   ├── premium/            # /admin/premium → プレミアム管理
|   ├── blacklist/          # /admin/blacklist → ブラックリスト
|   ├── hidden/             # /admin/hidden → 非表示コンテンツ
|   ├── shop-requests/      # /admin/shop-requests → 盆栽園登録申請
|   └── usage/              # /admin/usage → 利用状況
|
|   └── api/                 # APIルート (Route Handlers)
|       ├── auth/           # 認証API (NextAuth.js)
|       ├── upload/         # 画像アップロードAPI
|       ├── webhooks/       # 外部サービスからの通知 (Stripe等)
|       ├── admin/          # 管理者用API
|       ├── cron/           # 定期実行タスクAPI
|       ├── health/         # ヘルスチェックAPI
|       ├── og/             # OGP画像生成API
|       ├── badges/         # バッジAPI
|       ├── ad-frame/       # 広告フレームAPI
|       └── maintenance/    # メンテナンスモードAPI
|
|   └── auth/               # 認証コールバック
|       └── callback/       # OAuth コールバックハンドラ
|
|   └── maintenance/        # メンテナンスページ
|
|   ├── layout.tsx          # ルートレイアウト (全ページ共通)
|   ├── page.tsx            # トップページ (/)
|   ├── providers.tsx       # グローバルプロバイダー (React Query等)
|   ├── globals.css         # グローバルCSS
|   ├── global-error.tsx    # グローバルエラーハンドラ
|   ├── robots.ts          # robots.txt 生成
|   ├── sitemap.ts          # サイトマップ生成
|   └── feed.xml/           # RSSフィード

```

Route Group (丸括弧フォルダ) の使い分け:

(auth)、(main)、(public)、(legal) はすべて Route Group です。URLには影響しませんが、**レイアウトを分離**するために使います。

```

(auth)/login/page.tsx  → URL: /login    → シンプルなレイアウト
(main)/feed/page.tsx   → URL: /feed      → 3カラムレイアウト
(public)/about/page.tsx → URL: /about     → 公開ページ用レイアウト
(legal)/terms/page.tsx → URL: /terms     → 法的ページ用レイアウト

```

[2] components/ ディレクトリ - UIコンポーネント

`components/` は再利用可能なUIパーツを格納するフォルダです。 **機能（ドメイン）ごと** にサブフォルダを分けて整理します。

```

components/
├─ ui/                # shadcn/ui ベースの基本コンポーネント
│   │                # (Button, Card, Input, Dialog, Avatar 等)
│   └─ ...            # プロジェクト全体で使う汎用的なUIパーツ
├─ auth/              # 認証関連コンポーネント
│   │                # (LoginForm, RegisterForm, PasswordResetForm 等)
│   └─ ...
├─ post/              # 投稿関連コンポーネント
│   │                # (PostCard, PostForm, LikeButton, CommentList 等)
│   └─ ...
├─ user/              # ユーザー関連コンポーネント
│   │                # (UserCard, FollowButton, ProfileHeader 等)
│   └─ ...
├─ feed/              # フィード (タイムライン) 関連
│   │                # (FeedList, FeedFilters 等)
│   └─ ...
├─ shop/              # 盆栽園関連コンポーネント
│   │                # (ShopCard, ShopMap, ReviewForm 等)
│   └─ ...
├─ event/             # イベント関連コンポーネント
│   │                # (EventCard, EventCalendar 等)
│   └─ ...
├─ comment/           # コメント関連コンポーネント
│   │                # (CommentForm, CommentThread 等)
│   └─ ...
├─ notification/      # 通知関連コンポーネント
│   │                # (NotificationList, NotificationItem 等)
│   └─ ...
├─ search/            # 検索関連コンポーネント
│   │                # (SearchForm, SearchResults 等)
│   └─ ...
├─ message/           # メッセージ関連コンポーネント
│   │                # (MessageList, MessageForm 等)
│   └─ ...
├─ settings/          # 設定関連コンポーネント
│   │                # (ProfileEditForm, NotificationSettings 等)
│   └─ ...

```

```

├─ layout/                # レイアウトコンポーネント
│   └─ ...                # (Header, Sidebar, BottomNav, RightSidebar 等)
├─ common/               # 共通コンポーネント
│   └─ ...                # (Loading, ErrorMessage, InfiniteScroll 等)
├─ ads/                  # 広告コンポーネント
│   └─ ...                # (AdBanner, InfeedAd 等)
├─ analytics/            # アナリティクスコンポーネント
├─ bonsai/               # 盆栽コレクション関連
├─ contact/              # お問い合わせ関連
├─ draft/                # 下書き関連
├─ pwa/                  # PWA (プログレッシブWebアプリ) 関連
├─ report/               # 通報関連
├─ seo/                  # SEO関連 (メタタグ等)
├─ subscription/         # サブスクリプション (有料会員) 関連
└─ theme/                # テーマ (ダーク/ライトモード) 関連

```

コンポーネントの命名・配置の原則:

1. 機能ごとにフォルダを分ける: 投稿に関するものは `post/`、ユーザーに関するものは `user/` に
2. Server Component をデフォルトにする: `'use client'` は必要な場合のみ付与
3. 末端のコンポーネントにのみ `'use client'`: ボタンやフォームなどインタラクティブなパーツ
4. 共通パーツは `common/` に: 複数の機能で使い回すコンポーネント
5. UIプリミティブは `ui/` に: shadcn/ui で生成したベースコンポーネント

[3] lib/ ディレクトリ - ビジネスロジック

`lib/` はアプリケーションのビジネスロジック、外部サービスとの連携、ユーティリティ関数を格納するフォルダです。

```

lib/
├─ db.ts                # Prismaクライアント設定（DB接続の中心）
├─ auth.ts              # NextAuth.js 設定（認証の中心）
├─ auth.config.ts       # NextAuth.js の設定オプション
├─ utils.ts              # 汎用ユーティリティ関数
├─ cache.ts              # キャッシュ管理
├─ redis.ts              # Upstash Redis クライアント
├─ stripe.ts             # Stripe クライアント設定
├─ rate-limit.ts         # レート制限ロジック
├─ logger.ts             # ロギングユーティリティ
├─ sanitize.ts           # 入力サニタイズ（XSS対策）
├─ csrf.ts               # CSRF保護
├─ premium.ts            # プレミアム会員関連
├─ two-factor.ts         # 2段階認証
├─ fingerprint.ts        # デバイスフィンガープリント
├─ login-tracker.ts       # ログイン追跡
├─ mention-utils.ts       # メンション（@ユーザー名）処理
├─ prefectures.ts         # 都道府県データ
├─ file-validation.ts     # ファイルバリデーション
├─ client-image-compression.ts # クライアント側画像圧縮
├─ security-checks.ts     # セキュリティチェック
├─ security-logger.ts     # セキュリティログ
├─ cron-auth.ts          # Cronジョブの認証
├─
├─ actions/              # Server Actions（サーバー側のデータ操作）
│   ├─ post.ts           # 投稿の作成・更新・削除
│   ├─ user.ts            # ユーザー情報の更新
│   ├─ auth.ts            # ログイン・登録・パスワードリセット
│   ├─ comment.ts         # コメント操作
│   ├─ like.ts            # いいね操作
│   ├─ follow.ts          # フォロー操作
│   ├─ bookmark.ts        # ブックマーク操作
│   ├─ search.ts          # 検索処理
│   ├─ shop.ts            # 盆栽園操作
│   ├─ event.ts           # イベント操作
│   ├─ review.ts          # レビュー操作
│   ├─ notification.ts    # 通知操作
│   ├─ report.ts          # 通報操作
│   ├─ subscription.ts    # サブスクリプション操作
│   ├─ message.ts         # メッセージ操作
│   ├─ draft.ts           # 下書き操作
│   ├─ block.ts           # ブロック操作
│   ├─ mute.ts            # ミュート操作
│   ├─ feed.ts            # フィード取得
│   ├─ admin.ts           # 管理者操作
│   ├─ admin/             # 管理者用Server Actions（サブフォルダ）
│   └─ ...                # その他の機能
├─
├─ storage/              # ストレージ抽象化レイヤー

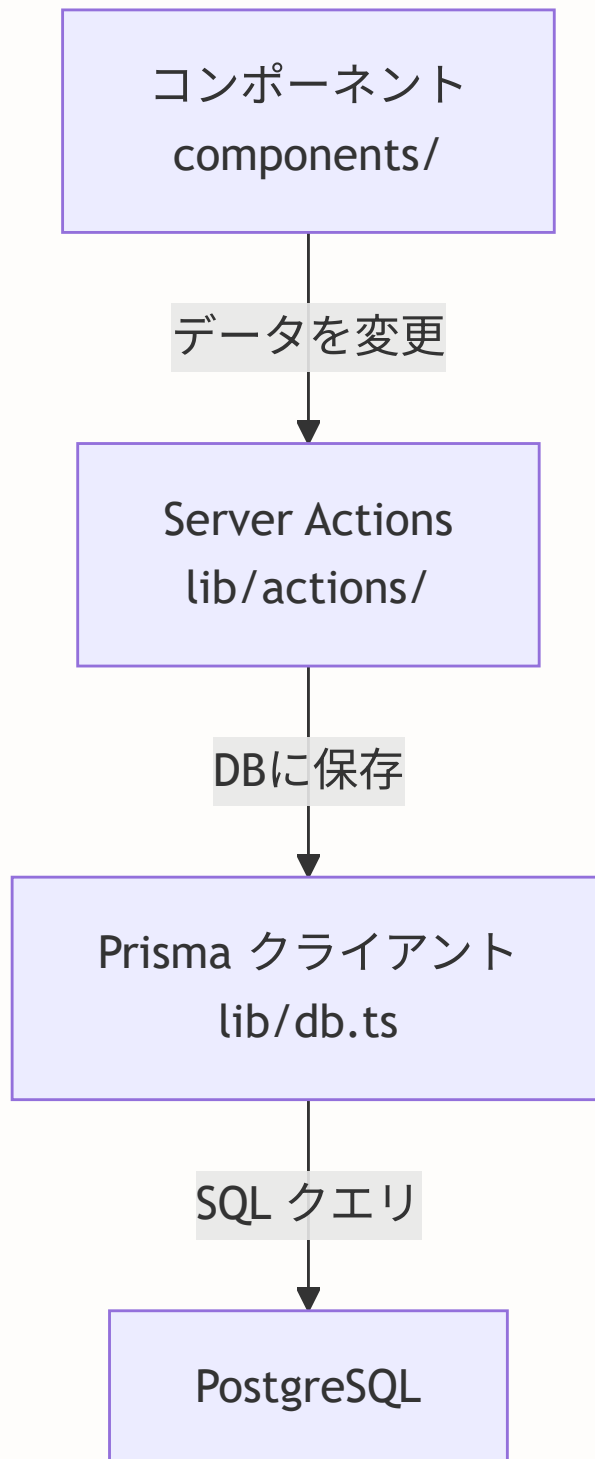
```

```

└─┬ index.ts          # local / R2 / Supabase の切り替え
  │
└─┬ email/            # メール送信抽象化レイヤー
  │ └─ index.ts       # console / Resend / Azure の切り替え
  │
└─┬ search/           # 全文検索
  │ └─ fulltext.ts    # like / trgm / bigm の切り替え
  │
└─┬ security/         # セキュリティモジュール
  │ │ └─ index.ts     # セキュリティ機能の統合
  │ │ └─ nonce.ts     # CSPナンス生成
  │ │
└─┬ scraping/         # スクレイピング
  │ └─ bonsai-events.ts # 盆栽イベント情報の取得
  │
└─┬ constants/        # 定数定義
  │ │ └─ locations.ts  # 地域・都道府県の定数
  │ │ └─ report.ts     # 通報理由の定数
  │ │
└─┬ validations/      # バリデーションルール
  │ └─ password.ts     # パスワード強度チェック
  │
└─┬ services/         # サービスレイヤー
  │ └─ usage.ts        # 利用状況サービス
  │
└─┬ types/            # lib内で使う型定義

```

lib/ の設計思想:



コンポーネントは `lib/actions/` のServer Actionsを呼び出し、Server Actionsは `lib/db.ts` のPrisma クライアントを使ってデータベースを操作します。この層構造により、各部分の責任が明確に分離されます。

BON-LOGでの `lib/db.ts` の実装 (Prisma 6 + PrismaPg アダプター)

BON-LOG では Prisma 6 で導入されたアダプターシステムを使用しています。従来の Prisma 5 以前の書き方とは異なり、PostgreSQL 接続プール (`pg` パッケージ) をアダプター経由で Prisma に渡します。

```
// lib/db.ts (実際のコード)
import { PrismaClient } from '@prisma/client'
import { PrismaPg } from '@prisma/adapter-pg'
import { Pool } from 'pg'

const globalForPrisma = global as unknown as { prisma: PrismaClient }

// CI/テスト環境ではダミーDBのため接続しない
const isDummyDatabase = process.env.DATABASE_URL?.includes('dummy') || false

// PostgreSQL 接続プールを作成 (本番: SSL有効、開発: SSL無効)
const pool = isDummyDatabase ? null : new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } : false
})

// PrismaPg アダプター (PrismaとPostgreSQLドライバーを接続)
const adapter = pool ? new PrismaPg(pool) : null

// シングルトンパターン: ホットリロード時の接続過多を防止
export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    ... (adapter && { adapter }),
    log: process.env.NODE_ENV === 'development' ? ['query', 'error', 'warn'] : ['error']
  })

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
```

BON-LOGでの使用箇所: すべての Server Actions (`lib/actions/*.ts`) および API Routes (`app/api/*/route.ts`) で `import { prisma } from '@lib/db'` として使用。

実装しない場合の影響: データベースへのアクセスが完全に機能しなくなり、投稿・ユーザー情報など全てのデータ操作が不可能になる。また、`@prisma/adapter-pg` を使わずに直接 `PrismaClient` を作成すると、Next.js の開発モード (ホットリロード) で接続が増加し続ける「Too many connections」エラーが発生する。

[4] hooks/ - カスタム React フック

```
hooks/
├─ use-keyboard-shortcuts.ts # キーボードショートカット管理
└─ use-toast.ts             # トースト通知（画面下部のメッセージ表示）
```

カスタムフックは、複数のコンポーネントで再利用したい **状態管理ロジック** や **副作用（side effect）** をまとめるための仕組みです。 `use` で始まるファイル名は React の慣習です。

[5] prisma/ - データベース

```
prisma/
├─ schema.prisma          # DBスキーマ定義（テーブルの設計図）
├─ seed.ts                # シードデータ（初期データ投入スクリプト）
└─ migrations/            # マイグレーションファイル（スキーマ変更の履歴）
    ├─ 20240101_init/
    ├─ 20240201_add_genres/
    └─ ...
```

- **schema.prisma**: データベースの全テーブル定義。ここを変更して `npx prisma db push` を実行すると、テーブル構造が更新される
- **seed.ts**: `npx prisma db seed` で実行され、ジャンルマスタや管理者アカウント等の初期データを投入する
- **migrations/**: `npx prisma migrate dev` で生成されるマイグレーションファイル。スキーマ変更の履歴を管理する

[6] types/ - TypeScript型定義

```
types/
└─ next-auth.d.ts        # NextAuth.js の型拡張
```

TypeScriptの型定義を拡張するためのフォルダです。 `next-auth.d.ts` では、NextAuth.js のセッションオブジェクトに `user.id` を追加するなど、ライブラリの型を拡張しています。

[7] public/ - 静的ファイル

```
public/
├─ images/           # 画像ファイル（ロゴ等）
├─ uploads/         # アップロード画像（開発環境用）
├─ logo.png          # BON-LOG ロゴ
├─ icon-192.png      # PWA用アイコン（192x192）
├─ icon-512.png      # PWA用アイコン（512x512）
├─ apple-touch-icon.png # iOS用アイコン
├─ favicon.ico        # ブラウザタブのアイコン
├─ site.webmanifest  # PWA マニフェスト
├─ sw.js              # Service Worker（PWA用）
├─ offline.html       # オフラインページ
├─ ads.txt            # 広告認証ファイル
└─ robots.txt        # クローラー制御ファイル
```

`public/` フォルダに置いたファイルは、`/ファイル名` でアクセスできます。例: `public/logo.png` → `http://localhost:3000/logo.png`

[8] tests/ と [9] e2e/ - テスト

```
__tests__/           # ユニットテスト・統合テスト（Vitest）
├─ api/              # APIルートのテスト
├─ app/              # ページコンポーネントのテスト
├─ components/       # コンポーネントのテスト
├─ hooks/            # カスタムフックのテスト
├─ lib/              # ビジネスロジックのテスト
├─ coverage-boost/   # カバレッジ向上用テスト
├─ proxy.test.ts     # Proxyのテスト
└─ utils/            # テスト用ユーティリティ

e2e/                 # E2Eテスト（Playwright）
├─ auth.setup.ts     # 認証セットアップ
├─ auth.spec.ts      # 認証フローのE2Eテスト
├─ feed.spec.ts      # フィードのE2Eテスト
├─ search.spec.ts    # 検索のE2Eテスト
└─ user-profile.spec.ts # ユーザープロフィールのE2Eテスト
```

- `__tests__`: Vitest を使ったユニットテスト・統合テスト。コンポーネントの表示やロジックの正しさを確認
- `e2e`: Playwright を使った E2E（エンドツーエンド）テスト。ブラウザ上で実際のユーザー操作をシミュレート

[10] scripts/ - ユーティリティスクリプト

```
scripts/  
├─ setup-fts.ts      # 全文検索 (pg_trgm/pg_bigm) のセットアップ  
└─ check-fts.ts     # 全文検索の設定確認
```

データベースの初期設定や運用タスクなど、アプリケーション本体には含まれない 補助的なスクリプトを格納します。

プロジェクトルートの設定ファイル一覧

プロジェクトルートには多くの設定ファイルがあります。最初は「多すぎてよくわからない」と感じるかもしれませんが、それぞれが明確な役割を持っています。

ファイル名	用途	編集頻度
<code>package.json</code>	プロジェクト情報・依存パッケージ・スクリプト定義	中（パッケージ追加時）
<code>package-lock.json</code>	パッケージのバージョンロック（自動生成）	自動
<code>tsconfig.json</code>	TypeScript のコンパイル設定	低
<code>next.config.ts</code>	Next.js のアプリケーション設定	中
<code>proxy.ts</code>	リクエスト前処理（認証ルート保護等）	中
<code>docker-compose.yml</code>	Docker コンテナの設定	低
<code>eslint.config.mjs</code>	ESLint（コード品質チェック）の設定	低
<code>postcss.config.mjs</code>	PostCSS（Tailwind CSS 用）の設定	低
<code>vitest.config.ts</code>	Vitest（ユニットテスト）の設定	低
<code>vitest.setup.tsx</code>	Vitest のセットアップ（グローバルモック等）	低
<code>playwright.config.ts</code>	Playwright（E2Eテスト）の設定	低
<code>components.json</code>	shadcn/ui のコンポーネント設定	低
<code>vercel.json</code>	Vercel デプロイ設定	低
<code>Dockerfile</code>	本番用 Docker イメージ定義	低
<code>Dockerfile.dev</code>	開発用 Docker イメージ定義	低
<code>instrumentation.ts</code>	Next.js Instrumentation（Sentry等）	低
<code>sentry.client.config.ts</code>	Sentry クライアント側設定	低
<code>sentry.server.config.ts</code>	Sentry サーバー側設定	低
<code>sentry.edge.config.ts</code>	Sentry Edge Runtime 設定	低

package.json の scripts セクション

`package.json` の `scripts` セクションには、開発で頻繁に使うコマンドのショートカットが定義されています。`npm run <スクリプト名>` で実行できます。

```

{
  "scripts": {
    "postinstall": "prisma generate",
    "dev": "next dev",
    "build": "prisma generate && next build",
    "build:deploy": "prisma generate && prisma migrate deploy && next build",
    "start": "next start",
    "lint": "eslint",
    "db:seed": "npx tsx prisma/seed.ts",
    "db:setup-fts": "npx tsx scripts/setup-fts.ts",
    "test": "vitest run",
    "test:watch": "vitest",
    "test:coverage": "vitest run --coverage",
    "test:ci": "vitest run --coverage",
    "test:e2e": "playwright test",
    "test:e2e:ui": "playwright test --ui",
    "test:e2e:headed": "playwright test --headed",
    "test:e2e:debug": "playwright test --debug",
    "test:all": "npm run test && npm run test:e2e"
  }
}

```

コマンド	実行方法	用途
<code>npm run dev</code>	開発時に最も使う	開発サーバー起動（ホットリロード対応）
<code>npm run build</code>	デプロイ前	本番用ビルド生成
<code>npm run start</code>	ビルド後	本番サーバー起動
<code>npm run lint</code>	コミット前	コード品質チェック
<code>npm run db:seed</code>	DB初期化後	初期データ投入（ジャンルマスタ等）
<code>npm test</code>	開発時	ユニットテスト実行
<code>npm run test:coverage</code>	品質確認時	テストカバレッジ付きで実行
<code>npm run test:e2e</code>	機能確認時	ブラウザ自動テスト実行
<code>npm run test:all</code>	プルリクエスト前	全テスト一括実行

postinstall スクリプトについて:

postinstall は特殊なスクリプトで、**npm install** の完了後に**自動的に**実行されます。BON-LOG では **prisma generate** (Prismaクライアントの型生成) を自動実行しています。これにより、**npm install** するだけで Prisma クライアントが使える状態になります。

ファイルの探し方のコツ

プロジェクトが大きくなると「あのファイルはどこにあるんだっけ?」となることがあります。以下のルールを覚えておくと、ファイルを素早く見つけられます。

ファイルの探し方ガイド:

「このページのUIを変えたい」

→ `app/(main)/[機能名]/page.tsx` を探す

→ 例: フィードページなら `app/(main)/feed/page.tsx`

「このボタンのデザインを変えたい」

→ `components/[機能名]/[コンポーネント名].tsx` を探す

→ 例: いいねボタンなら `components/post/LikeButton.tsx`

「データの保存処理を変えたい」

→ `lib/actions/[機能名].ts` を探す

→ 例: 投稿の作成なら `lib/actions/post.ts`

「データベースのテーブル構造を変えたい」

→ `prisma/schema.prisma` を開く

「環境変数を追加・変更したい」

→ `.env.local` を編集 → サーバー再起動

「VS Code でファイルを素早く開きたい」

→ `Ctrl + P` (Mac: `Cmd + P`) でファイル名検索

理解度チェック

- ☐ `app/` の Route Group (`(auth)`、`(main)`、`(public)`、`(legal)`) のそれぞれの役割を説明できますか？
- ☐ `components/` と `lib/` の違い (UI vs ロジック) を理解していますか？
- ☐ `lib/actions/` フォルダの Server Actions が何をするものか分かりますか？
- ☐ `prisma/schema.prisma` の役割を説明できますか？

- ☐ `public/` フォルダに置いたファイルへのアクセス方法を知っていますか？
- ☐ `npm run dev` と `npm run build` の違いを説明できますか？
- ☐ `package.json` の `scripts` セクションの役割を理解していますか？
- ☐ プロジェクト内でファイルを探すコツを3つ以上言えますか？

package.json の dependencies 完全解説

`package.json` の `dependencies`（本番依存）と `devDependencies`（開発依存）に記載されているすべてのパッケージを、カテゴリごとに詳しく解説します。

ここがポイント！

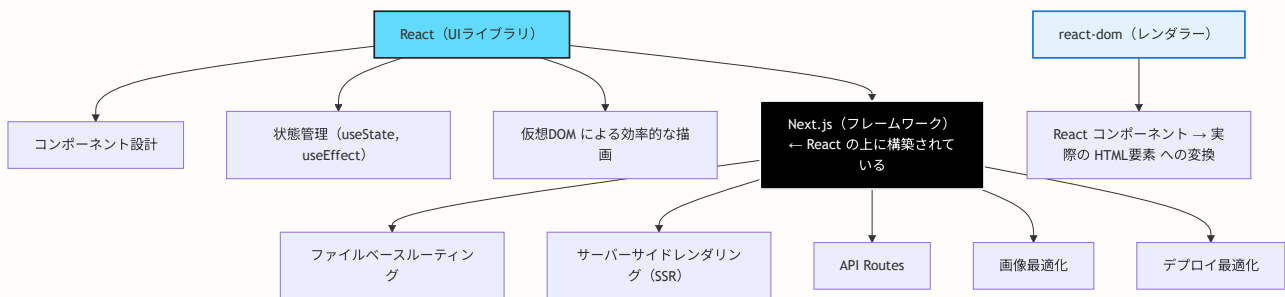
`dependencies` はアプリケーションの実行に必要なパッケージ、`devDependencies` は開発・テスト時にのみ必要なパッケージです。本番環境にデプロイする際、`devDependencies` は通常インストールされません。

dependencies（本番で必要）	devDependencies（開発時のみ必要）
React, Next.js	TypeScript
Prisma Client	ESLint
認証ライブラリ	Vitest（テスト）
UIコンポーネント	Playwright（E2E）
決済・メール etc.	型定義ファイル
本番に含まれる	本番には含まれない

[A] フレームワーク・コアライブラリ

パッケージ名	バージョン	説明	関連する章
<code>next</code>	16.1.1	Next.js 本体。React ベースのフルスタックフレームワーク。ルーティング、SSR、API Routes、画像最適化など全てを提供	第3章～全章
<code>react</code>	19.2.3	React 本体。UI を「コンポーネント」という部品で構築するライブラリ	第3章
<code>react-dom</code>	19.2.3	React をブラウザの DOM (HTML要素) に描画するためのライブラリ。React 本体とセットで使う	第3章

React と Next.js の関係図:

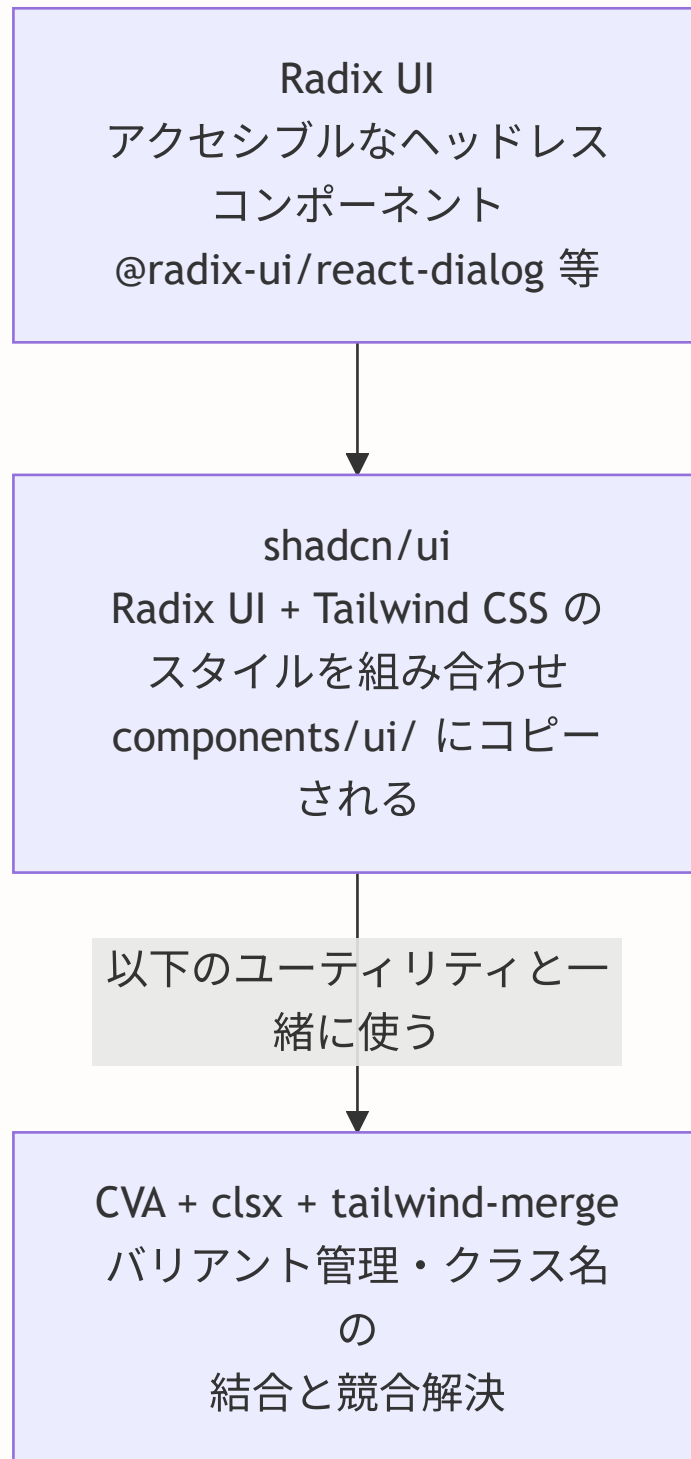


初心者向け解説: React と react-dom はなぜ別パッケージ？

React のコア (コンポーネント設計、状態管理の仕組み) は ブラウザに限らず使えるように設計されています。例えば React Native (スマホアプリ開発) でも React のコアは同じです。 `react-dom` は「ブラウザ向けの描画エンジン」という位置づけです。

[B] UI コンポーネント・スタイリング

パッケージ名	バージョン	説明	関連する章
<code>@radix-ui/react-alert-dialog</code>	^1.1.15	アクセシブルなアラートダイアログ（確認モーダル）。「本当に削除しますか？」のような確認UI	第7章
<code>@radix-ui/react-avatar</code>	^1.1.11	ユーザーアバター画像の表示。画像読み込み失敗時のフォールバック表示も対応	第6章
<code>@radix-ui/react-dialog</code>	^1.1.15	汎用的なモーダルダイアログ。投稿作成フォーム等に使用	第7章
<code>@radix-ui/react-dropdown-menu</code>	^2.1.16	ドロップダウンメニュー。投稿の「…」メニュー等に使用	第7章
<code>@radix-ui/react-label</code>	^2.1.8	フォームのラベル要素。アクセシビリティ対応	第5章
<code>@radix-ui/react-slot</code>	^1.2.4	コンポーネント合成の基盤。shadcn/ui の内部で使用	-
<code>@radix-ui/react-switch</code>	^1.2.6	トグルスイッチ。設定画面のON/OFF切り替えに使用	第11章
<code>@radix-ui/react-tabs</code>	^1.1.13	タブ切り替えUI。プロフィールページの投稿/メディア切り替え等	第6章
<code>@radix-ui/react-tooltip</code>	^1.2.8	ツールチップ（マウスホバー時の説明表示）	第7章
<code>class-variance-authority</code>	^0.7.1	コンポーネントのバリエーション（種類）管理。ボタンの primary/secondary 等の切り替え	第3章
<code>clsx</code>	^2.1.1	CSS クラス名を条件付きで結合するユーティリティ	第3章
<code>tailwind-merge</code>	^3.4.0	Tailwind CSS のクラス名の競合を解決。 <code>clsx</code> と組み合わせて使用	第3章
<code>lucide-react</code>	^0.562.0	アイコンライブラリ。ハートマーク、ホームアイコン等、数千種類のSVGアイコン	第3章
<code>recharts</code>	^3.6.0	グラフ・チャートライブラリ。管理画面の統計グラフ表示に使用	第15章

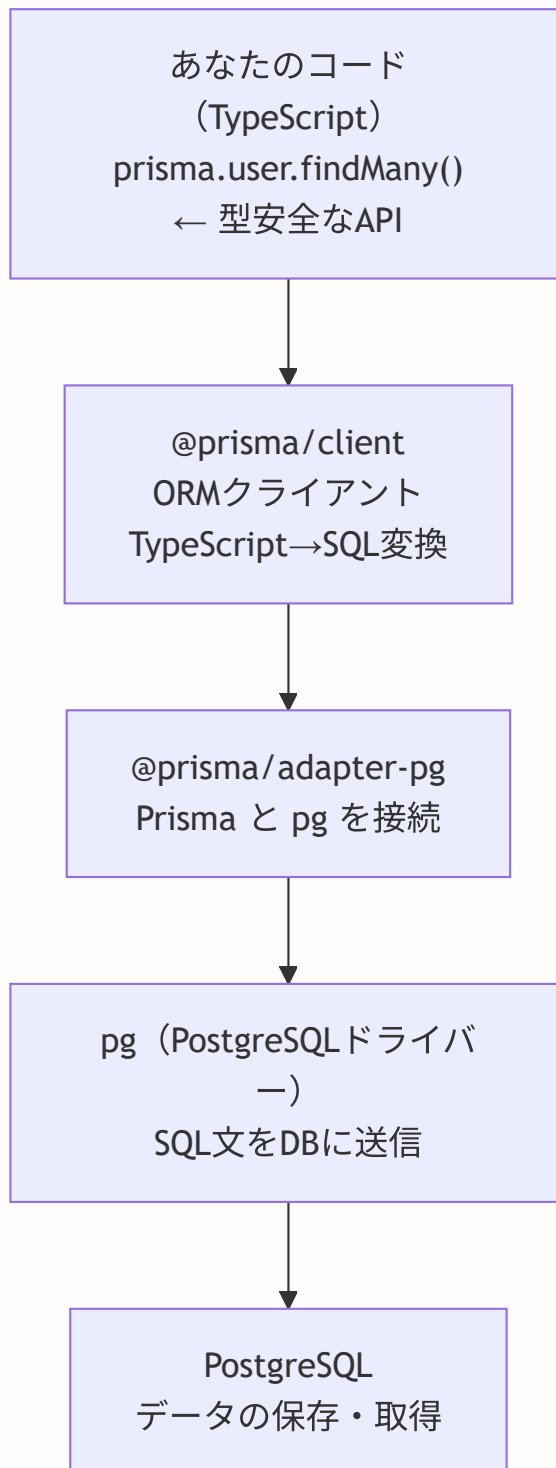


ここがポイント！

`@radix-ui/*` のパッケージは「ヘッドレス（見た目なし）UI」と呼ばれます。アクセシビリティ（キーボード操作、スクリーンリーダー対応）は完璧に実装済みで、見た目だけを自分で付ける設計です。shadcn/ui はこれに Tailwind CSS のスタイルを組み合わせた「レシピ」を提供してくれるツールです。

[C] データベース・ORM

パッケージ名	バージョン	説明	関連する章
<code>@prisma/client</code>	^6.8.2	Prisma クライアント。TypeScript から型安全にデータベース操作を行う	第4章
<code>@prisma/adapter-pg</code>	^6.8.2	Prisma の PostgreSQL アダプター。pg ドライバーとの接続に使用	第4章
<code>pg</code>	^8.16.3	Node.js 用の PostgreSQL ドライバー。データベースとの低レベル通信を担当	第4章



BON-LOGでの使用箇所: `lib/db.ts` で `PrismaClient`、`PrismaPg`（アダプター）、`Pool`（`pg` パッケージ）を組み合わせでデータベース接続を初期化しています。Prisma 6 からはアダプターシステムが必須になり、`@prisma/adapter-pg` と `pg` の両方が依存パッケージとして必要です。

実装しない場合の影響: `@prisma/adapter-pg` がないと Prisma 6 で PostgreSQL に 接続できません。
`pg` がないと接続プール（Pool）を作成できず、本番環境でのデータベース接続が大幅に遅くなります。

[D] 認証・セキュリティ

パッケージ名	バージョン	説明	関連する章
<code>next-auth</code>	^5.0.0-beta.30	NextAuth.js (Auth.js v5)。認証機能の全体を管理（ログイン、セッション、JWT）	第5章
<code>@auth/prisma-adapter</code>	^2.11.1	NextAuth.js と Prisma を接続するアダプター。セッション情報をDBに保存	第5章
<code>bcryptjs</code>	^3.0.3	パスワードのハッシュ化ライブラリ。平文パスワードを安全な形式に変換	第5章
<code>otplib</code>	^13.2.1	ワンタイムパスワード（TOTP）生成。2段階認証機能に使用	第5章
<code>qrcode</code>	^1.5.4	QRコード生成。2段階認証アプリ連携時のQRコード表示に使用	第5章
<code>@fingerprintjs/fingerprintjs</code>	^5.0.1	ブラウザフィンガープリント。不正アクセス検知やデバイス識別に使用	第5章
<code>zod</code>	^4.3.5	スキーマバリデーション。ユーザー入力の検証を型安全に行う	第5章

ユーザー
ログインフォームで
メール+パスワード入力

bcryptjs
パスワードをハッシュ比較
(一方向変換)

next-auth
認証成功→JWTトークン発行
セッション情報をDBに記録

zod: 入力値バリデーション
otplib+qrcode: 2段階認証

[E] 状態管理・データフェッチング

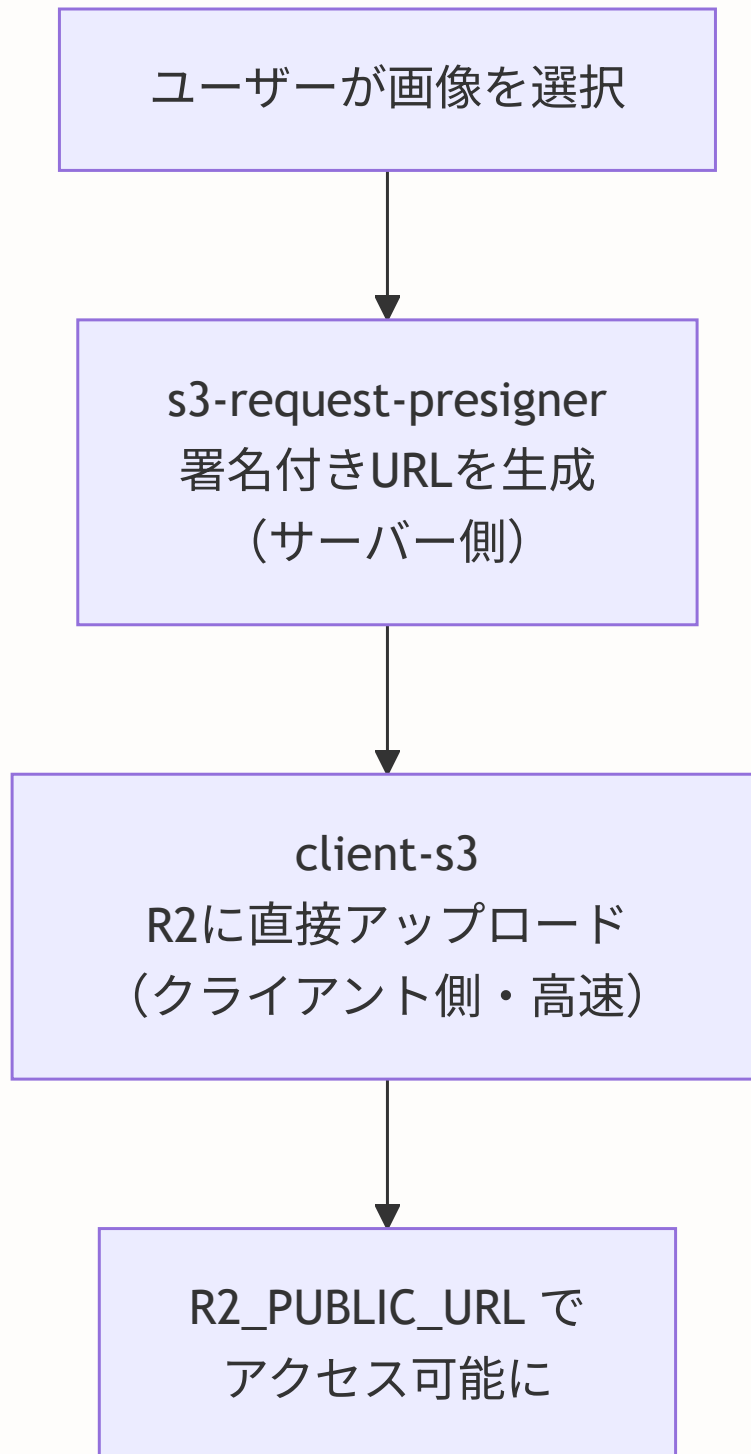
パッケージ名	バージョン	説明	関連する章
@tanstack/react-query	^5.90.16	サーバー状態管理。APIからのデータ取得、キャッシュ、再取得を自動管理	第7章

注意（状態管理について）

本プロジェクトではクライアント状態に **useState / Context** を使っています。Zustand は使っていません。チュートリアルでは「選択肢」として Ch04 で Zustand を紹介しているので、学習用に読んで問題ありません。

[F] ストレージ・ファイル操作

パッケージ名	バージョン	説明	関連する章
@aws-sdk/client-s3	^3.971.0	AWS S3 互換のストレージ操作。Cloudflare R2 への画像アップロードに使用	第8章
@aws-sdk/s3-request-presigner	^3.974.0	署名付きURL生成。一時的なアクセス許可URLを作成（セキュアなアップロード）	第8章



[G] キャッシュ

パッケージ名	バージョン	説明	関連する章
@upstash/redis	^1.36.1	Upstash Redis クライアント。レート制限、セッションキャッシュ、一時データ保存に使用	第10章

Redis とは？

Redis はインメモリデータベース（メモリ上にデータを保存）です。PostgreSQL が「ノートに書いて保存する」だとすると、Redis は「ホワイトボードに書く」イメージ。超高速だけど電源を切ると消える。レート制限（1分に5回まで等）のような短期間のカウントに最適です。

[H] メール送信

パッケージ名	バージョン	説明	関連する章
<code>resend</code>	^6.7.0	メール送信API。パスワードリセット、通知メール等の送信に使用	第12章

[I] 決済

パッケージ名	バージョン	説明	関連する章
<code>stripe</code>	^20.1.2	Stripe 決済ライブラリ。有料会員（プレミアムプラン）の課金処理に使用	第13章

[J] 地図

パッケージ名	バージョン	説明	関連する章
<code>leaflet</code>	^1.9.4	オープンソースの地図ライブラリ。盆栽園マップの表示に使用	第9章
<code>react-leaflet</code>	^5.0.0	Leaflet の React ラッパー。React コンポーネントとして地図を使える	第9章
<code>@types/leaflet</code>	^1.9.21	Leaflet の TypeScript 型定義	第9章

[K] ユーティリティ

パッケージ名	バージョン	説明	関連する章
<code>date-fns</code>	^4.1.0	日付操作ライブラリ。「3時間前」「2024年1月1日」等の日付フォーマットに使用	第7章
<code>react-intersection-observer</code>	^10.0.0	要素の画面表示を検知。無限スクロール（スクロールで自動読み込み）に使用	第7章

[L] エラー監視

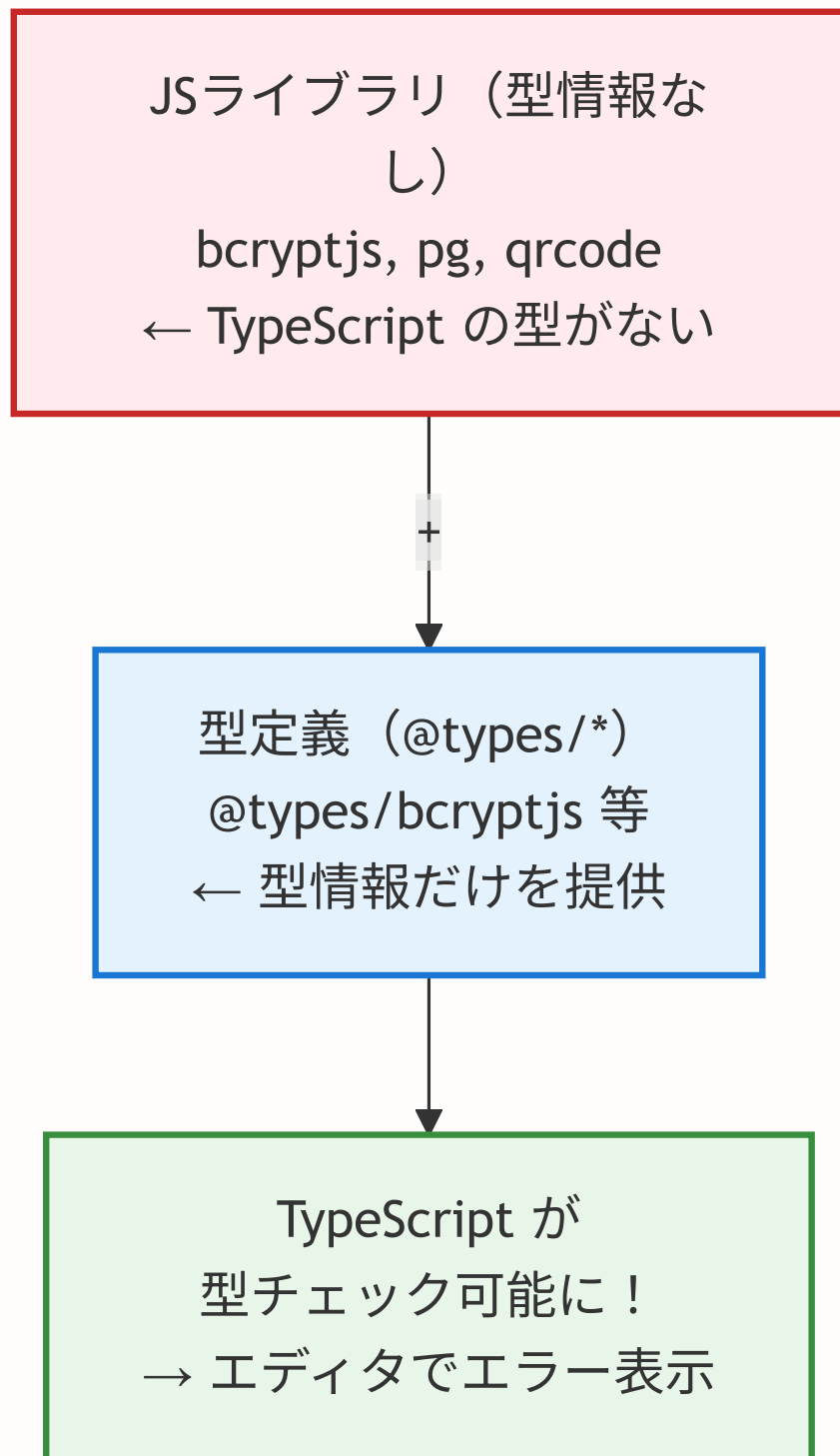
パッケージ名	バージョン	説明	関連する章
<code>@sentry/nextjs</code>	^10.34.0	Sentry エラー監視。本番環境でのエラー自動検知・通知に使用	第14章

[M] devDependencies（開発用パッケージ）

パッケージ名	バージョン	説明
<code>typescript</code>	^5	TypeScript コンパイラ。型チェックを行う
<code>@types/node</code>	^20	Node.js の型定義。 <code>process.env</code> 等に型を提供
<code>@types/react</code>	^19	React の型定義
<code>@types/react-dom</code>	^19	React DOM の型定義
<code>@types/bcryptjs</code>	^2.4.6	bcryptjs の型定義
<code>@types/pg</code>	^8.16.0	pg（PostgreSQLドライバ）の型定義
<code>@types/qrcode</code>	^1.5.6	qrcode の型定義
<code>eslint</code>	^9	ESLint 本体。コードの品質・スタイルチェック
<code>eslint-config-next</code>	16.1.1	Next.js 公式の ESLint ルール集
<code>tailwindcss</code>	^4	Tailwind CSS 本体。ユーティリティファーストの CSS フレームワーク
<code>@tailwindcss/postcss</code>	^4	Tailwind CSS を PostCSS プラグインとして使うためのアダプター
<code>tw-animate-css</code>	^1.4.0	Tailwind CSS 用のアニメーションユーティリティ
<code>prisma</code>	^6.8.2	Prisma CLI。マイグレーション実行、クライアント生成等の開発ツール
<code>tsx</code>	^4.21.0	TypeScript ファイルを直接実行するツール。シードスクリプト等に使用
<code>vitest</code>	^4.0.18	Vitest テストフレームワーク本体
<code>@vitest/coverage-v8</code>	^4.0.18	Vitest 用 V8 カバレッジプロバイダー
<code>jsdom</code>	^28.1.0	ブラウザ環境（DOM操作）をシミュレートする環境
<code>@testing-library/jest-dom</code>	^6.9.1	DOM アサーション拡張。 <code>toBeInTheDocument()</code> 等のマッチャーを提供
<code>@testing-library/react</code>	^16.3.1	React コンポーネントのテストユーティリティ。 <code>render()</code> , <code>screen</code> 等

<code>@testing-library/user-event</code>	^14.6.1	ユーザー操作（クリック、タイピング等）のシミュレート
<code>@playwright/test</code>	^1.57.0	Playwright E2E テストフレームワーク。実際のブラウザを自動操作

@types/ パッケージの役割.*



理解度チェック

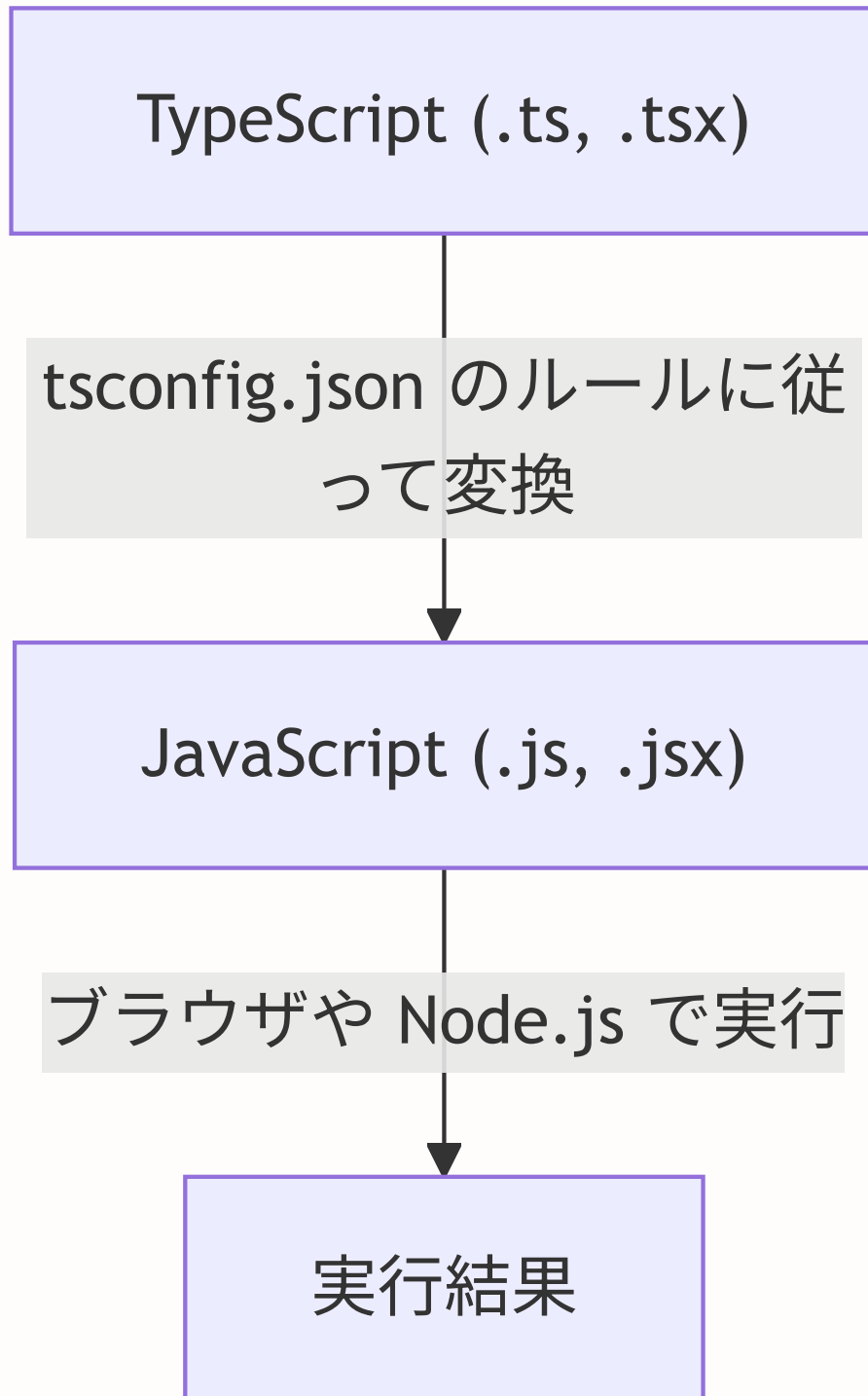
- ☐ `dependencies` と `devDependencies` の違いを説明できますか？
- ☐ `@radix-ui/*` パッケージがなぜ多数あるのか理解していますか？
- ☐ React Query と Zustand の使い分けを説明できますか？
- ☐ `@types/*` パッケージが必要な理由を説明できますか？
- ☐ `bcryptjs` がなぜ認証に必要なか説明できますか？

tsconfig.json 完全解説

`tsconfig.json` は TypeScript コンパイラの設定ファイルです。「TypeScript をどのようにコンパイル（変換）するか」のルールを定義します。

ここがポイント！

TypeScript はそのままではブラウザや Node.js で実行できません。JavaScriptに変換（コンパイル）する必要があります。`tsconfig.json` は、その変換のルールブックです。



以下が BON-LOG プロジェクトの `tsconfig.json` の全内容と、各設定の解説です。

```

{
  // =====
  // compilerOptions: TypeScript コンパイラの動作設定
  // =====
  "compilerOptions": {

    // =====
    // target: 出力する JavaScript のバージョン
    // =====
    // "ES2017" = ECMAScript 2017 の構文に変換
    // async/await が使えるバージョン
    // これより古い (ES5等) にすると互換性は上がるがコードが冗長になる
    // Next.js が自動で追加の変換を行うため、ES2017 で十分
    "target": "ES2017",

    // =====
    // lib: 使用できる組み込み型定義のリスト
    // =====
    // "dom"          → document, window, HTMLElement 等のブラウザAPI の型
    // "dom.iterable" → NodeList の forEach 等、イテレーション対応の型
    // "esnext"       → 最新の JavaScript 機能 (Promise, Map, Set 等) の型
    "lib": [
      "dom",
      "dom.iterable",
      "esnext"
    ],

    // =====
    // types: グローバルに読み込む型定義パッケージ
    // =====
    // "node"          → process.env, __dirname 等の Node.js API の型
    // "vitest/globals" → describe, it, expect 等の Vitest API の型
    // "@testing-library/jest-dom" → toBeInTheDocument() 等のカスタムマッチャーの型
    "types": [
      "node",
      "vitest/globals",
      "@testing-library/jest-dom"
    ],

    // =====
    // allowJs: JavaScript ファイルも TypeScript プロジェクトに含める
    // =====
    // true にすると .js ファイルも import できる
    // 既存の JavaScript コード (vitest.config.ts 等) との共存に必要
    "allowJs": true,

    // =====
    // skipLibCheck: node_modules 内の型定義チェックをスキップ
    // =====
    // true にするとビルド速度が大幅に向上
    // ライブラリ同士の型の矛盾 (まれにある) でエラーにならなくなる
  }
}

```



```

// 自分のコードの型チェックには影響なし
"skipLibCheck": true,

// -----
// strict: 厳格な型チェックを有効化
// -----
// true にすると以下が全て有効になる:
//   strictNullChecks    → null/undefined を厳密にチェック
//   strictFunctionTypes  → 関数の型パラメータを厳密にチェック
//   strictBindCallApply → bind, call, apply の型チェック
//   noImplicitAny        → 暗黙の any 型を禁止
//   noImplicitThis       → this の型推論失敗をエラーに
//   alwaysStrict        → 全ファイルに "use strict" を付与
//
// バグの早期発見に非常に効果的。新規プロジェクトでは必ず true 推奨
"strict": true,

// -----
// noEmit: JavaScript ファイルを出力しない
// -----
// Next.js は自前のビルドツール (SWC) で変換するため
// TypeScript コンパイラには型チェックだけを担当させる
"noEmit": true,

// -----
// esModuleInterop: CommonJS / EModule の相互運用
// -----
// import React from 'react' のようなデフォルトインポートを可能にする
// これがないと import * as React from 'react' と書く必要がある
"esModuleInterop": true,

// -----
// module: モジュールシステムの種類
// -----
// "esnext" = 最新の EModule (import/export) 構文を使用
// Next.js のバンドラーが最終的な変換を行う
"module": "esnext",

// -----
// moduleResolution: モジュールの解決方法
// -----
// "bundler" = バンドラー (webpack, Turbopack等) に合わせた解決方法
// Next.js 13.4+ で推奨される設定
// 拡張子なしのインポートや package.json の exports フィールドを正しく解決
"moduleResolution": "bundler",

// -----
// resolveJsonModule: JSON ファイルのインポートを許可
// -----
// import data from './data.json' のような JSON インポートが可能に
// 型推論も自動で行われる
"resolveJsonModule": true,

```

```

// =====
// isolatedModules: ファイル単位のトランスパイルを保証
// =====
// 各ファイルが独立してコンパイルできることを保証
// Next.js の SWC (高速コンパイラ) が要求する設定
// const enum や namespace の一部機能が使えなくなるが、実用上問題なし
"isolatedModules": true,

// =====
// jsx: JSX の変換方法
// =====
// "react-jsx" = React 17+ の新しいJSX変換を使用
// import React from 'react' を書かなくても JSX が使える
// (古い "react" 設定では毎ファイルで React のインポートが必要だった)
"jsx": "react-jsx",

// =====
// incremental: 差分コンパイルを有効化
// =====
// 前回のコンパイル結果をキャッシュして次回のコンパイルを高速化
// .tsbuildinfo ファイルが生成される
"incremental": true,

// =====
// plugins: TypeScript 言語サービスプラグイン
// =====
// Next.js プラグイン: エディタ (VS Code) で以下の機能を追加
// - Server Component / Client Component の自動判定
// - use client / use server の適切な使用チェック
// - next/link, next/image の補完強化
"plugins": [
  {
    "name": "next"
  }
],

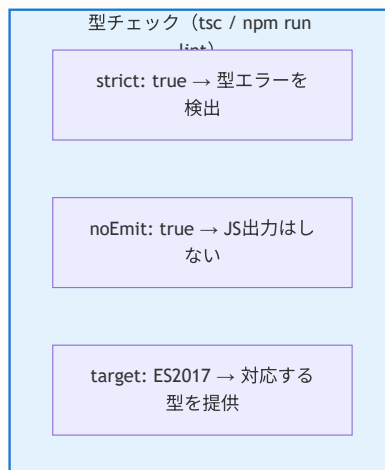
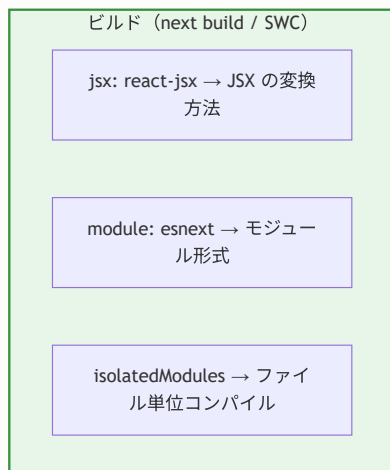
// =====
// paths: パスエイリアス (インポートパスの短縮)
// =====
// "@/*" → プロジェクトルートからの相対パス
//
// 例: import { prisma } from "@lib/db"
//     → 実際は ./lib/db を参照
//
// これがないと "../../..../lib/db" のような長い相対パスが必要になる
"paths": {
  "@/*": [
    "./*"
  ]
},

```

```
// include: コンパイル対象のファイルパターン
// =====
// "next-env.d.ts"          → Next.js の型定義ファイル (自動生成)
// "**/*.ts", "**/*.tsx"      → 全ての TypeScript ファイル
// "**/*.mts"                → ESM形式の TypeScript ファイル
// ".next/types/**/*.ts"    → Next.js が生成する型ファイル
// ".next/dev/types/**/*.ts" → 開発時に Next.js が生成する型ファイル
"include": [
  "next-env.d.ts",
  "**/*.ts",
  "**/*.tsx",
  "**/*.mts",
  ".next/types/**/*.ts",
  ".next/dev/types/**/*.ts"
],

// =====
// exclude: コンパイル対象外のディレクトリ
// =====
// "node_modules" → 依存パッケージ (膨大なため除外)
// ".next"         → ビルド出力 (コンパイル対象にする必要なし)
"exclude": [
  "node_modules",
  ".next"
]
}
```

tsconfig.json の各設定が影響する範囲:



BON-LOGでの使用箇所: `tsconfig.json` はコード全体の型チェックルールを定義します。 `strict: true` により全ファイルで厳格な型チェックが行われ、 `paths: { "@/*": ["./*"] }` により `import { prisma } from '@lib/db'` のような絶対パスインポートが使えます。 `types: ["vitest/globals"]` はテストファイルで `describe`、`it`、`expect` を import なしで使えるようにします。

実装しない場合の影響:

- `strict: false` にすると型チェックが緩くなりバグが実行時に初めて発見される
- `paths` がないと全ファイルで `../../../lib/db` のような長い相対パスが必要になる
- `types: ["vitest/globals"]` がないとテストファイルで型エラーが発生する

理解度チェック

- ☐ `strict: true` が有効化する具体的なチェック内容を3つ挙げられますか？
- ☐ `noEmit: true` が設定されている理由を説明できますか？
- ☐ `paths` の `@/*` エイリアスを使うメリットを説明できますか？
- ☐ `jsx: "react-jsx"` と古い `"react"` 設定の違いを説明できますか？

next.config.ts 完全解説

`next.config.ts` は Next.js アプリケーション全体の設定ファイルです。画像の最適化、外部サービスとの連携、ビルド設定などを定義します。

以下が BON-LOG プロジェクトの `next.config.ts` の全内容と、各設定の解説です。

```
// =====
// Next.js 設定ファイル
// =====
// TypeScript で書かれている (next.config.js でも可)
// 型の補完が効くため .ts 推奨

import type { NextConfig } from "next"; // Next.js設定の型定義をインポート
import { withSentryConfig } from "@sentry/nextjs"; // Sentry統合のラッパー関数

// =====
// メインの Next.js 設定オブジェクト
// =====
const nextConfig: NextConfig = {

  // =====
  // output: ビルド出力形式
  // =====
  // 'standalone' にすると、node_modules なしで動作する
  // 独立したビルド出力が生成される
  //
  // Docker でデプロイする際に必須:
  //   - コンテナサイズを大幅に削減 (数GB → 数百MB)
  //   - node_modules をコピーする必要がなくなる
  //
  // Vercel にデプロイする場合は自動的に無視される
  output: 'standalone',

  // =====
  // images: 画像最適化の設定
  // =====
  images: {

    // =====
    // remotePatterns: 外部画像の許可リスト
    // =====
    // Next.js の <Image> コンポーネントは、セキュリティのため
    // 外部ドメインの画像をデフォルトでブロックする
    // ここに許可するドメインパターンを列挙する
    remotePatterns: [
      // Cloudflare R2 (メインのストレージ)
      {
        protocol: 'https',
        hostname: '*.r2.dev', // R2 のパブリックURL
      },
      // Cloudflare R2 (カスタムドメイン使用時)
      {
        protocol: 'https',
        hostname: '*.r2.cloudflarestorage.com',
      },
      // Supabase Storage
      {
```

```

    protocol: 'https',
    hostname: '*.supabase.co',
  },
  // Unsplash (ランディングページの写真素材)
  {
    protocol: 'https',
    hostname: 'images.unsplash.com',
  },
],

// =====
// contentDispositionType: ダウンロード時の動作
// =====
// 'attachment' にすると、画像ファイルを直接ブラウザで開かず
// ダウンロードとして扱う (XSS対策)
contentDispositionType: 'attachment',

// =====
// contentSecurityPolicy: 画像用の CSP
// =====
// SVG に含まれるスクリプトの実行を完全にブロック
contentSecurityPolicy: "default-src 'self'; script-src 'none'; sandbox;",

// =====
// unoptimized: 画像最適化の無効化 (開発環境のみ)
// =====
// 開発環境ではプライベートIP (localhost) の制限を回避するため
// 画像最適化を無効化。本番環境では最適化が有効
unoptimized: process.env.NODE_ENV === 'development',
},

// =====
// experimental: 実験的機能
// =====
experimental: {
  serverActions: {
    // Server Actions のリクエストボディサイズ上限
    // デフォルトは 1MB だが、画像アップロードに対応するため 10MB に拡大
    bodySizeLimit: '10mb',
  },
},
};

// =====
// Sentry 統合設定
// =====
// withSentryConfig() で Next.js 設定をラップすると:
//   - ビルド時にソースマップが Sentry にアップロードされる
//   - エラー発生時にソースマップを使って元のコードの位置を表示
//   - パフォーマンスモニタリングが有効になる
export default withSentryConfig(nextConfig, {
  // Sentry 組織・プロジェクト名 (環境変数で設定)
  org: process.env.SENTRY_ORG,

```

```
project: process.env.SENTRY_PROJECT,  
  
// CI環境以外ではアップロードログを抑制（静かなビルド）  
silent: !process.env.CI,  
  
// クライアントサイドのソースマップアップロード範囲を拡大  
widenClientFileUpload: true,  
  
// ソースマップ関連の警告を抑制  
sourcemaps: {  
  ignore: [  
    '**/node_modules/**',           // 依存パッケージを除外  
    '**/*_client-reference-manifest*', // Next.js 内部ファイル  
    '**/*-manifest.js',             // マニフェストファイル  
  ],  
},  
  
// 広告ブロッカーを回避するトンネルルート  
// /monitoring パスを経由して Sentry にデータを送信  
// これにより、sentry.io ドメインがブロックされていても動作する  
tunnelRoute: '/monitoring',  
});
```

Image コンポーネント
src, width, height を指定



Next.js 画像最適化エンジ
ン

- ・ リサイズ: 自動縮小
- ・ フォーマット:
WebP/AVIF
- ・ 遅延読み込み
- ・ キャッシュ: 再利用



画像サイズ 50-80% 削減
ページ読み込み高速化

※ remotePatterns で許可されたドメインの画像のみ最適化対象

BON-LOGでの使用箇所: `next.config.ts` はすべてのページ・API ルートに影響します。特に `remotePatterns` は `<Image>` コンポーネントで Cloudflare R2 や Supabase の 画像を表示するために必須です。 `bodySizeLimit: '10mb'` は画像アップロードの Server Actions (`lib/actions/post.ts` 等) で必要です。

実装しない場合の影響:

- `remotePatterns` がいない場合: 外部URLの画像が「Invalid src prop」エラーになり、プロフィール画像・投稿画像が表示されない
- `bodySizeLimit` が未設定の場合: 大きな画像ファイルのアップロードが失敗する (デフォルトは 1MB のため、高解像度画像で 413 エラーが発生)
- `output: 'standalone'` がいない場合: Docker コンテナが巨大になりデプロイに時間がかかる

理解度チェック

- ☐ `output: 'standalone'` が Docker デプロイに必要な理由を説明できますか？
- ☐ `remotePatterns` に外部ドメインを追加する理由を説明できますか？
- ☐ `withSentryConfig()` がビルド時に何をしているか説明できますか？
- ☐ `bodySizeLimit: '10mb'` が必要な理由を説明できますか？

docker-compose.yml 完全解説

`docker-compose.yml` は Docker Compose の設定ファイルです。複数のコンテナ (サービス) を定義し、一括で起動・停止できます。

ここがポイント！

Docker は「アプリケーションを箱 (コンテナ) に入れて動かす」技術です。Docker Compose はその「箱の組み合わせ」を定義するファイルです。

「PostgreSQL と Next.js を
一緒に起動したい」



docker-compose.yml に定
義
services:
postgres: データベースの
箱
app: アプリケーションの
箱



docker compose up -d
← このコマンド1つで全部
起動！

以下が BON-LOG プロジェクトの `docker-compose.yml` の全内容と解説です。

```

# =====
# BON-LOG Docker Compose
# ローカル開発環境用
# =====

# services: 起動するコンテナの定義
# 各サービスが1つのコンテナに対応する
services:

# =====
# [1] PostgreSQL データベース
# =====
# 常に起動される基本サービス（プロファイル指定なし）
postgres:

# 使用する Docker イメージ
# postgres:16-alpine = PostgreSQL 16 の軽量版（Alpine Linux ベース）
# Alpine 版は通常版の約1/3のサイズ（約80MB vs 約250MB）
image: postgres:16-alpine

# コンテナに付ける名前（docker ps で表示される）
container_name: bonsai-postgres

# 再起動ポリシー：異常終了時は自動再起動、手動停止時は再起動しない
restart: unless-stopped

# PostgreSQL の環境変数（初回起動時にDB作成に使用）
environment:
  POSTGRES_USER: postgres          # データベースユーザー名
  POSTGRES_PASSWORD: postgres      # パスワード（開発用なので簡易）
  POSTGRES_DB: bonsai_sns          # 作成するデータベース名

# ポートマッピング：ホスト側:コンテナ側
# localhost:5432 でコンテナ内の PostgreSQL に接続できる
ports:
  - "5432:5432"

# ボリューム：データの永続化
# コンテナを停止・削除してもデータが残る
# postgres_data という名前付きボリュームに PostgreSQL のデータを保存
volumes:
  - postgres_data:/var/lib/postgresql/data

# ヘルスチェック：コンテナの正常動作を定期的に確認
# 他のサービスが「PostgreSQLが起動完了するまで待つ」のに使用
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres -d bonsai_sns"]
  interval: 10s      # 10秒ごとにチェック
  timeout: 5s        # 5秒以内に応答がなければ失敗
  retries: 5         # 5回連続失敗でunhealthy判定

```

```

# =====
# [2] Next.js アプリケーション（開発モード）
# =====
# profiles: [dev] → docker compose --profile dev up で起動
app:

  # Dockerfile.dev を使ってイメージをビルド
  build:
    context: .                # ビルドコンテキスト（プロジェクトルート）
    dockerfile: Dockerfile.dev # 開発用の Dockerfile

  container_name: bonsai-app

  restart: unless-stopped

  # localhost:3000 でアクセス可能
  ports:
    - "3000:3000"

  # アプリケーションの環境変数
  environment:
    # データベース接続URL
    # "postgres" はサービス名で、Docker内部DNSにより名前解決される
    - DATABASE_URL=postgres://postgres:postgres@postgres:5432/bonsai_sns
    - NEXTAUTH_URL=http://localhost:3000
    - NEXTAUTH_SECRET=docker-dev-secret-change-in-production
    - NEXT_PUBLIC_APP_URL=http://localhost:3000
    - STORAGE_PROVIDER=local      # ローカルファイル保存（開発用）
    - EMAIL_PROVIDER=console      # メールをコンソール出力（開発用）

  # ボリュームマウント
  volumes:
    - ./app                # ソースコードをコンテナにマウント（ホットリロード用）
    - /app/node_modules    # node_modules はコンテナ内のものを使用
    - /app/.next            # .next もコンテナ内のものを使用

  # 依存関係: PostgreSQL が healthy になるまで待機
  depends_on:
    postgres:
      condition: service_healthy

  # プロファイル: --profile dev を指定した場合のみ起動
  profiles:
    - dev

# =====
# [3] Next.js アプリケーション（本番モード）
# =====
# profiles: [prod] → docker compose --profile prod up で起動
app-prod:

  # 本番用の Dockerfile を使用
  build:

```

```
context: .
dockerfile: Dockerfile    # 本番用（マルチステージビルド、最適化済み）

container_name: bonsai-app-prod

restart: unless-stopped

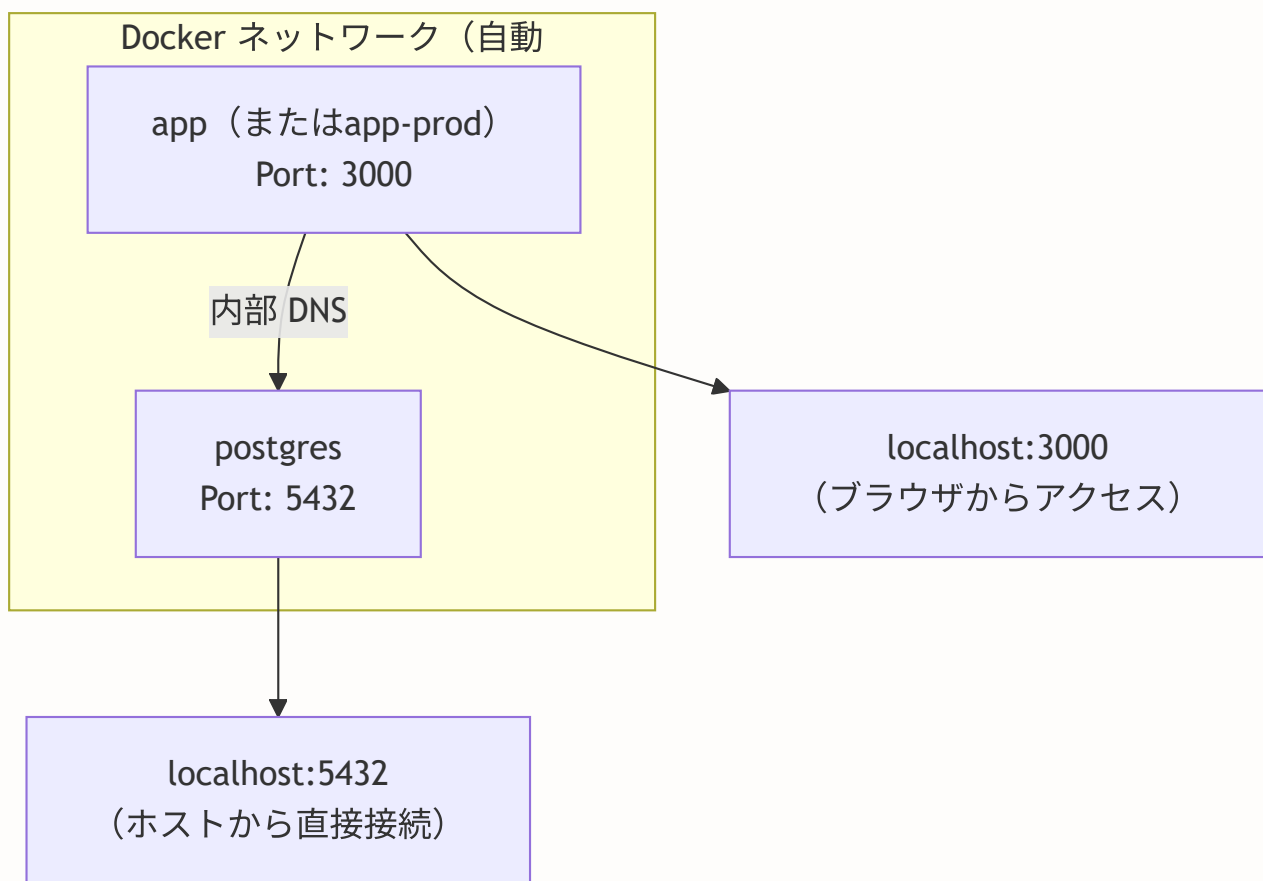
ports:
  - "3000:3000"

environment:
  - DATABASE_URL=postgresql://postgres:postgres@postgres:5432/bonsai_sns
  - NEXTAUTH_URL=http://localhost:3000
  - NEXTAUTH_SECRET=docker-dev-secret-change-in-production
  - NEXT_PUBLIC_APP_URL=http://localhost:3000

depends_on:
  postgres:
    condition: service_healthy

profiles:
  - prod

# =====
# ボリューム定義
# =====
# 名前付きボリュームでデータを永続化
# docker compose down -v でボリュームも削除される
volumes:
  postgres_data:    # PostgreSQL のデータを保存するボリューム
```



※ コンテナ間では "postgres" というサービス名で接続できる ※ ホスト（あなたのPC）からは "localhost" で接続

プロファイルによる起動パターン:

```
docker compose up -d postgres
→ PostgreSQL のみ起動（最も一般的な開発パターン）
→ Next.js はローカルで npm run dev
```

```
docker compose --profile dev up -d
→ PostgreSQL + Next.js（開発モード）を起動
→ ソースコード変更が即座に反映（ホットリロード）
```

```
docker compose --profile prod up -d
→ PostgreSQL + Next.js（本番モード）を起動
→ 本番ビルドの動作確認用
```

BON-LOGでの使用箇所: `docker-compose.yml` はローカル開発環境専用の設定ファイルです。開発者が `docker compose up -d postgres` を実行することで、PostgreSQL 16 がローカルの 5432 番ポートで起動し、`.env.local` の `DATABASE_URL` と接続します。Next.js アプリ本体はローカルで `npm run dev` として起動するのが標準的な開発パターンです。

実装しない場合の影響: データベースがない状態になり、アプリケーションが起動しても全ての投稿・ユーザーデータの読み書きが失敗します。Docker を使わない場合は、ローカルに PostgreSQL を直接インストールするか、本番の Supabase に接続することで代替できますが、開発環境の再現性が低下します。

理解度チェック

- ☐ `docker compose up -d postgres` で何が起動するか説明できますか？
- ☐ `volumes: postgres_data` の役割（なぜ必要か）を説明できますか？
- ☐ `depends_on` と `healthcheck` の関係を説明できますか？
- ☐ `profiles` の仕組みと使い分けを説明できますか？

`.env.local.example` 完全解説

`.env.local.example` は環境変数のテンプレートファイルです。このファイルを `.env.local` にコピーして、実際の値を設定します。

ここがポイント！

環境変数とは「アプリケーションの外部から設定を注入する仕組み」です。パスワードやAPIキーなどの機密情報をソースコードに直接書かず、環境変数として管理することでセキュリティを保ちます。

`.env.local.example`
テンプレート
→ Gitに含める

コピーして設定

`.env.local`
機密情報を含む
→ Gitに含めない！

Next.jsが自動読み込み

`process.env.xxx`
でアクセス

コード内で使用

PrismaClient 等

NEXT_PUBLIC_ プレフィックスの重要な違い:

```

NEXT_PUBLIC_APP_URL=http://localhost:3000
↑ "NEXT_PUBLIC_" で始まる → ブラウザ（クライアント側）でも使える
  → JavaScript バンドルに含まれる → ユーザーに見える！
  → 機密情報は絶対に NEXT_PUBLIC_ で始めてはいけない

NEXTAUTH_SECRET=your-secret-key
↑ "NEXT_PUBLIC_" なし → サーバー側でのみ使える
  → ブラウザからはアクセス不可 → 安全

```

以下、各環境変数をカテゴリごとに解説します。

[A] データベース（PostgreSQL）

環境変数	値の例	説明
DATABASE_URL	postgresql://postgres:postgres@localhost:5432/bonsai_sns	PostgreSQL 接続URL。Prisma がデータベースに接続するために使用
DIRECT_URL	postgresql://postgres.[ref]:[pass]@... supabase.com:5432/postgres	Supabase使用時の直接接続URL。マイグレーション実行時に使用（PgBouncer非経由）

部分	値の例	説明
<code>postgresql://</code>	プロトコル	PostgreSQL接続を意味する
<code>postgres</code>	ユーザー名	DB接続ユーザー
<code>postgres</code>	パスワード	DB接続パスワード
<code>localhost</code>	ホスト	接続先サーバー
<code>5432</code>	ポート	PostgreSQLのポート番号
<code>bonsai_sns</code>	DB名	データベース名

環境	接続先	説明
開発環境 (Docker)	<code>postgresql://postgres:postgres@localhost:5432/bonsai_sns</code>	ローカルの Docker コンテナに接続
本番環境 (Supabase)	<code>postgresql://postgres.[ref]:[pass]@... pooler.supabase.com:6543/postgres?pgbouncer=true</code>	Supabase の クラウド DB に接続（接続 プール経由）

[B] NextAuth.js（認証）

環境変数	値の例	説明
<code>NEXTAUTH_URL</code>	<code>http://localhost:3000</code>	アプリケーションのベースURL。コールバックURLの生成に使用
<code>NEXTAUTH_SECRET</code>	<code>openssl rand -base64 32</code> で生成	JWT トークンの署名に使う秘密鍵。漏洩するとセッション偽造が可能になるため、本番では必ず強力な値を設定

警告: NEXTAUTH_SECRET のセキュリティ

本番環境では必ず以下のコマンドで生成した強力な値を使用してください:

```
openssl rand -base64 32
```

弱いシークレットを使用すると、JWT トークンが偽造され、アカウント乗っ取りやセッションハイジャックのリスクがあります。

[C] アプリケーション

環境変数	値の例	説明
NEXT_PUBLIC_APP_URL	http://localhost:3000	公開用のアプリURL。OGP画像のURL生成、メール内のリンク等に使用。NEXT_PUBLIC_ なのでクライアント側でも使用可能

[D] ストレージ（画像アップロード）

環境変数	値の例	説明
STORAGE_PROVIDER	local / r2 / supabase	ストレージの種類を選択。local は開発用（ローカルファイル保存）、r2 は本番推奨
R2_ACCOUNT_ID	Cloudflare のアカウントID	Cloudflare R2 の接続に必要
R2_ACCESS_KEY_ID	R2 の API トークン ID	S3互換APIの認証キー
R2_SECRET_ACCESS_KEY	R2 の API シークレット	S3互換APIの秘密鍵
R2_BUCKET_NAME	bonsai-uploads	R2 バケット名
R2_PUBLIC_URL	https://pub-xxx.r2.dev	R2 バケットの公開URL。画像表示に使用

環境	STORAGE_PROVIDER	特徴
開発環境	<code>local</code>	設定不要、即座に使える。画像は public/uploads/ に保存。再起動で消えない
本番環境 (推奨)	<code>r2</code>	Cloudflare R2 (S3互換の低コストストレージ)。月10GBまで無料。CDN配信で高速。要: R2_ACCOUNT_ID, R2_ACCESS_KEY_ID 等の設定

[E] Redis (キャッシュ・レート制限)

環境変数	値の例	説明
<code>UPSTASH_REDIS_REST_URL</code>	<code>https://xxx.upstash.io</code>	Upstash Redis の REST API エンドポイント
<code>UPSTASH_REDIS_REST_TOKEN</code>	<code>AXxx ...</code>	Upstash Redis の認証トークン

Redis が未設定の場合:

Redis の環境変数が設定されていない場合、レート制限機能が無効化されます。開発環境では問題ありませんが、本番環境ではスパム対策のため設定を推奨します。

[F] メール送信

環境変数	値の例	説明
<code>EMAIL_PROVIDER</code>	<code>console</code> / <code>resend</code> / <code>azure</code>	メール送信方法。 <code>console</code> は開発用 (コンソール出力)、 <code>resend</code> は本番推奨
<code>RESEND_API_KEY</code>	<code>re_XXXXXXXXXXXX</code>	Resend の API キー
<code>EMAIL_FROM</code>	<code>BON-LOG</code> <code><noreply@resend.dev></code>	送信元メールアドレス

[G] 検索設定

環境変数	値の例	説明
<code>SEARCH_MODE</code>	<code>like</code> / <code>trgm</code> / <code>bigm</code>	全文検索モード。 <code>like</code> はデフォルト (LIKE検索)、 <code>trgm</code> は PostgreSQL の pg_trgm 拡張使用 (推奨)

[H] Stripe（決済）

環境変数	値の例	説明
<code>STRIPE_SECRET_KEY</code>	<code>sk_test_xxx</code> / <code>sk_live_xxx</code>	Stripe の秘密鍵。 <code>sk_test_</code> はテスト用、 <code>sk_live_</code> は本番用
<code>STRIPE_WEBHOOK_SECRET</code>	<code>whsec_xxx</code>	Webhook の署名検証用シークレット
<code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code>	<code>pk_test_xxx</code>	Stripe の公開鍵（クライアント側で使用）
<code>STRIPE_PRICE_ID_MONTHLY</code>	<code>price_xxx</code>	月額プランの価格ID
<code>STRIPE_PRICE_ID_YEARLY</code>	<code>price_xxx</code>	年額プランの価格ID

[I] エラー監視（Sentry）

環境変数	値の例	説明
<code>SENTRY_DSN</code>	<code>https://xxx@xxx.ingest.sentry.io/xxx</code>	Sentry のデータソース名（サーバー側エラー送信先）
<code>NEXT_PUBLIC_SENTRY_DSN</code>	<code>https://xxx@xxx.ingest.sentry.io/xxx</code>	Sentry DSN のクライアント版（ブラウザ側エラー送信先）
<code>SENTRY_AUTH_TOKEN</code>	<code>sntrys_xxx</code>	ソースマップアップロード用の認証トークン

[J] 広告設定

環境変数	値の例	説明
<code>NEXT_PUBLIC_AD_PROVIDER</code>	<code>ninja</code> / <code>adsense</code>	広告プロバイダー。 <code>ninja</code> は忍者AdMax（審査不要）、 <code>adsense</code> は Google AdSense
<code>NEXT_PUBLIC_NINJA_AD_ID_SIDEAR</code>	広告枠ID	忍者AdMax サイドバー広告のID
<code>NEXT_PUBLIC_NINJA_AD_ID_INFEED</code>	広告枠ID	忍者AdMax インフィード広告のID
<code>NEXT_PUBLIC_NINJA_AD_ID_POST_DETAIL</code>	広告枠ID	忍者AdMax 投稿詳細広告のID

理解度チェック

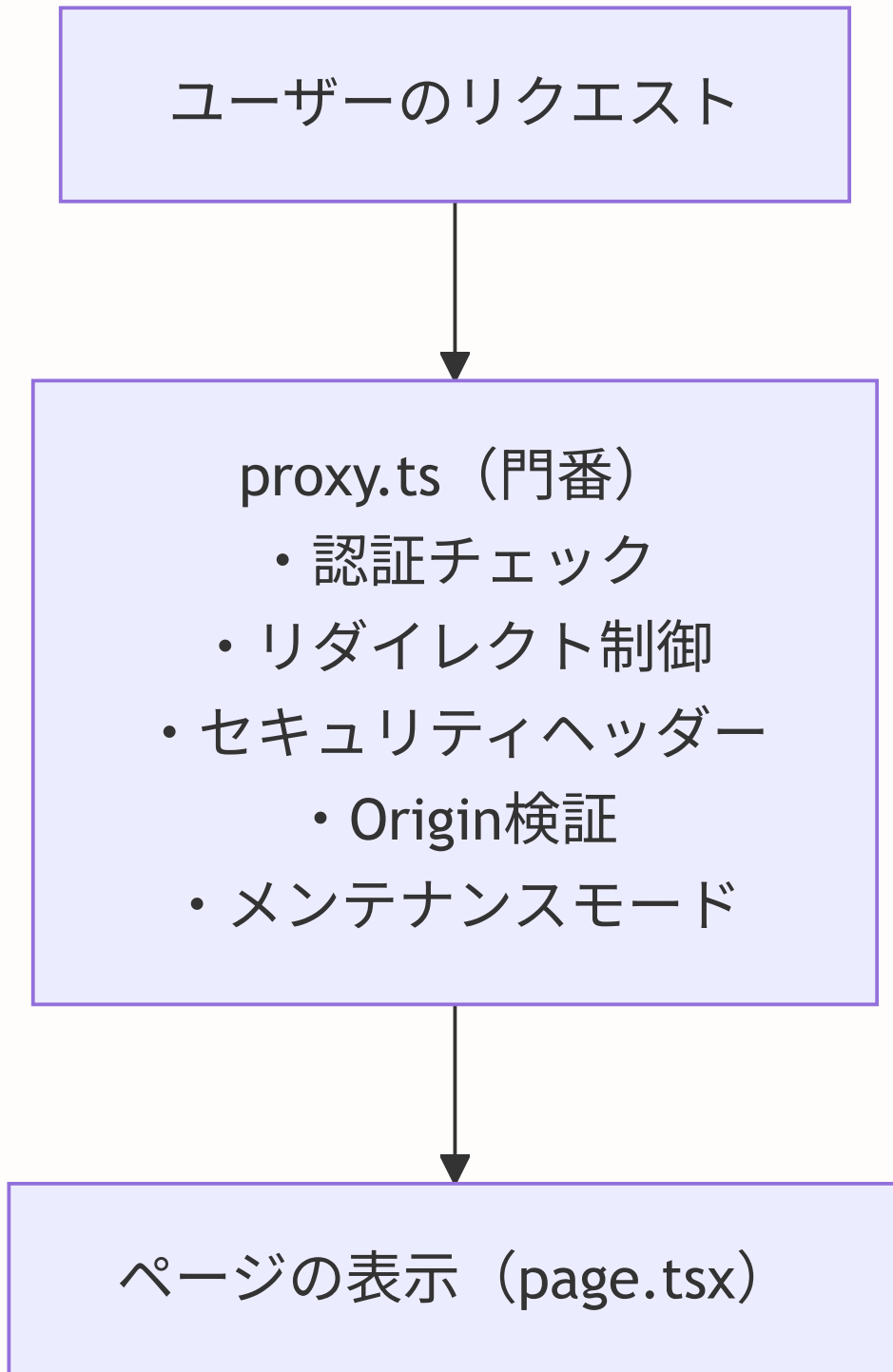
- ☐ `.env.local` がなぜ `.gitignore` に含まれているか説明できますか？
- ☐ `NEXT_PUBLIC_` プレフィックスの有無でどう変わるか説明できますか？
- ☐ `DATABASE_URL` の各パートの意味を説明できますか？
- ☐ 開発環境と本番環境で `STORAGE_PROVIDER` を変える理由を説明できますか？

proxy.ts 概要

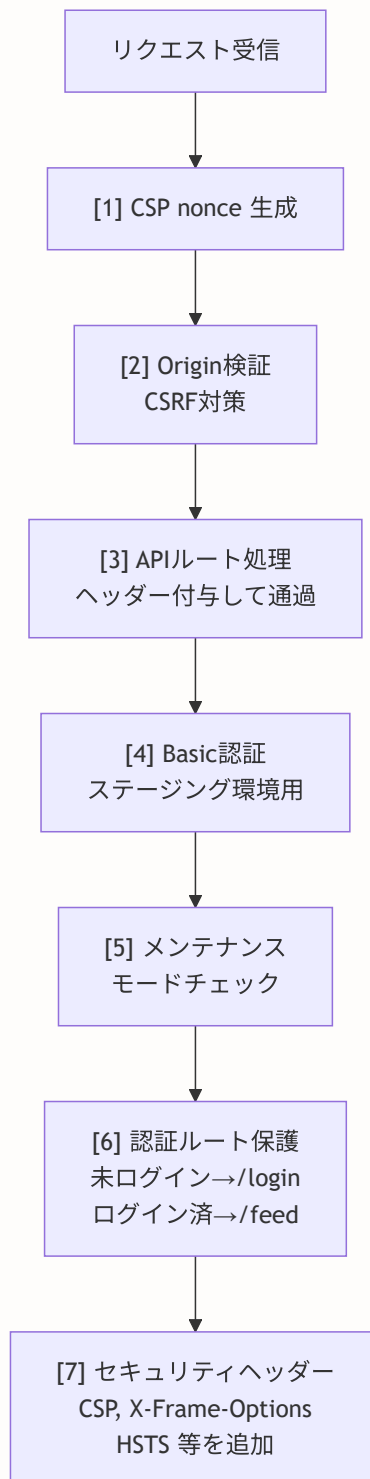
`proxy.ts` はプロジェクトルートに配置される特殊なファイルで、すべてのリクエストが処理される前に実行されます（Next.js 16の proxy）。

ここがポイント！

proxy.ts は「門番」のような役割です。ユーザーがどのページにアクセスしても、まず proxy を通過します。



BON-LOG の proxy.ts は以下の機能を持っています。




```
// matcher: proxy が適用されるパスのパターン
export const config = {
  matcher: [
    // 静的ファイル（画像、CSS等）以外の全てのリクエストに適用
    '/((?!_next/static|_next/image|favicon.ico|site\\.webmanifest|.*\\.?(?:svg|png|jpg|jpeg|
  ],
}
```

注意: proxy.ts の詳細な実装については第5章（認証）で詳しく解説します。ここでは全体像を把握しておけば十分です。

理解度チェック

- ☐ proxy.ts がリクエスト処理のどの段階で実行されるか説明できますか？
- ☐ `matcher` 設定で静的ファイルが除外されている理由を説明できますか？
- ☐ 未認証ユーザーが `/feed` にアクセスしたとき何が起こるか説明できますか？

eslint.config.mjs 解説

`eslint.config.mjs` は ESLint の設定ファイルです。コードの品質チェックとスタイル統一のルールを定義します。

ここがポイント！

ESLint は「コードの文法チェッカー」です。プログラムとしてはエラーにならないが、バグの原因になりやすいコードや、チームの規約に沿わないコードを自動的に検出します。

ESLint の役割:

あなたのコード → ESLint でチェック → 問題を報告

例:

```
const x = 5; // xを使わずに放置
```

→ 警告: 'x' is assigned a value but never used

```
if (user = null) // = と == の間違い
```

→ エラー: Expected a conditional expression

以下が BON-LOG プロジェクトの `eslint.config.mjs` の全内容と解説です。

```
// =====
// ESLint 設定ファイル (Flat Config 形式)
// =====
// ESLint 9 から導入された新しい設定形式
// 従来の .eslintrc.json に代わるもの
// .mjs 拡張子 = ESMODULE 形式 (import/export が使える)

// defineConfig: 型補完を有効にするヘルパー関数
// globalIgnores: プロジェクト全体で無視するファイルパターン
import { defineConfig, globalIgnores } from "eslint/config";

// Next.js 公式の ESLint ルール集
// core-web-vitals: パフォーマンスに関するルール (画像最適化、フォント読み込み等)
import nextVitals from "eslint-config-next/core-web-vitals";

// Next.js + TypeScript 用のルール集
// TypeScript 固有のベストプラクティス (型安全性等)
import nextTs from "eslint-config-next/typescript";

const eslintConfig = defineConfig([
  // =====
  // [1] Next.js Core Web Vitals ルール
  // =====
  // - next/image を使わない <img> タグの検出
  // - next/link を使わない <a> タグの検出
  // - next/script を使わない <script> タグの検出
  // - パフォーマンスに影響する問題の検出
  ...nextVitals,

  // =====
  // [2] Next.js + TypeScript ルール
  // =====
  // - @typescript-eslint/no-unused-vars (未使用変数の検出)
  // - @typescript-eslint/no-explicit-any (any 型の検出)
  // - その他 TypeScript のベストプラクティス
  ...nextTs,

  // =====
  // [3] グローバル無視パターン
  // =====
  // ESLint のチェック対象から除外するファイル/ディレクトリ
  globalIgnores([
    ".next/**", // Next.js ビルド出力
    "out/**", // 静的エクスポート出力
    "build/**", // ビルド出力
    "next-env.d.ts", // Next.js 自動生成の型定義
    "coverage/**", // テストカバレッジレポート
  ]),

  // =====
  // [4] カスタムルール
```

```
// -----
{
  rules: {
    // アンダースコアで始まる変数は未使用でも警告しない
    // 例: const _unusedVar = someFunction()
    //     → 意図的に使わない変数であることを明示
    //
    // argsIgnorePattern: 関数の引数
    // 例: function handle(_event, data) { ... }
    //
    // varsIgnorePattern: 変数宣言
    // 例: const _temp = calculation()
    //
    // caughtErrorsIgnorePattern: catch のエラー変数
    // 例: try { ... } catch (_err) { ... }
    "@typescript-eslint/no-unused-vars": [
      "warn", // "error" ではなく "warn" (ビルドは止めない)
      {
        argsIgnorePattern: "^_",
        varsIgnorePattern: "^_",
        caughtErrorsIgnorePattern: "^_",
      },
    ],
  },
},
},
});

export default eslintConfig;
```

ESLint の実行タイミング:

- [1] エディタ上でリアルタイムチェック
VS Code + ESLint 拡張機能
→ コードを書いた瞬間に赤/黄色の波線で警告
- [2] コマンドで手動実行
npm run lint
→ プロジェクト全体をチェック
- [3] CI/CD で自動実行
GitHub Actions で PR 作成時に自動チェック
→ エラーがあると PR がマージできない

理解度チェック

- ☐ ESLint が検出する問題の具体例を3つ挙げられますか？
- ☐ `_` で始まる変数名が特別扱いされる理由を説明できますか？
- ☐ `globalIgnores` で `.next/**` を除外している理由を説明できますか？

vitest.config.ts と vitest.setup.tsx 解説

vitest.config.ts (テスト設定ファイル)

`vitest.config.ts` は Vitest テストフレームワークの設定ファイルです。テストファイルの検索パターン、テスト環境、カバレッジ設定などを定義します。

```
// =====
// Vitest 設定ファイル
// =====
// Vitest では vite の設定を直接利用できる:
//   - SWC による TypeScript/JSX の高速変換
//   - next.config.ts の読み込み
//   - .env ファイルの読み込み
//   - パスエイリアス(@/*)の解決

// Vitest では next/jest のような前処理は不要
// Vite が TypeScript/JSX の変換やパスエイリアスの解決を自動的に行う

/** @type {import('vitest').Config} */
const customConfig = {

  // =====
  // testMatch: テストファイルのパターン
  // =====
  // 以下のパターンに一致するファイルをテスト対象とする:
  //   __tests__/ フォルダ内の .test.ts, .test.tsx, .spec.ts 等
  //   任意の場所の .test.ts, .spec.ts 等
  testMatch: [
    '**/__tests__/**/*.test.ts',
    '**/**/*.test.ts',
  ],

  // =====
  // testEnvironment: テスト実行環境
  // =====
  // 'jsdom' = ブラウザ環境をシミュレート
  // document, window, localStorage 等が使える
  // React コンポーネントのテストに必須
  testEnvironment: 'jsdom',

  // =====
  // setupFilesAfterEnv: テスト前に実行するセットアップファイル
  // =====
  // vitest.setup.tsx でグローバルモック等を設定
  // 全てのテストファイルの前に実行される
  setupFilesAfterEnv: ['<rootDir>/vitest.setup.tsx'],

  // =====
  // moduleNameMapper: モジュール名のエイリアス解決
  // =====
  // @/ で始まるインポートをプロジェクトルートに解決
  // 例: @/lib/db → <rootDir>/lib/db
  moduleNameMapper: {
    '^@/(.*)$': '<rootDir>/$1',
  },
}
```

```

// -----
// collectCoverageFrom: カバレッジ計測対象
// -----
// lib/, components/, app/ 内の TypeScript ファイルが対象
// 型定義(.d.ts)、node_modules、テストファイル自体は除外
collectCoverageFrom: [
  'lib/**/*.{ts,tsx}',
  'components/**/*.{ts,tsx}',
  'app/**/*.{ts,tsx}',
  '!**/*.d.ts',
  '!**/node_modules/**',
  '!**/__tests__/**',
],

// -----
// coverageThreshold: カバレッジの最低ライン
// -----
// この値を下回るとテストが失敗する
// 段階的に引き上げていくことを推奨
coverageThreshold: {
  global: {
    branches: 30,    // 条件分岐の 30% 以上をテスト
    functions: 30,   // 関数の 30% 以上をテスト
    lines: 35,       // コード行の 35% 以上をテスト
    statements: 35,  // 文の 35% 以上をテスト
  },
},

// テストタイムアウト: 10秒 (デフォルトは5秒)
// DB接続を含むテスト等、時間がかかるテスト用に余裕を持たせる
testTimeout: 10000,

// -----
// transformIgnorePatterns: 変換対象外の指定
// -----
// 通常、node_modules 内のファイルは変換しない
// ただし ESMODULE で書かれたパッケージは変換が必要
// ここに列挙されたパッケージは変換対象に「含める」
transformIgnorePatterns: [
  '/node_modules/(?!(isomorphic-dompurify|dompurify|@panva|jose|nanoid|uuid|next-auth|@a',
  '^\\.\\.\\.module\\.\\.\\.css|sass|scss)$',
],

// モジュール検索ディレクトリ
moduleDirectories: ['node_modules', '<rootDir>/'],

// テスト対象外のパス
testPathIgnorePatterns: [
  '<rootDir>/node_modules/',
  '<rootDir>/.next/',
  '<rootDir>/e2e/',           // E2Eテストは Playwright で実行
],

```

```
// モジュール解決で無視するパス
modulePathIgnorePatterns: [
  '<rootDir>/.next/',
],

// Haste衝突を防ぐ設定（大規模プロジェクトで稀に発生）
haste: {
  forceNodeFilesystemAPI: true,
},
}

// Vitest の設定をエクスポート
export default customConfig
```

vitest.setup.tsx（テストセットアップファイル）

`vitest.setup.tsx` は全てのテストの前に実行されるセットアップファイルです。グローバルモック（テスト用の偽の実装）を定義します。


```

// =====
// Vitest セットアップファイル
// =====
// 全テストファイルの実行前に1回だけ読み込まれる
// ここでグローバルなモック（偽の実装）を設定する

// =====
// [1] ポリフィル（互換性のための補完）
// =====
// TextEncoder/TextDecoder は Node.js の util モジュールから取得
// jsdom 環境では標準で利用できない場合があるため
import { TextEncoder, TextDecoder } from 'util'
global.TextEncoder = TextEncoder
global.TextDecoder = TextDecoder

// =====
// [2] Vitest DOM 拡張マッチャー
// =====
// toBeInTheDocument(), toHaveTextContent() 等の
// DOM要素用のアサーションメソッドを追加
import '@testing-library/jest-dom'

// =====
// [3] データベースモック (Prisma)
// =====
// テスト時に実際のDBに接続しないように、
// prisma の各メソッドをモック（偽の関数）に置き換え
vi.mock('@lib/db', () => ({
  prisma: {
    user: {
      findUnique: vi.fn(), // 呼び出しを記録するだけの偽関数
      findMany: vi.fn(),
      create: vi.fn(),
      update: vi.fn(),
      delete: vi.fn(),
    },
    post: { /* 同様 */ },
    follow: { /* 同様 */ },
    followRequest: { /* 同様 */ },
    $transaction: vi.fn(),
  },
}))

// =====
// [4] 認証モック (NextAuth.js)
// =====
// テスト時は常に認証済み状態をシミュレート
vi.mock('@lib/auth', () => ({
  auth: vi.fn().mockResolvedValue({
    user: {
      id: 'test-user-id',

```

```

        name: 'Test User',
        email: 'test@example.com',
      },
    },
    signIn: vi.fn(),
    signOut: vi.fn(),
  })

// -----
// [5] Next.js モック
// -----
// useRouter, useSearchParams 等のフックをモック化
// テスト環境ではルーティングが存在しないため
vi.mock('next/navigation', () => ({
  useRouter: () => ({
    push: vi.fn(),      // ページ遷移の代わりに呼び出しを記録
    replace: vi.fn(),
    back: vi.fn(),
    refresh: vi.fn(),
  }),
  useSearchParams: () => new URLSearchParams(),
  usePathname: () => '/',
}))

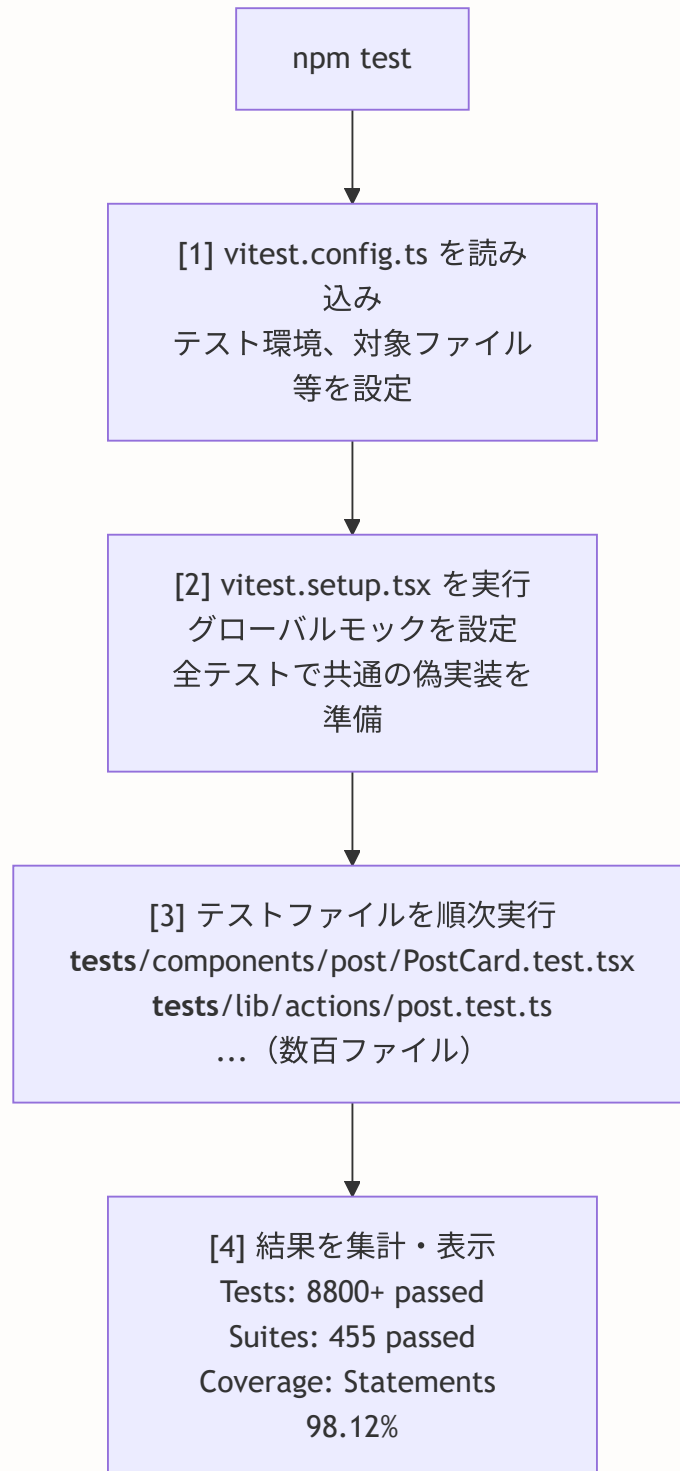
// next/image → 通常の <img> タグに置き換え
// next/link → 通常の <a> タグに置き換え
// (テスト環境では最適化機能が不要なため)

// -----
// [6] React Query モック
// -----
// useQuery, useMutation をモック化
// テスト時にAPIリクエストを送信しない
vi.mock('@tanstack/react-query', () => ({
  useQuery: vi.fn().mockReturnValue({
    data: undefined,
    isLoading: false,
    error: null,
  }),
  useMutation: vi.fn().mockReturnValue({
    mutate: vi.fn(),
    isLoading: false,
  }),
}))

// -----
// [7] ブラウザ API モック
// -----
// jsdom 環境で利用できないブラウザ API を偽装
if (typeof window !== 'undefined') {
  // IntersectionObserver: 要素の表示/非表示検知 (無限スクロール用)
  // matchMedia: メディアクエリの判定 (レスポンシブ対応)
  // ResizeObserver: 要素サイズの変化検知

```

```
// scrollTo: スクロール操作
// localStorage: ローカルストレージ
}
```



ここがポイント！

テストで「モック」を使う理由:

実際のテスト対象: PostCard コンポーネント

テストなし（実環境）:

PostCard → Prisma → PostgreSQL → 実データ

→ テストが遅い、DBの状態に依存、不安定

モック使用（テスト環境）:

PostCard → モック（偽のPrisma）→ 固定データを返す

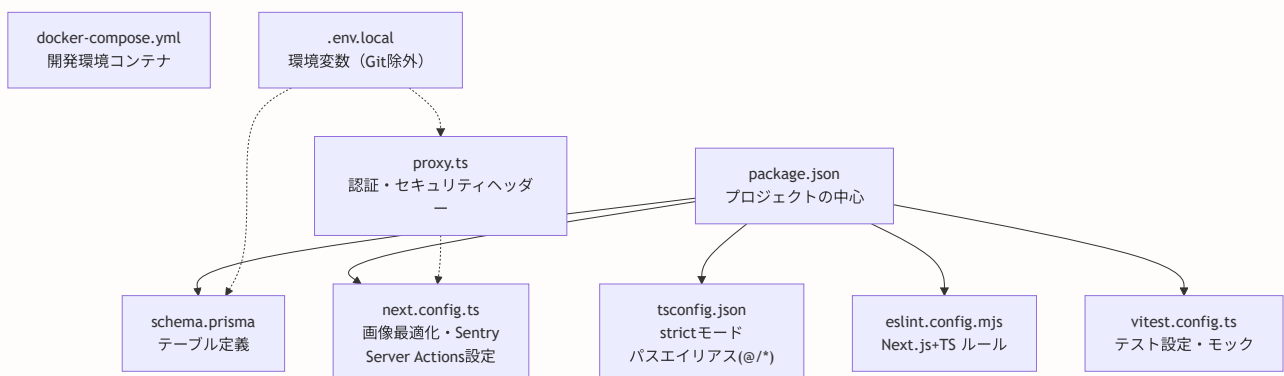
→ テストが高速、安定、再現性あり

理解度チェック

- ☐ `vitest.config.ts` の `testEnvironment: 'jsdom'` が必要な理由を説明できますか？
- ☐ `vitest.setup.tsx` でグローバルモックを設定する理由を説明できますか？
- ☐ `transformIgnorePatterns` で一部のパッケージを除外する理由を説明できますか？
- ☐ テストカバレッジの `branches` と `lines` の違いを説明できますか？
- ☐ `vi.fn()` が何をするものか説明できますか？

設定ファイル群のまとめ

ここまで解説した設定ファイルの関係を整理しましょう。



ここがポイント！

設定ファイルは一度理解すれば、普段の開発で頻繁に触ることはありません。最も重要なのは以下の3つです：

1. **package.json** - パッケージの追加・スクリプトの実行
2. **.env.local** - 環境変数の設定（新しいサービス導入時）
3. **next.config.ts** - Next.js の設定変更（画像ドメイン追加等）

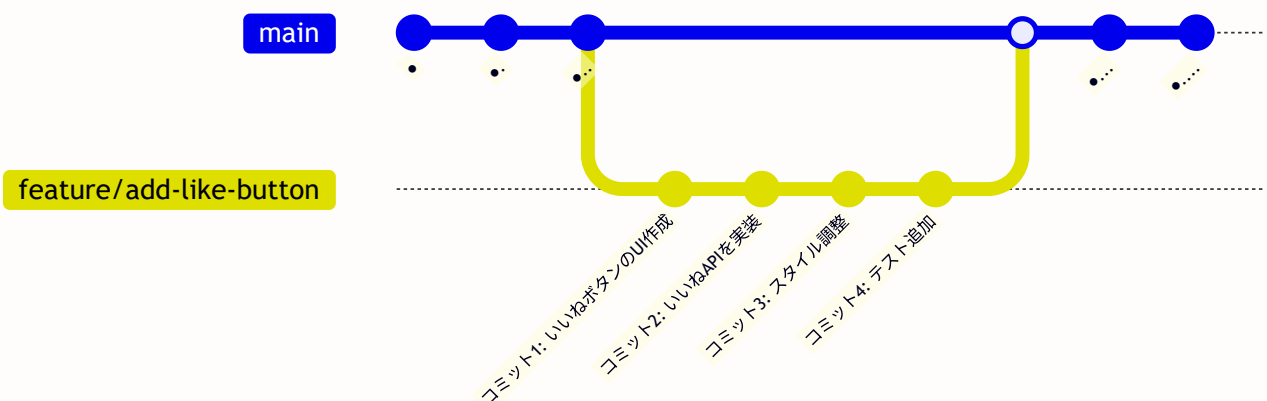
1.15 開発ワークフローガイド（Git ブランチ運用・プルリクエスト・コードレビュー）

このセクションで学ぶこと

- Git ブランチとは何か、なぜ使うのか
- BON-LOG で採用するブランチ運用ルール（GitHub Flow）
- プルリクエスト（PR）の作成手順と書き方
- コードレビューの受け方・やり方
- マージからデプロイまでの流れ

ブランチとは何か

Git の「ブランチ（branch）」は、コードの変更を**本流から分離して行う仕組み**です。木の幹から枝が分かれるイメージで理解できます。



- main ブランチは常に「動く状態」を保つ
- 新機能の開発は「機能ブランチ」で行う
- 完成したら main にマージ（統合）する

なぜブランチを使うのか？

ブランチを使わずに main に直接コミットすると、以下の問題が起こります：

- 開発途中のバグが本番環境に反映されてしまう
- 複数人が同時に編集すると、頻繁に競合（コンフリクト）が発生する
- 「この変更をやっぱりやめたい」時に、どこまで戻せばいいか分からなくなる

ブランチを使えば、これらの問題をすべて回避できます。

BON-LOG のブランチ運用ルール（GitHub Flow）

BON-LOG では **GitHub Flow**（ギットハブフロー）という、シンプルで広く使われているブランチ運用ルールを採用します。

GitHub Flow の流れ:

1. main から新しいブランチを作成
`git checkout -b feature/add-like-button`
2. ブランチで開発・コミットを繰り返す
`git add .`
`git commit -m "いいねボタンのUI作成"`
3. GitHub にプッシュ
`git push origin feature/add-like-button`
4. プルリクエスト (PR) を作成
GitHub上で「このブランチをmainにマージしてください」と申請
5. コードレビュー
チームメンバーがコードを確認・フィードバック
6. マージ
レビューが承認されたら main にマージ
7. デプロイ
main ブランチの変更が自動的に本番環境に反映

ブランチの命名規則

ブランチ名は「何の作業をしているか」が一目で分かるようにします。

ブランチ名の構成: **種類/説明** (種類 = 作業の種類、説明 = 何をするか (英語、ハイフン区切り))

ブランチ名の例	意味
<code>feature/add-like-button</code>	新機能: いいねボタンの追加
<code>feature/user-profile-page</code>	新機能: ユーザープロフィールページ
<code>fix/login-redirect-error</code>	バグ修正: ログイン後のリダイレクトエラー
<code>fix/image-upload-crash</code>	バグ修正: 画像アップロード時のクラッシュ
<code>refactor/post-card-component</code>	リファクタリング: PostCardコンポーネントの整理
<code>docs/update-readme</code>	ドキュメント: READMEの更新
<code>chore/update-dependencies</code>	雑務: 依存パッケージのアップデート
<code>test/add-post-tests</code>	テスト: 投稿関連のテスト追加

種類	用途	例
<code>feature/</code>	新しい機能の追加	<code>feature/add-bookmark</code>
<code>fix/</code>	バグの修正	<code>fix/broken-pagination</code>
<code>refactor/</code>	動作を変えずにコードを整理	<code>refactor/extract-utils</code>
<code>docs/</code>	ドキュメントの変更	<code>docs/api-usage</code>
<code>chore/</code>	設定変更、依存関係の更新	<code>chore/upgrade-next</code>
<code>test/</code>	テストの追加・修正	<code>test/auth-flow</code>

ブランチ操作の実践

```
# === ブランチの基本操作 ===

# 現在のブランチを確認
git branch
# 実行結果の例:
# * main                ← * がついているのが現在のブランチ
#   feature/add-like-button

# main ブランチから新しいブランチを作成して切り替え
git checkout -b feature/add-like-button
# 実行結果の例:
# Switched to a new branch 'feature/add-like-button'
# (「feature/add-like-button」ブランチに切り替わった)

# 作業を行い、変更をコミット
git add .
git commit -m "いいねボタンのUI作成"
# 実行結果の例:
# [feature/add-like-button abc1234] いいねボタンのUI作成
# 2 files changed, 45 insertions(+)

# さらに作業を続けてコミット
git add .
git commit -m "いいねAPIの実装"

# GitHub にプッシュ（初回は -u オプションをつける）
git push -u origin feature/add-like-button
# 実行結果の例:
# Enumerating objects: 10, done.
# ...
# To github.com:username/bonsai-sns-project.git
# * [new branch]      feature/add-like-button → feature/add-like-button

# === ブランチの切り替え ===

# main ブランチに戻る
git checkout main

# 別のブランチに切り替え
git checkout feature/add-like-button

# === main の最新を取り込む ===
# 長期間ブランチで作業していると、main に新しい変更が入っていることがある
# その場合、main の変更を自分のブランチに取り込む

# まず main の最新を取得
git checkout main
git pull origin main
```

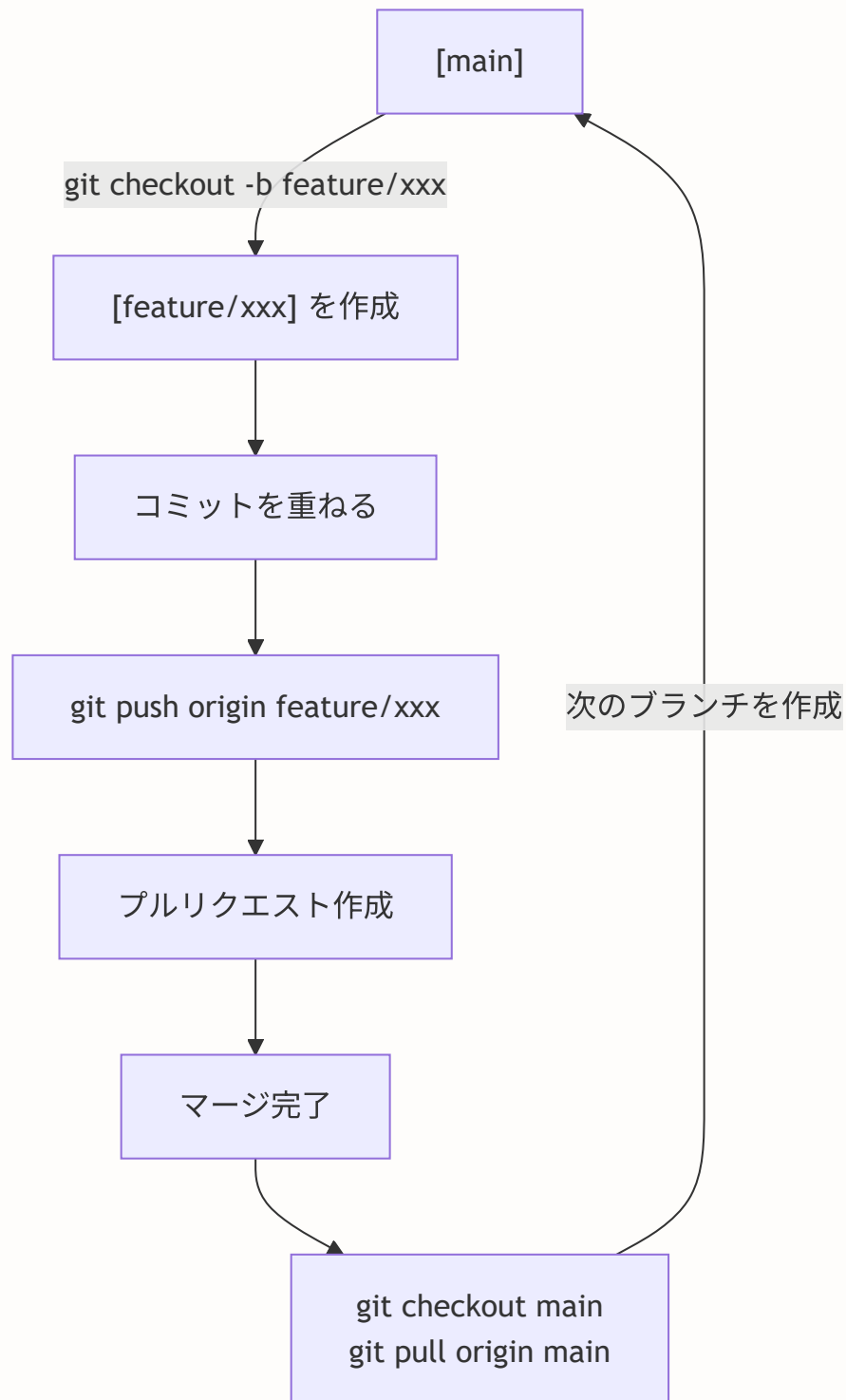
自分のブランチに戻って main の変更をマージ

```
git checkout feature/add-like-button
```

```
git merge main
```

コンフリクト（競合）が発生した場合は手動で解決する

（コンフリクトの解決方法は後述）



コンフリクト（競合）の解決

複数人が同じファイルの同じ箇所を変更した場合、「コンフリクト（競合）」が発生します。Git はどちらの変更を採用すべきか判断できないため、手動で解決する必要があります。

コンフリクトが発生した場合のファイル表示例：

```
<<<<<< HEAD
  <button className="text-red-500">いいね</button>
=====
  <button className="text-blue-500">お気に入り</button>
>>>>>> feature/rename-like-button
```

意味：

```
<<<<<< HEAD      ← 現在のブランチの変更（ここから）
あなたの変更内容
=====          ← 区切り線
相手の変更内容
>>>>>> ブランチ名 ← 相手のブランチの変更（ここまで）
```

解決方法：

1. どちらかの変更を残す（もう一方を削除）
2. 両方を組み合わせた新しいコードを書く
3. <<<<<<, =====, >>>>>> の行をすべて削除する
4. ファイルを保存する
5. `git add . && git commit -m "コンフリクトを解決"`

VS Code でのコンフリクト解決

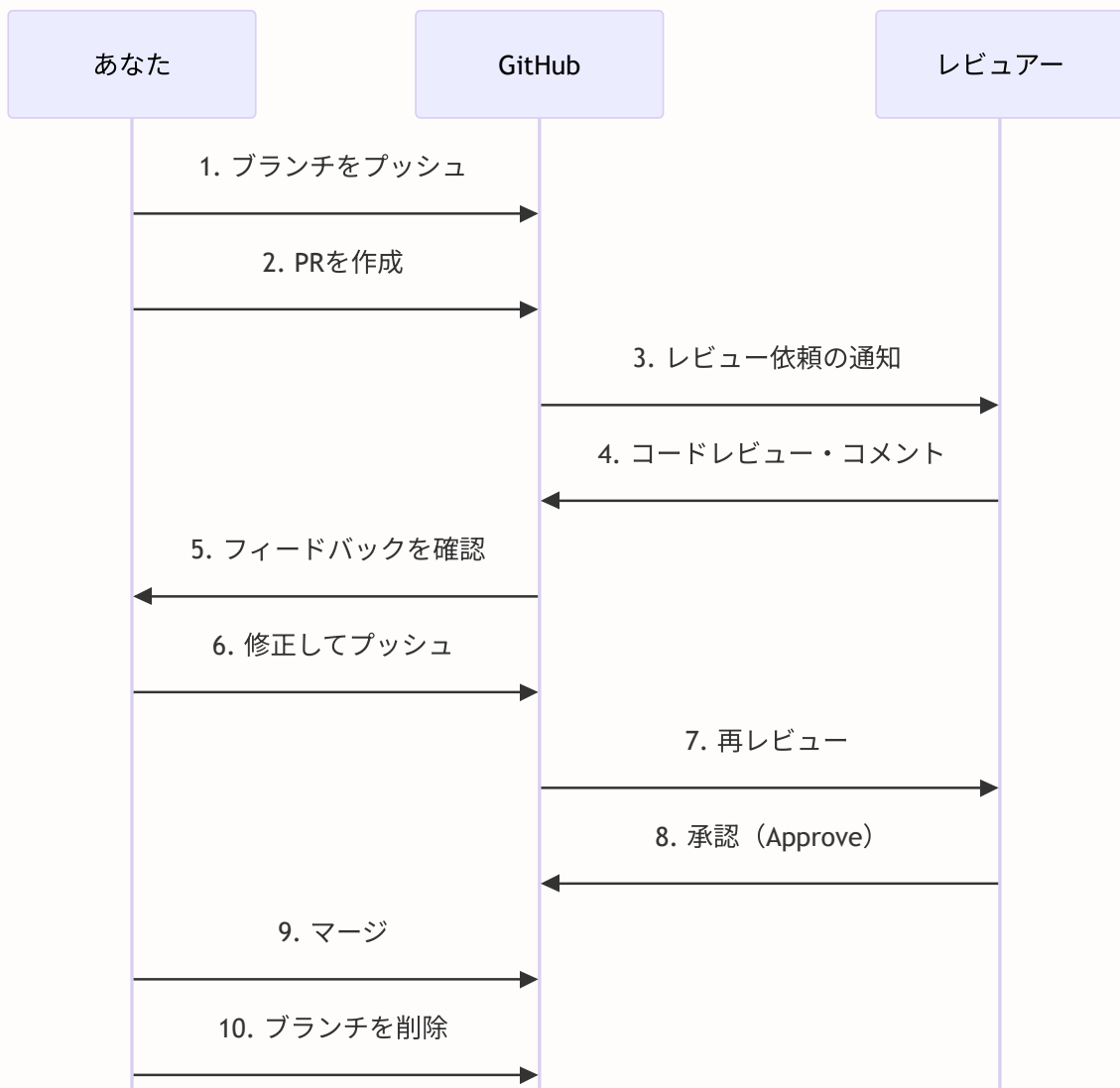
VS Code はコンフリクトを自動検出し、以下のボタンを表示してくれます：

- 「Accept Current Change」：自分の変更を採用
- 「Accept Incoming Change」：相手の変更を採用
- 「Accept Both Changes」：両方を残す
- 「Compare Changes」：変更を並べて比較

ボタンをクリックするだけで解決できるため、手動でマーカーを削除する必要はありません。

プルリクエスト（PR）の作成

プルリクエスト（Pull Request、略してPR）は、「自分のブランチの変更を main にマージしてください」という申請です。

**GitHub での PR 作成手順:**

1. GitHub のリポジトリページにアクセス
2. ブランチをプッシュした後に表示される「Compare & pull request」ボタンをクリック
 - または「Pull requests」タブ → 「New pull request」をクリック
3. 以下の情報を入力:

タイトル（1行で簡潔に）：
いいねボタンの追加

本文（テンプレートに沿って記載）：

概要

投稿に対する「いいね」機能を追加しました。

変更内容

- いいねボタンのUIコンポーネント（LikeButton.tsx）を作成
- いいねのServer Action（lib/actions/like.ts）を実装
- いいねの状態管理をReact Queryで実装
- いいねカウントの表示を追加

動作確認

- [] いいねボタンをクリックするといいねが追加される
- [] もう一度クリックするといいねが解除される
- [] いいね数が正しく表示される
- [] ログインしていない状態ではいいねできない

スクリーンショット

（ここに画面キャプチャを貼り付け）

4. 「Create pull request」をクリック

良い PR の書き方のポイント:

1. **タイトルは簡潔に**: 「何をしたか」が一目で分かるように
2. **概要を書く**: なぜこの変更が必要なのかを説明
3. **変更内容を列挙**: 何を変えたかを箇条書きで
4. **動作確認チェックリスト**: テストした項目をリストアップ
5. **スクリーンショットを添付**: UI変更がある場合は必須
6. **PR は小さく保つ**: 1つの PR で1つの機能。巨大な PR はレビューが大変

コードレビューの基本

コードレビューは、他の開発者が書いたコードを確認し、品質を保つための重要なプロセスです。

レビューを受ける側のマナー:

✓ やるべきこと:

- セルフレビュー: PR を出す前に自分で差分を確認する
- テストを書く: 新しいコードにはテストを追加する
- コメントに丁寧に返信する
- 指摘を素直に受け入れる (指摘 = 攻撃ではない)
- 修正したらコメントで報告する

✗ やってはいけないこと:

- テストなしで PR を出す
- 指摘を無視する
- 「動いているからOK」と主張する
- 巨大な PR を出す (500行以上の変更は分割を検討)

レビューする側のマナー:**✓ やるべきこと:**

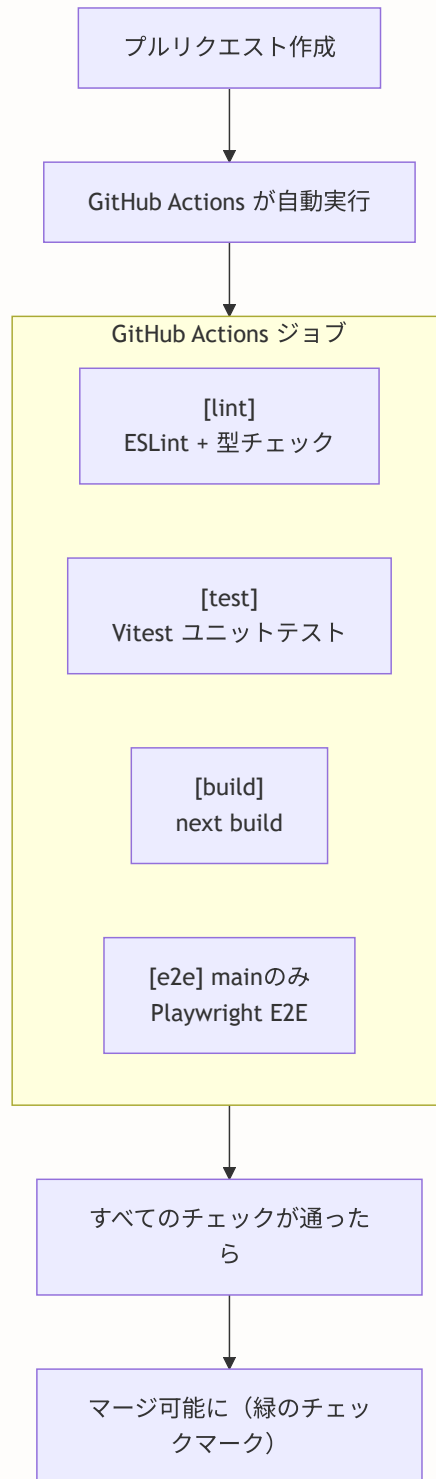
- 建設的なコメントを書く (「こうすると良いかも」)
- なぜそう思うかの理由を添える
- 良いコードには褒めるコメントを残す
- 必須の修正と提案を区別する (nit: は軽微な指摘)

✗ やってはいけないこと:

- 人格を否定するようなコメント
- 理由なく「ダメ」と言う
- 些細な点に固執して全体を止める
- レビューを何日も放置する

CI/CD (継続的インテグレーション / 継続的デリバリー)

BON-LOG では GitHub Actions を使って、PR やプッシュ時に自動テストを実行します。

**CI が失敗した場合:**

GitHub の PR ページに赤い x マークが表示されます。「Details」をクリックすると、どのテストが失敗したかを確認できます。失敗を修正してプッシュすると、CI が再実行されます。

マージ後のクリーンアップ

PR がマージされたら、以下の手順でブランチを整理します。

```
# main ブランチに切り替え
git checkout main

# main の最新を取得
git pull origin main

# マージ済みのローカルブランチを削除
git branch -d feature/add-like-button
# 実行結果の例:
# Deleted branch feature/add-like-button (was abc1234).

# リモートのブランチも削除 (GitHub上で自動削除設定がない場合)
git push origin --delete feature/add-like-button
```

理解度チェック

- ☐ ブランチとは何か、なぜ使うのかを説明できますか？
- ☐ `git checkout -b` コマンドで新しいブランチを作成できますか？
- ☐ ブランチの命名規則（`feature/`, `fix/` 等）を理解していますか？
- ☐ プルリクエストの作成手順を説明できますか？
- ☐ コンフリクトが発生する原因と解決方法を理解していますか？
- ☐ CI/CD が何をするものか説明できますか？

1.16 VS Code 開発環境の最適化

このセクションで学ぶこと

- BON-LOG 開発に最適な VS Code 拡張機能の詳細設定
- 生産性を上げるワークスペース設定
- デバッグ設定の構築
- コードスニペットの活用
- マルチカーソル・一括編集のテクニック

BON-LOG 推奨拡張機能 -- 詳細ガイド

1.4節で基本の拡張機能を紹介しましたが、ここではさらに詳しい設定と 追加の拡張機能を紹介します。

必須拡張機能の詳細設定

ESLint (コード品質チェック)

拡張機能ID: `dbaeumer.vscode-eslint`

役割:

コードを書いている最中に、文法ミスや推奨されない書き方をリアルタイムで検出して波線で警告する。

例:

```
const x = 1;    ← x が使われていない場合、黄色の波線が表示される
let y: any = 2; ← any 型の使用に警告が表示される
```

設定のポイント:

保存時に自動修正を有効にすると、ESLint が自動で修正できる問題 (import の並び替え、不要なセミコロンの削除等) が自動的に修正される。

Prettier (コード自動整形)

拡張機能ID: `esbenp.prettier-vscode`

役割:

コードのインデント、改行、クォーテーション (' vs ") 、セミコロンの有無などを自動的に統一する。

例 (保存前) :

```
const name="BON-LOG"
const users = [  "alice",
  "bob","charlie"  ]
```

例 (保存後 = Prettier が自動整形) :

```
const name = "BON-LOG";
const users = ["alice", "bob", "charlie"];
```

設定のポイント:

プロジェクトルートに `.prettierrc` ファイルがある場合、その設定が優先されます。チーム全員が同じフォーマットになります。

Tailwind CSS IntelliSense (クラス名補完)

拡張機能ID: `bradlc.vscode-tailwindcss`

役割:

Tailwind CSS のクラス名を入力途中で候補を表示し、
各クラスが実際に適用する CSS プロパティをホバーで表示する。

例:

```
<div className="bg-    ← ここで候補が表示される
  bg-red-500          → background-color: #ef4444
  bg-blue-500         → background-color: #3b82f6
  bg-green-500        → background-color: #22c55e
  ...
```

ホバーで確認:

"text-lg" にマウスを置くと

→ `font-size: 1.125rem; line-height: 1.75rem;` と表示される

設定のポイント:

`className` だけでなく、`clsx()` や `cn()` 内のクラス名も
補完が効くように設定できます。

追加推奨拡張機能

拡張機能	ID	用途	おすすめ度
Error Lens	<code>usernamehw.errorlens</code>	エラー・警告をコード行に直接表示	高
Auto Rename Tag	<code>formulahendry.auto-rename-tag</code>	HTMLタグの開始/終了を同時に変更	高
Path Intellisense	<code>christian-kohler.path-intellisense</code>	ファイルパスの自動補完	高
Material Icon Theme	<code>pkief.material-icon-theme</code>	ファイルアイコンを見やすく変更	中
Docker	<code>ms-azuretools.vscode-docker</code>	Dockerコンテナの管理をGUIで	中
Thunder Client	<code>rangav.vscode-thunder-client</code>	API テスト (Postman の代替)	中
GitHub Copilot	<code>github.copilot</code>	AIによるコード補完 (有料)	中
TODO Highlight	<code>wayou.vscode-todo-highlight</code>	TODO, FIXME コメントをハイライト	低
Bracket Pair Colorizer	VS Code 内蔵	括弧のペアを色分け (設定で有効化)	低

Error Lens の効果:

通常の VS Code:

```
const x = 1;
~~~
```

← 波線だけで、何が問題が分かりにくい

Error Lens 導入後:

```
const x = 1; // 'x' is declared but its value is never read. ts(6133)
```

← エラーメッセージが行末に直接表示される！

ワークスペース設定 (.vscode/settings.json)

プロジェクト固有の VS Code 設定は、`.vscode/settings.json` に記述します。この設定はプロジェクトを開いた全員に適用されます。

```
// .vscode/settings.json
{
  // === エディタ基本設定 ===
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },

  // === TypeScript 設定 ===
  "typescript.preferences.importModuleSpecifier": "non-relative",
  "typescript.tsdk": "node_modules/typescript/lib",
  "typescript.enablePromptUseWorkspaceTsdk": true,

  // === Tailwind CSS 設定 ===
  "tailwindCSS.experimental.classRegex": [
    ["clsx\\(([^)]*)\\)", "(?:'|\\\"|'|`)([']*)?(?:'|\\\"|'|`)"],
    ["cn\\(([^)]*)\\)", "(?:'|\\\"|'|`)([']*)?(?:'|\\\"|'|`)"],
    ["cva\\(([^)]*)\\)", "\\\"'`([^\\"'`]*).*?\\\"'`"]
  ],

  // === ファイル設定 ===
  "files.insertFinalNewline": true,
  "files.trimTrailingWhitespace": true,
  "files.associations": {
    "*.css": "tailwindcss"
  },

  // === 検索除外設定 ===
  "search.exclude": {
    "**/node_modules": true,
    "**/.next": true,
    "**/coverage": true,
    "**/.git": true
  },

  // === Prisma 設定 ===
  "[prisma]": {
    "editor.defaultFormatter": "Prisma.prisma"
  }
}
```

ユーザー設定 vs ワークスペース設定の違い:

ユーザー設定（個人の好み）

場所: VS Code のグローバル設定

影響: すべてのプロジェクトに適用

例: フォントサイズ、テーマ、キーバインド

ワークスペース設定（プロジェクトのルール）

場所: `.vscode/settings.json`

影響: このプロジェクトを開いた時のみ適用

例: フォーマッター、リンター、TypeScript 設定

ワークスペース設定はユーザー設定より優先されます。

推奨拡張機能の一括インストール

`.vscode/extensions.json` ファイルを作成すると、プロジェクトを開いた時に VS Code が推奨拡張機能のインストールを提案してくれます。

```
// .vscode/extensions.json
{
  "recommendations": [
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "bradlc.vscode-tailwindcss",
    "Prisma.prisma",
    "eamodio.gitlens",
    "usernamehw.errorlens",
    "formulahendry.auto-rename-tag",
    "christian-kohler.path-intellisense"
  ]
}
```

VS Code のデバッグ設定

Next.js アプリケーションをデバッグする際に役立つ設定です。

```
// .vscode/launch.json
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Next.js: デバッグ（サーバーサイド）",
      "type": "node",
      "request": "launch",
      "runtimeExecutable": "${workspaceFolder}/node_modules/.bin/next",
      "runtimeArgs": ["dev"],
      "env": {
        "NODE_OPTIONS": "--inspect"
      },
      "console": "integratedTerminal"
    },
    {
      "name": "Next.js: デバッグ（クライアントサイド）",
      "type": "chrome",
      "request": "launch",
      "url": "http://localhost:3000"
    }
  ]
}
```

デバッグの使い方:

1. VS Code の左サイドバーで「実行とデバッグ」アイコン（虫のマーク）をクリック
2. 上部のドロップダウンから設定を選択
3. 緑の再生ボタン（▶）をクリック
4. コード上でブレークポイント（赤い丸）を設定
→ その行で実行が一時停止し、変数の値を確認できる

ブレークポイントの設定方法:

行番号の左側をクリック → 赤い丸が表示される

再度クリックで解除

一時停止中にできること:

- 変数の値を確認（マウスホバー）
- ステップ実行（1行ずつ進める）
- コールスタック（関数の呼び出し順）を確認
- コンソールで式を評価

生産性を上げるキーボードショートカット（追加）

1.4節で基本的なショートカットを紹介しましたが、開発が進むにつれて以下のショートカットも覚えると効率が上がります。

操作	Windows	Mac	説明
シンボルの名前変更	F2	F2	変数名や関数名を一括で変更（安全なリネーム）
定義に移動	F12	F12	関数やコンポーネントの定義場所にジャンプ
参照を検索	Shift + F12	Shift + F12	その関数が使われている箇所を一覧表示
行を上下に移動	Alt + ↑/↓	Option + ↑/↓	カーソル行を上下に移動
行をコピーして下に挿入	Shift + Alt + ↓	Shift + Option + ↓	現在行を複製
同じ単語を全選択	Ctrl + Shift + L	Cmd + Shift + L	ファイル内の同じ単語をすべて選択して一括編集
サイドバーの表示切替	Ctrl + B	Cmd + B	サイドバーの表示/非表示
問題パネルを開く	Ctrl + Shift + M	Cmd + Shift + M	エラーと警告の一覧を表示

理解度チェック

- ☐ Error Lens 拡張機能の効果を説明できますか？
- ☐ ユーザー設定とワークスペース設定の違いを理解していますか？
- ☐ `.vscode/extensions.json` の役割を説明できますか？
- ☐ ブレークポイントを使ったデバッグの手順を説明できますか？
- ☐ **F2**（シンボルの名前変更）と **Ctrl + H**（検索置換）の違いが分かりますか？

1.17 トラブルシューティング完全ガイド

このセクションで学ぶこと

- Node.js・npm に関する問題の解決方法

- Docker・PostgreSQL に関する問題の解決方法
- Next.js の起動・ビルドエラーの解決方法
- Prisma・データベース接続の問題の解決方法
- Windows / Mac 固有の問題と対処法
- エラーメッセージの読み方と検索のコツ

エラーメッセージの読み方

プログラミングをしていると、必ずエラーに遭遇します。エラーメッセージは「何が問題か」を教えてくれる重要なヒントです。

エラーメッセージの例: `Error: Cannot find module '@/lib/db'`

部分	例	意味
エラーの種類	<code>Error</code>	エラーの分類
エラーの内容	<code>Cannot find module</code>	モジュールが見つからない
詳細	<code>'@/lib/db'</code>	どのモジュールが見つからないか

スタックトレースの例: `at Object.<anonymous> (/Users/yuya/project/app/page.tsx:3:1)`

部分	例	意味
スタックトレース	<code>at Object.<anonymous></code>	エラーに至る関数の呼び出し順
エラーが発生した場所	<code>/Users/yuya/project/app/page.tsx:3:1</code>	ファイル:行:列

読み方のコツ:

1. 最初の行 (Error: ...) が最も重要。何が問題かが書かれている
2. ファイルパスと行番号で、どこでエラーが起きたか特定する
3. スタックトレースは上から順に読む (上が直接の原因)
4. 全てを理解する必要はない。キーワードを拾ってGoogle検索する

Node.js ・ npm のトラブル

問題: `node --version` が認識されない

エラー例:

```
'node' is not recognized as an internal or external command
(Windowsの場合)
```

```
command not found: node
(Macの場合)
```

原因と解決方法:

原因1: Node.js がインストールされていない

→ Node.js 公式サイト (<https://nodejs.org/>) から LTS 版をインストール

原因2: PATH が通っていない (インストール直後)

→ ターミナルを閉じて開き直す

→ それでもダメなら PC を再起動

原因3: インストール先を変更してしまった

→ Node.js を再インストール (デフォルトの場所にインストール)

確認コマンド:

```
# Windows
```

```
where node
```

```
# 期待する出力: C:\Program Files\nodejs\node.exe
```

```
# Mac
```

```
which node
```

```
# 期待する出力: /usr/local/bin/node または /opt/homebrew/bin/node
```

問題: `npm install` でエラーが発生する

エラー例1: EACCES (権限エラー)

```
npm ERR! Error: EACCES: permission denied
```

解決方法 (Mac) :

```
# npm のグローバルインストール先の権限を変更
```

```
sudo chown -R $(whoami) $(npm config get prefix)/{lib/node_modules,bin,share}
```

```
# または nvm (Node Version Manager) を使用する (推奨)
```

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
```

```
nvm install --lts
```

```
nvm use --lts
```

エラー例2: ERESOLVE (依存関係の競合)

```
npm ERR! ERESOLVE unable to resolve dependency tree
```

解決方法:

```
# --legacy-peer-deps オプションで互換性チェックを緩和
```

```
npm install --legacy-peer-deps
```

```
# または node_modules と lock ファイルを削除して再インストール
```

```
rm -rf node_modules package-lock.json
```

```
npm install
```

エラー例3: ENOENT (ファイルが見つからない)

```
npm ERR! enoent ENOENT: no such file or directory, open 'package.json'
```

解決方法:

```
# package.json があるフォルダ (プロジェクトルート) にいるか確認
```

```
ls package.json
```

```
# または
```

```
dir package.json
```

```
# いない場合は正しいフォルダに移動
```

```
cd ~/Desktop/Bonsai/bonsai-sns-project
```

エラー例4: ネットワークエラー

```
npm ERR! network request to https://registry.npmjs.org/ ... failed
```

解決方法:

```
# インターネット接続を確認
```

```
# プロキシ環境の場合:
```

```
npm config set proxy http://proxy.example.com:8080
```

```
npm config set https-proxy http://proxy.example.com:8080
```

```
# レジストリを確認・リセット
```

```
npm config get registry
# https://registry.npmjs.org/ であることを確認
```

問題: Node.js のバージョンが古い

確認方法:

```
node --version
# v16.x.x や v18.x.x と表示される場合 → アップデートが必要
```

解決方法:

方法1: 公式サイトから最新 LTS 版をダウンロードして上書きインストール
<https://nodejs.org/>

方法2: nvm を使用（推奨。複数バージョンの切り替えが可能）

Mac の場合:

```
nvm install 20
nvm use 20
nvm alias default 20
```

Windows の場合:

nvm-windows をインストール: <https://github.com/coreybutler/nvm-windows>

```
nvm install 20
nvm use 20
```

Docker ・ PostgreSQL のトラブル

問題: Docker コマンドが認識されない

エラー例:

```
'docker' is not recognized as an internal or external command
command not found: docker
```

解決方法:

1. Docker Desktop がインストールされているか確認
2. Docker Desktop が起動しているか確認
→ タスクバー（Windows） / メニューバー（Mac）にクジラアイコンがあるか
3. クジラアイコンのアニメーションが止まるまで待つ（起動に1~2分かかる）
4. ターミナルを開き直す

問題: `docker compose up` でポートが使用中

エラー例:

```
Error response from daemon: Ports are not available:
exposing port TCP 0.0.0.0:5432 → 0.0.0.0:0:
listen tcp4 0.0.0.0:5432: bind: address already in use
```

原因: 5432番ポートが他のプログラムに使われている

解決方法:

手順1: 何が使っているか確認

```
# Windows
netstat -ano | findstr :5432
# 出力例: TCP    0.0.0.0:5432    0.0.0.0:0    LISTENING    1234
#                                           ^^^^ プロセスID
```

```
# Mac
lsof -i :5432
# 出力例: postgres 1234 ... TCP *:postgresql (LISTEN)
```

手順2: そのプロセスを停止

```
# ローカルにインストールされた PostgreSQL を停止する場合:
# Windows: サービス管理画面で postgresql を停止
# Mac: brew services stop postgresql
```

```
# 他の Docker コンテナが使っている場合:
docker ps
docker stop <コンテナ名>
```

手順3: それでもダメなら `docker-compose.yml` のポートを変更
ports:

```
- "5433:5432"    ← ホスト側のポートを 5433 に変更
```

```
# .env.local の DATABASE_URL も合わせて変更
DATABASE_URL="postgresql://postgres:postgres@localhost:5433/bonsai_sns"
```

問題: Docker のイメージダウンロードが遅い・失敗する

エラー例:

```
error pulling image configuration: ...  
net/http: TLS handshake timeout
```

解決方法:

1. インターネット接続を確認
2. Docker Desktop の設定で DNS を変更
Settings > Docker Engine > 以下を追加

```
{  
  "dns": ["8.8.8.8", "8.8.4.4"]  
}
```
3. Docker Desktop を再起動
4. 再度 `docker compose up -d postgres` を実行

問題: PostgreSQL コンテナが起動してもすぐ停止する

確認方法:

```
docker compose ps  
# STATUS が "Exited" になっている場合
```

```
docker compose logs postgres  
# エラーの原因がログに表示される
```

よくある原因と解決方法:

原因1: データが壊れている

```
docker compose down -v    ← ボリュームごと削除  
docker compose up -d postgres ← 再作成
```

原因2: メモリ不足

Docker Desktop の Settings > Resources でメモリを増やす
(最低 2GB、推奨 4GB)

原因3: ディスク容量不足

```
docker system prune -a    ← 未使用のイメージ等を削除
```

Next.js のトラブル

問題: `npm run dev` でビルドエラー

エラー例1: モジュールが見つからない

```
Module not found: Can't resolve '@/lib/db'
```

解決方法:

1. ファイルが存在するか確認

```
ls lib/db.ts
```

2. `tsconfig.json` の `paths` 設定を確認

`"@/*": ["./*"]` が設定されているか

3. TypeScript のキャッシュをクリア

```
rm -rf .next
```

```
npm run dev
```

エラー例2: 型エラー

```
Type error: Property 'xxx' does not exist on type 'yyy'
```

解決方法:

1. エラーメッセージのファイルパスと行番号を確認

2. VS Code でそのファイルを開き、赤い波線の箇所を修正

3. 型定義が足りない場合は、型を追加する

4. Prisma の型が更新されていない場合:

```
npx prisma generate
```

エラー例3: 構文エラー

```
SyntaxError: Unexpected token
```

解決方法:

1. エラーが示すファイルの該当行を確認

2. 括弧の閉じ忘れ、カンマの漏れ等がないか確認

3. VS Code のエラー表示（赤い波線）を参考に修正

問題: ブラウザで「This site can't be reached」

チェックリスト:

- [] 1. ターミナルで `npm run dev` が実行中か確認
 - "Ready in xxxms" と表示されているか
- [] 2. URL が正しいか確認
 - `http://localhost:3000` (`https` ではなく `http`)
- [] 3. ポート番号が正しいか確認
 - ターミナルに表示されている `Local:` の番号と一致するか
- [] 4. ファイアウォールがブロックしていないか
 - セキュリティソフトの設定を確認
- [] 5. 他のプログラムがポート3000を使っていないか
 - `npm run dev -- -p 3001` で別ポートを試す

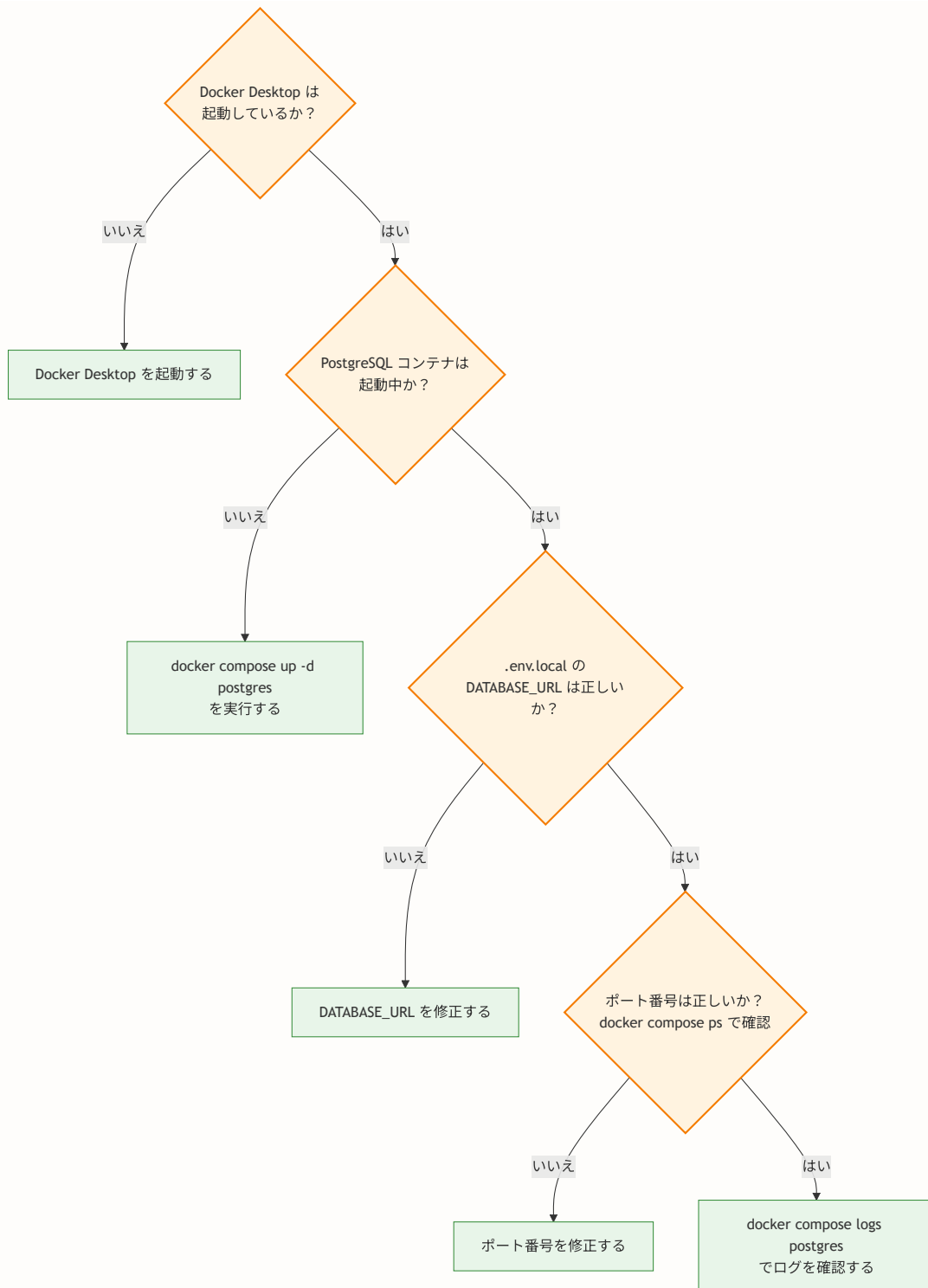
Prisma・データベース接続のトラブル

問題: `npx prisma db push` で接続エラー

エラー例:

```
Error: P1001: Can't reach database server at `localhost:5432`
```

接続エラーの原因特定フローチャート:



確認コマンド:


```
# Docker コンテナの状態確認
docker compose ps

# コンテナのログ確認
docker compose logs postgres

# 環境変数の確認 (DATABASE_URL が正しいか)
# .env.local を VS Code で開いて確認

# PostgreSQL に直接接続テスト
docker compose exec postgres psql -U postgres -d bonsai_sns -c "SELECT 1"
# 「1」 が返ってくれば接続成功
```

問題: `npx prisma generate` でエラー

エラー例:

```
Error: EPERM: operation not permitted
(Windows で特に発生しやすい)
```

解決方法:

```
# 1. node_modules を削除して再インストール
rm -rf node_modules
npm install

# 2. Prisma のキャッシュをクリア
rm -rf node_modules/.prisma
npx prisma generate

# 3. 管理者権限で実行 (Windows)
# PowerShell を管理者として実行してから:
npx prisma generate
```

Windows 固有の問題

問題: PowerShell の実行ポリシーでスクリプトが実行できない

エラー例:

```
... cannot be loaded because running scripts is disabled on this system
```

解決方法:

```
# PowerShell を管理者として実行し:  
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser  
  
# 確認  
Get-ExecutionPolicy  
# "RemoteSigned" と表示されればOK
```

問題: パスの区切り文字に関するエラー

問題:

Windows はパスの区切りに \ (バックスラッシュ) を使うが、
多くのツールは / (スラッシュ) を期待する

対処法:

1. Git Bash を使う (推奨)
 - / が使える Linux 風のターミナル
 - Git をインストールすると同梱される
2. VS Code のターミナルを Git Bash に変更
 - ターミナルの「+」横のドロップダウン > Select Default Profile > Git Bash
3. パスを書く際は / を使う (多くのツールで動作する)
 - `cd C:/Users/yuya/Desktop/Bonsai` ← OK

問題: `mkdir -p` が PowerShell で使えない

解決方法:

方法1: Git Bash を使う (推奨)
`mkdir -p app/(auth)/login` ← そのまま使える

方法2: PowerShell でのコマンド
`New-Item -ItemType Directory -Path "app/(auth)/login" -Force`

方法3: VS Code のエクスプローラーで手動作成
左サイドバーでフォルダを右クリック > 新しいフォルダー

Mac 固有の問題

問題: 「開発元を検証できないため開けません」

対処法:

1. システム設定 > プライバシーとセキュリティ
2. 「このまま開く」 ボタンをクリック
3. パスワードを入力して許可

または:

```
# ターミナルで Gatekeeper の警告を解除
xattr -cr /Applications/Docker.app
```

問題: M1/M2/M3/M4 Mac で Docker が遅い

対処法:

1. Docker Desktop の Settings > General
 - "Use Virtualization framework" にチェック
 - "VirtioFS" を選択 (ファイル共有が高速になる)
2. メモリ割り当てを調整
 - Settings > Resources > Memory
 - 最低 4GB、可能なら 6GB 以上
3. Docker のディスク使用量を管理
 - `docker system prune -a`
 - 使っていないイメージやコンテナを削除

エラー解決の一般的なアプローチ

問題に遭遇した際の体系的な解決方法を紹介します。

トラブル解決の5ステップ:

ステップ	やること	具体的なアクション
1	エラーメッセージを正確に読む	最初の1~2行に原因のヒントがある。ファイルパスと行番号を確認する
2	基本的なリセットを試す	ターミナルを閉じて開き直す。開発サーバーを再起動する (Ctrl + C → <code>npm run dev</code>)。 <code>.next</code> フォルダを削除する (<code>rm -rf .next</code>)
3	キャッシュをクリアする	npm のキャッシュ: <code>npm cache clean --force</code> 。node_modules の再作成: <code>rm -rf node_modules && npm install</code> 。Prisma のキャッシュ: <code>npx prisma generate</code>
4	エラーメッセージで検索する	Google で「エラーメッセージ」を検索。Stack Overflow の回答を参考にする。GitHub の Issues を確認する
5	最小限の再現を試す	問題が起きるコードだけを切り出す。新しいファイルで最小限のコードを書いて試す。問題の原因を絞り込む

理解度チェック

- ☐ エラーメッセージの読み方（種類、内容、ファイルパス）を理解していますか？
- ☐ `npm install` で権限エラーが出た場合の対処法を知っていますか？
- ☐ Docker のポート競合を解決する方法を説明できますか？
- ☐ Prisma の接続エラーの原因を特定するフローチャートを理解していますか？
- ☐ エラー解決の5ステップを説明できますか？

1.18 よくある質問（FAQ）

このセクションで学ぶこと

- 開発環境セットアップに関するよくある質問と回答
- 初心者が抱きやすい疑問の解消
- 学習を進める上でのアドバイス

全般的な質問

Q1: プログラミング未経験ですが、このチュートリアルについていけますか？

A: はい、大丈夫です。このチュートリアルは初心者を対象に書かれています。ただし、以下の心構えが大切です:

- **エラーは当たり前:** プロのエンジニアでもエラーに遭遇します。エラーが出ても焦らないでください
- **全部一度に覚えなくてOK:** 最初は「動く状態にする」ことが目標。理解は後からついてきます
- **手を動かすことが最重要:** 読むだけでなく、実際にコマンドを入力してください

Q2: どのくらいの時間で完成しますか？

A: 個人差がありますが、目安は以下の通りです:

学習ペースの目安:

第1章（環境構築）	: 1～2日
第2章（Web基礎）	: 2～3日
第3章（TypeScript）	: 3～5日
第4章（データベース）	: 2～3日
第5章（認証）	: 2～3日
第6章～（機能実装）	: 各章 2～5日

合計: 1～3ヶ月程度（1日1～2時間学習の場合）

- ※ プログラミング経験者は大幅に短縮されます
- ※ 途中で詰まっても気にしないでください

Q3: スマートフォンやタブレットで開発できますか？

A: 基本的にはPC（Windows / Mac）が必要です。スマートフォンやタブレットでは、Node.js や Docker を直接実行することが難しいためです。

代替案: GitHub Codespaces（ギットハブコードスペースズ）を使えば、ブラウザ上でクラウドの開発環境を利用できます。ただし、月に一定時間を超えると有料になります。

Q4: Windows と Mac のどちらが開発に向いていますか？

A: どちらでも開発可能です。このチュートリアルは両方に対応しています。各OSの特徴は以下の通りです:

Windows:

メリット: ゲーミングPCなど高性能な機種が豊富、価格が比較的安い

デメリット: パスの区切り文字、改行コードの違いでトラブルが起きやすい

おすすめ: WSL2 + Git Bash を使うことで、Mac/Linux に近い環境が得られる

Mac:

メリット: Unix ベースで開発ツールとの相性が良い、ターミナルが使いやすい

デメリット: 価格が高い

おすすめ: Homebrew をインストールして開発ツールを管理する

Q5: メモリ（RAM）は何GB必要ですか？

A: 最低8GB、推奨16GB以上です。

アプリケーション	メモリ使用量の目安
OS自体	2~4 GB
VS Code	0.5~1 GB
Docker Desktop	1~2 GB
Node.js (Next.js)	0.5~1 GB
Chrome（開発者ツール開）	0.5~1 GB
合計	4.5~9 GB

PCのメモリ	快適さ
8GB	動作するが、他のアプリを閉じる必要がある
16GB	快適に開発できる
32GB	余裕がある

技術に関する質問**Q6: Next.js と React の違いは何ですか？**

A: React は**UIライブラリ**、Next.js は React を含む**フレームワーク**です。

たとえるなら：

`React` = エンジン（車を動かす中核部品）

`Next.js` = 完成した車（エンジン + ボディ + タイヤ + ナビ + ...）

`React` だけでは：

- ルーティング（ページ遷移）を自分で設定する必要がある
- サーバーサイドレンダリングを自分で実装する必要がある
- 画像最適化を自分で実装する必要がある

`Next.js` を使えば：

- ファイルを置くだけでルーティングが自動設定される
- サーバーサイドレンダリングが標準で使える
- 画像最適化が組み込まれている
- APIルートが簡単に作れる

Q7: TypeScript は必須ですか？JavaScript だけではダメですか？

A: JavaScript でも動作しますが、TypeScript を**強く推奨**します。

TypeScript を使うメリット：

1. バグの早期発見

```
const age: number = "twenty"
// → エディタ上で即座にエラー表示（実行前に気づける）
```

2. コードの自動補完

```
user.    ← ここで「.」を打つと user の持つプロパティが候補表示される
→ user.name, user.email, user.id ...
```

3. リファクタリングの安全性

関数名や変数名を変更した時、使っている箇所がすべて自動更新される

4. ドキュメントの代わり

型定義がそのままコードの仕様書になる

```
function createUser(name: string, age: number): Promise<User>
→ 引数と戻り値が一目瞭然
```

Q8: Docker なしで開発できますか？

A: はい、PostgreSQL をローカルに直接インストールすることで Docker なしでも開発できます。ただし、Docker の使用を推奨します（セクション1.1の技術選定の背景を参照）。

```
# Docker を使わない場合の手順:

# 1. PostgreSQL を公式サイトからインストール
# https://www.postgresql.org/download/

# 2. データベースを作成
createdb bonsai_sns

# 3. .env.local の DATABASE_URL を設定
DATABASE_URL="postgresql://your-user:your-password@localhost:5432/bonsai_sns"

# ※ ユーザー名・パスワードはインストール時に設定したものを使用
```

Q9: このチュートリアルの後、何をすべきですか？

A: 次のセクション「1.19 学習ロードマップ」で詳しく解説しますが、大まかには以下のステップを推奨します:

1. まずチュートリアル全体を完走する
2. BON-LOG にオリジナルの機能を追加してみる
3. 自分のアイデアで新しいプロジェクトを作る
4. GitHub にポートフォリオとして公開する

Q10: エラーが出て解決できない場合、どこに質問すればいいですか？

A: 以下の場所で質問できます:

質問できる場所（おすすめ順）：

1. GitHub Issues
このプロジェクトの Issues に質問を投稿できます
[https://github.com/\[リポジトリURL\]/issues](https://github.com/[リポジトリURL]/issues)
2. Stack Overflow（スタックオーバーフロー）
世界最大のプログラミングQ&Aサイト
<https://stackoverflow.com/>
→ 英語で質問するとより多くの回答が得られる
3. Zenn（ゼン） / Qiita（キータ）
日本語のプログラミング記事・Q&Aサイト
→ 同じエラーで困っている人の記事が見つかることが多い
4. Discord コミュニティ
Next.js や React の公式 Discord サーバー
リアルタイムで質問・回答のやり取りができる

質問する際のポイント：

- エラーメッセージ全文を貼り付ける
- 何をしたらエラーが出たか（再現手順）を書く
- OS、Node.js のバージョンなどの環境情報を書く
- 自分で試した解決方法を書く

理解度チェック

- ☐ エラーが出た時に最初にすべきことを3つ挙げられますか？
- ☐ React と Next.js の違いを説明できますか？
- ☐ TypeScript を使うメリットを2つ以上挙げられますか？
- ☐ 質問する際に含めるべき情報を理解していますか？

1.19 学習ロードマップ

このセクションで学ぶこと

- BON-LOG チュートリアル全体の学習計画
- 各章で学ぶ技術スキルの全体像
- スキルの依存関係（何を先に学ぶべきか）

- 学習を継続するためのコツ
- チュートリアル完了後のキャリアパス

BON-LOG チュートリアル全体マップ

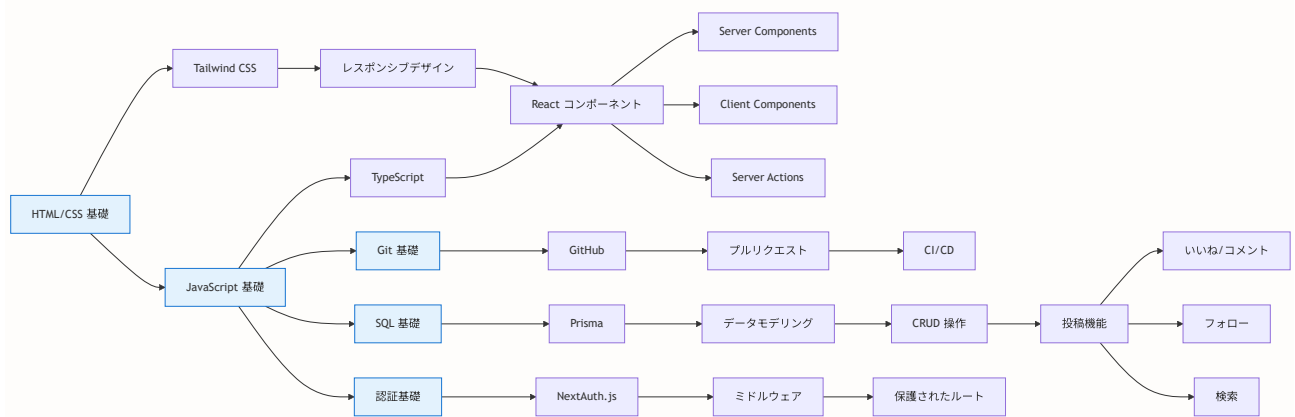
このチュートリアルは、段階的にスキルを積み上げる構成になっています。各章で学ぶスキルと、それが BON-LOG のどの機能に活かされるかを示します。

部	章	タイトル	学ぶ内容
第1部: 基礎固め	第1章	環境構築 ← 今ここ！	Node.js, VS Code, Git, Docker のセットアップ / 開発環境を「動く状態」にする
	第2章	Web開発の基礎	HTML, CSS, JavaScript の基本 / ブラウザでWebページが表示される仕組み
	第3章	TypeScript入門	型システムの基本 (string, number, boolean) / React コンポーネントの書き方 / Server Component と Client Component
	第4章	データベース設計	リレーショナルデータベースの基本 / Prisma スキーマの定義 / テーブルの作成とデータ操作 (CRUD)
第2部: 認証と基本機能	第5章	認証機能	NextAuth.js によるログイン・登録 / パスワードのハッシュ化 / セッション管理 (JWT) / 保護されたルートとミドルウェア
	第6章	UI/UXデザイン	Tailwind CSS によるスタイリング / shadcn/ui コンポーネントの活用 / レスポンシブデザイン (PC・スマホ対応) / 和風テーマの実装
	第7章	投稿機能	Server Actions によるデータ操作 / React Query によるデータ取得・キャッシュ / 無限スクロールの実装 / いいね・コメント・ブックマーク
第3部: 応用機能	第8章	画像アップロード	ファイルアップロードの仕組み / Cloudflare R2 への画像保存 / 画像の最適化 (リサイズ・圧縮)
	第9章	盆栽園マップ	Leaflet + OpenStreetMap による地図表示 / 位置情報の取得と表示 / レビュー・評価機能
	第10章	検索・通知	全文検索の実装 / Redis によるキャッシュとレート制限 / リアルタイム通知
第4部: 本番運用	第11章	設定・管理画面	ユーザー設定 / 管理者ダッシュボード
	第12章	メール送信	Resend によるメール送信 / メールテンプレートの作成
	第13章	決済機能	Stripe による有料会員機能 / Webhook の処理

	第 14 章	テスト	Vitest によるユニットテスト / Playwright による E2E テスト / CI/CD パイプライン
	第 15 章	デプロイ	Vercel へのデプロイ / 本番環境の設定 / パフォーマンス最適化

スキルの依存関係マップ

各スキルは以下のように関連しています。矢印の順に学べと、スムーズに理解が進みます。



学習を継続するための7つのコツ

コツ1: 小さなゴールを設定する

- ✗ 「アプリを完成させる」 (大きすぎる)
- ✓ 「今日はログインフォームを作る」 (具体的で達成しやすい)

コツ2: 毎日少しでも触る

- ✗ 週末に8時間まとめてやる (忘れる、疲れる)
- ✓ 毎日30分~1時間 (習慣になる、記憶が定着する)

コツ3: 完璧を求めない

- ✗ 「完全に理解してから次に進もう」 (永遠に進めない)
- ✓ 「だいたい分かったら次に進もう」 (後から戻って復習すればOK)

コツ4: エラーを恐れない

- ✗ エラーが出たら「自分にはプログラミングの才能がない」
- ✓ エラーが出たら「新しい学びのチャンス！」

コツ5: アウトプットする

- ✗ チュートリアルを読むだけ
- ✓ 学んだことをブログやSNSに書く
- ✓ コードを GitHub に公開する
- ✓ 学んだ技術で自分のアプリを作る

コツ6: コミュニティに参加する

- ✗ 一人で黙々と学ぶ (孤独で挫折しやすい)
- ✓ Discord、Twitter、勉強会に参加する
- ✓ 他の人のコードを読む
- ✓ 分からないことは質問する

コツ7: 休憩を大切にする

- ✗ 何時間も画面を見続ける
- ✓ 25分作業 → 5分休憩 (ポモドーロテクニック)
- ✓ 詰まったら散歩する (頭がリフレッシュされて解決策が浮かぶことがある)

チュートリアル完了後の道

BON-LOG チュートリアルを完了すると、以下のスキルが身についています。

身につくスキルセット:

カテゴリ	スキル
フロントエンド	HTML / CSS / JavaScript の基礎
	TypeScript で型安全なコードを書ける
	React コンポーネントを設計・実装できる
	Tailwind CSS でレスポンシブなUIを作れる
	Next.js の App Router で SPA/SSR アプリを構築できる
バックエンド	RESTful API の設計と実装
	Prisma + PostgreSQL でデータベース操作ができる
	NextAuth.js で認証機能を実装できる
	Server Actions でサーバーサイド処理を書ける
	外部サービス（Stripe, Cloudflare R2, Resend）と連携できる
インフラ・DevOps	Git / GitHub でバージョン管理ができる
	Docker で開発環境を構築できる
	Vercel にデプロイできる
	CI/CD パイプラインを構築できる
	本番環境の監視（Sentry）ができる
テスト	Vitest でユニットテストを書ける
	Playwright で E2E テストを書ける

次のステップの選択肢:

#	方向性	具体的なアクション
1	ポートフォリオを作る	BON-LOG を改良して自分のポートフォリオに / 新しいアイデアで独自のWebアプリを作る / GitHub に公開して実績にする
2	技術を深掘りする	React の高度なパターン（カスタムフック、Context、Suspense） / データベースの高度な設計（正規化、インデックス最適化） / パフォーマンス最適化（Core Web Vitals、キャッシュ戦略） / セキュリティ（OWASP Top 10、脆弱性対策）
3	関連技術を学ぶ	React Native（スマホアプリ開発） / GraphQL（APIの代替技術） / AWS / GCP / Azure（クラウドインフラ） / Kubernetes（コンテナオーケストレーション）
4	チーム開発に参加する	オープンソースプロジェクトにコントリビュート / ハッカソンに参加 / インターンシップや転職活動

週ごとの学習プラン（例）

以下は、1日1～2時間の学習を前提とした週ごとの学習プランの例です。あくまで目安ですので、自分のペースに合わせて調整してください。

第1週：環境構築と Web 基礎（第1章～第2章）

月：ツールのインストール（Node.js, VS Code, Git）
火：Docker のセットアップ、プロジェクト作成
水：環境変数の設定、動作確認
木：HTML の基礎（タグ、属性、構造）
金：CSS の基礎（セレクタ、プロパティ、レイアウト）
土：JavaScript の基礎（変数、関数、条件分岐）
日：復習 & 理解度チェック

第2週：TypeScript と React（第3章）

月：TypeScript の型システム（string, number, boolean）
火：型の応用（配列、オブジェクト、ユニオン型）
水：React コンポーネントの基本（JSX, props）
木：React の状態管理（useState, useEffect）
金：Server Component と Client Component
土：練習：シンプルなコンポーネントを作成
日：復習 & 理解度チェック

第3週：データベースと認証（第4章～第5章）

月：リレーショナルデータベースの概念
火：Prisma スキーマの定義
水：CRUD 操作（作成・読み取り・更新・削除）
木：NextAuth.js のセットアップ
金：ログイン・新規登録機能の実装
土：ミドルウェアによるルート保護
日：復習 & 理解度チェック

第4週：UIとメイン機能（第6章～第7章）

月：Tailwind CSS の基礎
火：shadcn/ui コンポーネントの活用
水：レスポンシブデザイン
木：投稿の作成（Server Actions）
金：タイムラインの表示（データ取得）
土：いいね・コメント機能
日：復習 & 理解度チェック

第5～8週：応用機能（第8章～第15章）

→ 各章を1～3日で進める
→ 最終的にデプロイまで完了

※ 上記は最短ペースの例です。

理解が追いつかない場合は、遠慮なくペースを落としてください。

大切なのは「続けること」です。

学習の記録を残す

学習の進捗を記録すると、モチベーションの維持に役立ちます。以下のような方法をおすすめします。

学習記録の方法:

方法	やること	効果
GitHub の草を生やす	毎日コミットすることで、GitHub のプロフィールに緑色の「草」（コントリビューショングラフ）が生える	連続記録が伸びると、休みたくなる効果がある
学習ノートを書く	Notion, メモ帳, 紙のノートなど何でもOK。「今日学んだこと」「つまづいたこと」「解決方法」を記録	後から見返すと、自分の成長が実感できる
SNS やブログで発信する	Zenn や Qiita に学習記事を書く。Twitter/X で #100DaysOfCode などのハッシュタグで投稿	アウトプットすることで理解が深まる
チェックリストで進捗管理	各章の「理解度チェック」を活用。全てにチェックがつくまで次に進まない	達成感が得られる

推奨学習リソース

BON-LOG チュートリアルと併せて活用できる学習リソースを紹介します。

リソース	種類	言語	おすすめポイント
Next.js 公式ドキュメント	公式ドキュメント	英語	最も正確で最新の情報源
React 公式チュートリアル	公式チュートリアル	英語	React の基本を体系的に学べる
TypeScript Handbook	公式ドキュメント	英語	TypeScript の網羅的な解説
Prisma ドキュメント	公式ドキュメント	英語	Prisma の全機能の解説
Zenn	技術ブログ	日本語	日本語の最新技術記事が豊富
Qiita	技術ブログ	日本語	日本語の技術記事。初心者向けも多い
MDN Web Docs	リファレンス	日本語	HTML/CSS/JavaScript の網羅的なリファレンス

理解度チェック

- ☐ チュートリアル全体の学習計画を把握できましたか？
- ☐ 各章で学ぶ主要な技術を3つ以上挙げられますか？

- ☐ 学習を継続するためのコツを3つ以上挙げられますか？
- ☐ チュートリアル完了後にどの方向に進みたいか、イメージできましたか？

第1章のまとめ

この章では、BON-LOG を開発するための環境構築を行いました。ここまですべて完了したことを振り返りましょう。

第1章で達成したこと:

達成項目	詳細
開発に必要な6つのツールをインストールした	Node.js (JavaScript実行環境) / npm (パッケージ管理ツール) / VS Code (コードエディタ) / Git (バージョン管理) / Docker Desktop (コンテナ実行環境) / Chrome (Webブラウザ)
ターミナルの基本操作を学んだ	cd (ディレクトリ移動) / ls / dir (ファイル一覧) / パスの概念 (絶対パス/相対パス)
Next.js プロジェクトを作成した	npx create-next-app で初期化 / npm run dev で開発サーバーを起動 / ブラウザで localhost:3000 にアクセス
PostgreSQL を Docker で起動した	docker-compose.yml を作成 / docker compose up -d postgres で起動 / docker compose ps で状態確認
環境変数を設定した	.env.local を作成 / DATABASE_URL を設定 / NEXT_PUBLIC_ の仕組みを理解
プロジェクト構造を理解した	app/ - ページとAPIルート / components/ - UIコンポーネント / lib/ - ビジネスロジック / prisma/ - データベーススキーマ
開発ワークフローを学んだ	Git ブランチの運用 (GitHub Flow) / プルリクエストの作成と流れ / CI/CD の概要
トラブルシューティングの方法を学んだ	エラーメッセージの読み方 / 一般的な問題の解決フロー / 質問の仕方

お疲れ様でした。環境構築は最も退屈で大変な作業ですが、ここを乗り越えれば、次の章からは実際にコードを書いていく楽しい段階に入ります。

次の章では、Webがどのように動いているかの基礎を学びます。HTML、CSS、JavaScriptの基本を理解してから、第3章でいよいよ TypeScript と React の世界に入っていきます。

次の章: [第2章: Web開発の基礎](#)